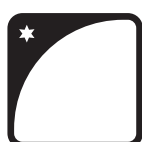
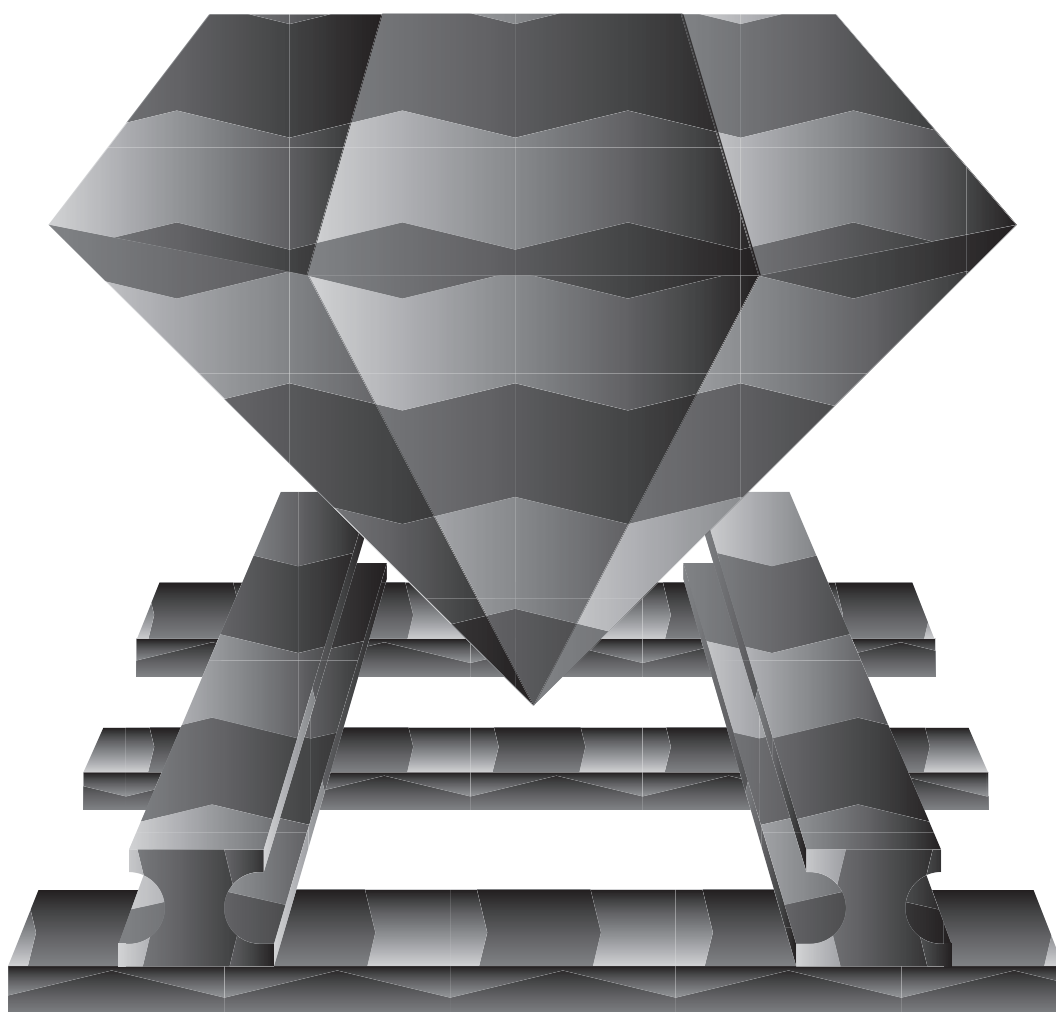


RR-71

Desenvolvimento Ágil para Web
2.0 com Ruby on Rails



Caelum
Ensino e Inovação

www.caelum.com.br

Conheça mais da Caelum.



Cursos Online

www.caelum.com.br/online



Casa do Código

Livros para o programador
www.casadocodigo.com.br



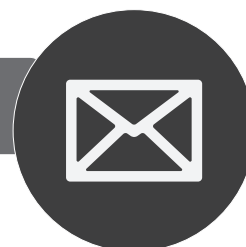
Blog Caelum

blog.caelum.com.br



Newsletter

www.caelum.com.br/newsletter



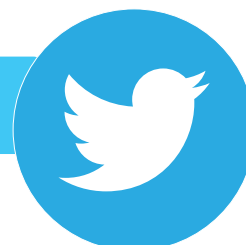
Facebook

www.facebook.com/caelumbr



Twitter

twitter.com/caelum



Conheça alguns de nossos cursos



FJ-11:

Java e Orientação a objetos



FJ-25:

Persistência com JPA2 e Hibernate



FJ-16:

Laboratório Java com Testes, XML e Design Patterns



FJ-26:

Laboratório Web com JSF2 e CDI



FJ-19:

Preparatório para Certificação de Programador Java



FJ-31:

Java EE avançado e Web Services



FJ-21:

Java para Desenvolvimento Web



FJ-91:

Arquitetura e Design de Projetos Java



RR-71:

Desenvolvimento Ágil para Web 2.0 com Ruby on Rails



RR-75:

Ruby e Rails avançados: lidando com problemas do dia a dia

Para mais informações e outros cursos, visite: caelum.com.br/cursos

- ✓ Mais de 8000 alunos treinados;
- ✓ Reconhecida nacionalmente;
- ✓ Conteúdos atualizados para o mercado e para sua carreira;
- ✓ Aulas com metodologia e didática cuidadosamente preparadas;
- ✓ Ativa participação nas comunidades Java, Rails e Scrum;
- ✓ Salas de aula bem equipadas;
- ✓ Instrutores qualificados e experientes;
- ✓ Apostilas disponíveis no site.



Sobre esta apostila

Esta apostila da Caelum visa ensinar de uma maneira elegante, mostrando apenas o que é necessário e quando é necessário, no momento certo, poupando o leitor de assuntos que não costumam ser de seu interesse em determinadas fases do aprendizado.

A Caelum espera que você aproveite esse material. Todos os comentários, críticas e sugestões serão muito bem-vindos.

Essa apostila é constantemente atualizada e disponibilizada no site da Caelum. Sempre consulte o site para novas versões e, ao invés de anexar o PDF para enviar a um amigo, indique o site para que ele possa sempre baixar as últimas versões. Você pode conferir o código de versão da apostila logo no final do índice.

Baixe sempre a versão mais nova em: www.caelum.com.br/apostilas

Esse material é parte integrante do treinamento Desenvolvimento Ágil para Web 2.0 com Ruby on Rails e distribuído gratuitamente exclusivamente pelo site da Caelum. Todos os direitos são reservados à Caelum. A distribuição, cópia, revenda e utilização para ministrar treinamentos são absolutamente vedadas. Para uso comercial deste material, por favor, consulte a Caelum previamente.

www.caelum.com.br

Sumário

1	Agilidade na Web	1
1.1	A agilidade	1
1.2	A comunidade Rails	2
1.3	Bibliografia	2
1.4	Tirando dúvidas	3
1.5	Para onde ir depois?	3
2	A linguagem Ruby	4
2.1	A história do Ruby e suas características	4
2.2	Instalação do interpretador	4
2.3	RubyGems	5
2.4	Bundler	6
2.5	Outras implementações de interpretadores Ruby	7
3	Ruby básico	10
3.1	Aprender Ruby?	10
3.2	Executando código Ruby no Terminal: IRB e arquivos .rb	10
3.3	Variáveis, Strings e Comentários	11
3.4	Variáveis e atribuições	12
3.5	Tipagem	13
3.6	Exercícios - Variáveis e Atribuições	13
3.7	String	14
3.8	Exercícios - Strings	16
3.9	Tipos e operações básicas	17
3.10	Exercícios - Tipos	18
3.11	Estruturas de controle	19
3.12	Exercícios - Estruturas de controle e Regexp	21
3.13	Desafios	22
4	Mais Ruby: classes, objetos e métodos	23
4.1	Mundo orientado a objetos	23
4.2	Métodos comuns	24
4.3	Definição de métodos	24
4.4	Exercícios - Métodos	25
4.5	Discussão: Enviando mensagens aos objetos	26
4.6	Classes	26
4.7	Exercícios - Classes	27
4.8	Desafio: Classes abertas	27
4.9	self	28

4.10	Desafio: self e o método puts	28
4.11	Atributos e propriedades: acessores e modificadores	29
4.12	Syntax Sugar: facilitando a sintaxe	30
4.13	Exercícios - Atributos e propriedades	31
4.14	Coleções	33
4.15	Exemplo: múltiplos parâmetros	34
4.16	Hashes	35
4.17	Exercícios - Arrays e Hashes	37
4.18	Blocos e Programação Funcional	39
4.19	Exercícios - Blocos	44
4.20	Para saber mais: Mais sobre blocos	44
4.21	Desafio: Usando blocos	45
4.22	Manipulando erros e exceptions	45
4.23	Exercício: Manipulando exceptions	46
4.24	Arquivos com código fonte ruby	47
4.25	Para saber mais: um pouco de IO	49
5	Metaprogramação e Padrões de Projeto	50
5.1	Métodos de Classe	50
5.2	Para saber mais: Singleton Classes	51
5.3	Exercícios - Ruby Object Model	52
5.4	Design Patterns: Singleton	53
5.5	Exercício: Design Pattern - Singleton	54
5.6	Convenções	55
5.7	Polimorfismo	56
5.8	Exercícios - Duck Typing	57
5.9	Design Patterns: Template Method	59
5.10	Exercício Opcional: Design Pattern - Template Method	60
5.11	Modulos	61
5.12	Design Patterns: Observer	62
5.13	Desafio: Design Pattern - Observer	64
5.14	Metaprogramação	64
5.15	Exercícios - Metaprogramação	67
6	Ruby on Rails	69
6.1	Ruby On Rails	69
6.2	Ambiente de Desenvolvimento	71
6.3	Criando um novo projeto Rails	72
6.4	Exercícios - Iniciando o projeto	73
6.5	Estrutura dos diretórios	75
6.6	O Banco de dados	76
6.7	Exercícios - Criando o banco de dados	76
6.8	A base da construção: Scaffold	78

6.9	Exercícios - Scaffold	78
6.10	Gerar as tabelas	78
6.11	Exercícios - Migrar tabela	79
6.12	Server	80
6.13	Exercícios - Iniciando o servidor	81
6.14	Documentação do Rails	81
6.15	Exercício opcional - Utilizando a documentação	83
7	Active Record	84
7.1	Motivação	84
7.2	Exercícios: Controle de Restaurantes	84
7.3	Modelo - O “M” do MVC	85
7.4	ActiveRecord	86
7.5	Rake	86
7.6	Criando Modelos	87
7.7	Migrations	88
7.8	Exercícios: Criando os modelos	89
7.9	Manipulando nossos modelos pelo console	91
7.10	Exercícios: Manipulando registros	94
7.11	Exercícios Opcionais	96
7.12	Finders	96
7.13	Exercícios: Buscas dinâmicas	97
7.14	Validações	98
7.15	Exercícios: Validações	100
7.16	Exercícios - Completando nosso modelo	101
7.17	O Modelo Qualificação	105
7.18	Exercícios - Criando o Modelo de Qualificação	105
7.19	Relacionamentos	109
7.20	Para Saber Mais: Auto-relacionamento	110
7.21	Para Saber Mais: Cache	111
7.22	Exercícios - Relacionamentos	111
7.23	Para Saber Mais - Eager Loading	115
7.24	Para Saber Mais - Named Scopes	116
7.25	Para Saber Mais - Modules	117
8	Controllers e Views	118
8.1	O “V” e o “C” do MVC	118
8.2	Hello World	118
8.3	Exercícios: Criando o controlador	119
8.4	Redirecionamento de Action e Action padrão	121
8.5	Trabalhando com a View: O ERB	122
8.6	Entendendo melhor o CRUD	124
8.7	Exercícios: Controlador do Restaurante	125

8.8	Helper	127
8.9	Exercícios: Utilizando helpers para criar as views	129
8.10	Partial	133
8.11	Exercícios: Customizando o cabeçalho	134
8.12	Layout	135
8.13	Exercícios: Criando o header	135
8.14	Outras formas de gerar a View	136
8.15	Filtros	137
9	Rotas e Rack	139
9.1	Rack	139
9.2	Exercícios - Testando o Rack	140
9.3	routes.rb	140
9.4	Pretty URLs	142
9.5	Named Routes	142
9.6	REST - resources	143
9.7	Actions extras em Resources	144
9.8	Diversas Representações	145
9.9	Para Saber Mais - Nested Resources	146
9.10	Rails e o Rack	146
9.11	Exercícios - Criando um rota para uma aplicação Rack	148
10	Completando o Sistema	149
10.1	Um pouco mais sobre o Scaffold	149
10.2	Exercícios	149
10.3	Selecionando Clientes e Restaurante no form de Qualificações	153
10.4	Exercícios	154
10.5	Exercícios Opcionais	158
10.6	Mais sobre os controllers	159
11	Calculations	162
11.1	Métodos	162
11.2	Média	162
11.3	Exercícios	163
12	Associações Polimórficas	165
12.1	Nosso problema	165
12.2	Alterando o banco de dados	166
12.3	Exercícios	167
13	Ajax com Rails	171
13.1	Adicionando comentários nas views	171
13.2	Criando as chamadas AJAX	171

13.3	Exercícios	175
13.4	Adicionando comentários	176
13.5	Exercícios	177
14	Algumas Gems Importantes	180
14.1	Engines	180
14.2	Exercícios - Título	181
14.3	File Uploads: Paperclip	182
14.4	Exercícios	182
14.5	Hpricot	184
14.6	Exercícios - Testando o Hpricot	185
15	Testes	186
15.1	O Porquê dos testes?	186
15.2	Test::Unit	186
15.3	Exercícios - Teste do modelo	189
15.4	Exercícios - Teste do controller	190
15.5	RSpec	191
15.6	Cucumber, o novo Story Runner	195
16	Apêndice A - Integrando Java e Ruby	199
16.1	O Projeto	199
16.2	Testando o JRuby	199
16.3	Exercícios	200
16.4	Compilando ruby para .class com Jruby	200
16.5	Rodando o .class do ruby na JVM	201
16.6	Importando um bytecode(.class) criado pelo jruby	201
16.7	Importando classes do Java para sua aplicação JRuby	201
16.8	Testando o JRuby com Swing	203
17	Apêndice B - Deployment	205
17.1	Webrick	205
17.2	CGI	206
17.3	FCGI - FastCGI	206
17.4	Lighttpd e Litespeed	206
17.5	Mongrel	207
17.6	Proxies Reversos	207
17.7	Phusion Passenger (mod_rails)	208
17.8	Ruby Enterprise Edition	208
17.9	Exercícios: Deploy com Apache e Passenger	209

CAPÍTULO 1

Agilidade na Web

“Não são os milagres que inclinam o realista para a fé. O verdadeiro realista, caso não creia, sempre encontrará em si força e capacidade para não acreditar no milagre, e se o milagre se apresenta diante dele como fato irrefutável, é mais fácil ele descrer de seus sentidos que admitir o fato”

– Fiodór Dostoiévski, em Irmãos Karamazov

1.1 A AGILIDADE

Quais são os problemas mais frequentes no desenvolvimento web? Seriam os problemas com AJAX? Escrever SQL? Tempo demais para gerar os CRUDs básicos?

Com tudo isso em mente, David Heinemeier Hansson, trabalhando na *37Signals*, começou a procurar uma linguagem de programação que pudesse utilizar para desenvolver os projetos de sua empresa. Mais ainda, criou um framework web para essa linguagem, que permitiria a ele escrever uma aplicação web de maneira simples e elegante.

Em agosto de 2010 foi lançada a versão final do Rails 3 e a Caelum ministra turmas com o material atualizado para esta versão desde então.

O que possibilita toda essa simplicidade do Rails são os recursos poderosos que Ruby oferece e que deram toda a simplicidade ao Rails. Esses recursos proporcionados pela linguagem Ruby são fundamentais de serem compreendidos por todos que desejam se tornar bons desenvolvedores Rails e por isso o começo desse curso foca bastante em apresentar as características da linguagem e seus diferenciais.

Um exemplo clássico da importância de conhecer mais a fundo a linguagem Ruby está em desvendar a “magia negra” por trás do Rails. Conceitos como metaprogramação, onde código é criado dinamicamente, são essenciais para o entendimento de qualquer sistema desenvolvido em Rails. É a meta programação que permite, por exemplo, que tenhamos classes extremamente enxutas e que garanta o relacionamento entre as

tabelas do banco de dados com nossas classes de modelo sem a necessidade de nenhuma linha de código, apenas usando de convenções.

Esse curso apresenta ainda os conceitos de programação funcional, uso de blocos, duck typing, enfim, tudo o que é necessário para a formação da base de conceitos que serão utilizados ao longo do curso e da vida como um desenvolvedor Rails.

1.2 A COMUNIDADE RAILS

A comunidade Rails é hoje uma das mais ativas e unidas do Brasil. Cerca de 10 eventos acontecem anualmente com o único propósito de difundir conhecimento e unir os desenvolvedores. Um exemplo dessa força é o Ruby Conf, maior evento de Ruby da América Latina, com presença dos maiores nomes nacionais e internacionais de Ruby on Rails, e a presença de uma track dedicada ao Ruby na QCon São Paulo.

Além dos eventos, diversos blogs sobre Rails tem ajudado diversos programadores a desvendar esse novo universo:

- <http://blog.caelum.com.br/> - Blog da Caelum
- <http://andersonleite.com.br/> - Anderson Leite
- <http://yehudakatz.com/> - Yehuda Katz
- <http://fabiokung.com/> - Fabio Kung
- <http://akitaonrails.com/> - Fábio Akita
- <http://blog.plataformatec.com.br/> - José Valim
- <http://nomedojogo.com/> - Carlos Brando
- <http://devblog.avdi.org/> - Avdi Grimm
- <http://blog.hasmanythrough.com/> - Josh Susser
- <http://rubyflow.com/> - Agrega conteúdo de vários sites

A Caelum aposta no Rails desde 2007, quando criamos o primeiro curso a respeito. E o ano de 2009 marcou o Ruby on Rails no Brasil, ano em que ele foi adotado por diversas empresas grandes e até mesmo órgãos do governo, como mencionado num post em nosso blog no começo do mesmo ano:

<http://blog.caelum.com.br/2009/01/19/2009-ano-do-ruby-on-rails-no-brasil/>

1.3 BIBLIOGRAFIA

- **Ruby on Rails: Coloque sua aplicação web nos trilhos - Vinícius Baggio Fuentes** Criado por um dos organizadores do GURU-SP, o livro mostra com exemplos práticos e explicações didáticas, como criar

uma aplicação Rails do começo ao fim. Mostra recursos importantes como o envio de e-mails, explica o MVC e ainda passa por diversos itens úteis do dia-dia, com deploy no Cloud. Você encontra o livro, inclusive em versão e-book em <http://www.casadocodigo.com.br>

- **Agile Web Development with Rails - Sam Ruby, Dave Thomas, David Heinemeier Hansson** Esse é o livro referência no aprendizado de Ruby on Rails, criado pelo autor do framework. Aqui, ele mostra através de um projeto, os principais conceitos e passos no desenvolvimento de uma aplicação completa. Existe uma versão em andamento para Rails 3.
- **Programming Ruby: The Pragmatic Programmers' Guide - Dave Thomas, Chad Fowler, Andy Hunt** Conhecido como “Pickaxe”, esse livro pode ser considerado a bíblia do programador Ruby. Cobre toda a especificação da linguagem e procura desvendar toda a “magia” do Ruby.
- **The Pragmatic Programmer: From Journeyman to Master - Andrew Hunt, David Thomas** As melhores práticas para ser um bom desenvolvedor: desde o uso de versionamento, ao bom uso do logging, debug, nomenclaturas, como consertar bugs, etc. Existe ainda um post no blog da Caelum sobre livros que todo desenvolvedor Rails deve ler: <http://blog.caelum.com.br/2009/08/25/a-trinca-de-ases-do-programador-rails/>

1.4 TIRANDO DÚVIDAS

Para tirar dúvidas dos exercícios, ou de Ruby e Rails em geral, recomendamos se inscrever na lista do GURU-SP (<http://groups.google.com/group/ruby-sp>), onde sua dúvida será respondida prontamente.

Também recomendamos duas outras listas:

- <http://groups.google.com/group/rubyonrails-talk>
- <http://groups.google.com/group/rails-br>

Fora isso, sinta-se à vontade para entrar em contato com seu instrutor e tirar todas as dúvidas que tiver durante o curso.

O fórum do GUJ.com.br, conceituado em java, possui também um subfórum de Rails:

<http://www.guj.com.br/>

1.5 PARA ONDE IR DEPOIS?

Além de fazer nossos cursos de Rails, você deve participar ativamente da comunidade. Ela é muito viva e ativa, e as novidades aparecem rapidamente. Se você ainda não tinha hábito de participar de fóruns, listas e blogs, essa é uma grande oportunidade.

Há ainda a possibilidade de participar de projetos opensource, e de você criar gems e plugins pro Rails que sejam úteis a toda comunidade.

CAPÍTULO 2

A linguagem Ruby

“Rails is the killer app for Ruby.”

– Yukihiro Matsumoto, Criador da linguagem Ruby

Nos primeiros capítulos conheceremos a poderosa linguagem de programação Ruby, base para completo entendimento do framework Ruby on Rails. Uma linguagem dinâmica e poderosa como o Ruby vai se mostrar útil não apenas para o uso no Rails, como também para outras tarefas e scripts do dia a dia.

Veremos a história, o contexto, suas versões e interpretadores e uma iniciação a sintaxe da linguagem.

2.1 A HISTÓRIA DO RUBY E SUAS CARACTERÍSTICAS

Ruby foi apresentada ao público pela primeira vez em 1995, pelo seu criador: **Yukihiro Matsumoto**, mundialmente conhecido como **Matz**. É uma linguagem orientada a objetos, com tipagem forte e dinâmica. Curiosamente é uma das únicas linguagens nascidas fora do eixo EUA - Europa que atingiram enorme sucesso comercial.

Uma de suas principais características é a expressividade que possui. Teve-se como objetivo desde o início que fosse uma linguagem muito simples de ler e ser entendida, para facilitar o desenvolvimento e manutenção de sistemas escritos com ela.

Ruby é uma linguagem interpretada e, como tal, necessita da instalação de um interpretador em sua máquina antes de executar algum programa.

2.2 INSTALAÇÃO DO INTERPRETADOR

Antes da linguagem Ruby se tornar popular, existia apenas um interpretador disponível: o escrito pelo próprio Matz, em C. É um interpretador simples, sem nenhum gerenciamento de memória muito complexo,

nem características modernas de interpretadores como a compilação em tempo de execução (conhecida como JIT). Hoje a versão mais difundida é a 1.9, também conhecida como **YARV** (Yet Another Ruby VM), já baseada em uma máquina virtual com recursos mais avançados.

A maioria das distribuições Linux possuem o pacote de uma das última versões estáveis pronto para ser instalado. O exemplo mais comum é o de instalação para o Ubuntu:

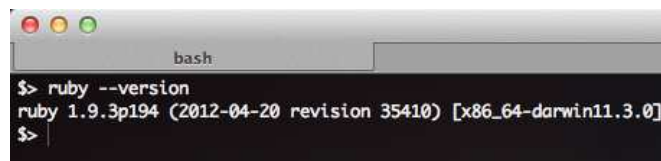
```
sudo apt-get install ruby1.9.1 ruby1.9.1-dev
```

Apesar de existirem soluções prontas para a instalação do Ruby em diversas plataformas (*one-click-installers* e até mesmo gerenciadores de versões de Ruby), sempre é possível baixá-lo pelo site oficial:

<http://ruby-lang.org>

Após a instalação, não deixe de conferir se o interpretador está disponível na sua variável de ambiente *PATH*:

```
$ ruby --version
ruby 1.9.3p194 (2012-04-20 revision 35410) [x86_64-darwin11.3.0]
```



2.3 RUBYGEMS

O Ruby possui um gerenciador de pacotes bastante avançado, flexível e eficiente: **RubyGems**. As *gems* podem ser vistas como bibliotecas reutilizáveis de código Ruby, que podem até conter algum código nativo (em C, Java, .Net). São análogos aos *jars* no ambiente Java, ou os *assemblies* do mundo .Net. RubyGems é um sistema gerenciador de pacotes comparável a qualquer um do mundo *NIX, como os *.debs* do *apt-get*, os *rpms* do *yum*, entre outros.

Para sermos capazes de instalar e utilizar as centenas de *gems* disponíveis, precisamos instalar além do interpretador Ruby, o RubyGems. Como exemplo simples temos o comando para instalação no Ubuntu:

```
sudo apt-get install rubygems1.9.1
```

MAIS SOBRE INSTALAÇÃO

No final da apostila apresentamos um apêndice exclusivo sobre a instalação completa do interpretador e ferramentas essenciais nas plataformas Linux, Macintosh e Windows.

Após instalado o RubyGems, podemos instalar os pacotes necessários para a criação de uma aplicação Rails, por exemplo, com o comando **rails new**:

```
rails new nome_da_aplicacao
```

2.4 BUNDLER

Ao desenvolver novas aplicações utilizando Ruby, notaremos que uma série de funcionalidades serão necessárias - ler e parsear JSON, fazer autenticação de usuário, entre outras coisas. A maioria dessas funcionalidades já foi implementada em alguma *gem*, e para usufruir desses recursos basta colocar a *gem* em nossa aplicação.

Nossos projetos podem precisar de uma ou mais *gems* para funcionar corretamente. Se quiséssemos compartilhar nosso projeto com a comunidade, como faríamos para fixar as *gems* necessárias para o funcionamento do projeto? Escreveríamos um arquivo de texto especificando quais *gems* o outro desenvolvedor deve instalar? Esse seria um processo manual? Há diversas formas de fazermos um bom controle de versão de código, mas como poderíamos, no mundo Ruby, controlar as *dependências* de *gems* necessárias ao nosso projeto?

Uma forma eficaz de controlar as dependências de nosso projeto Ruby é utilizar uma *gem* chamada *Bundler*. Instalar o *bundler* é bem simples. Basta rodar o comando:

```
gem install bundler
```

Com o Bundler, declaramos as dependências necessárias em um arquivo chamado *Gemfile*. Esse arquivo deverá conter, primeiramente, a fonte de onde o Bundler deve obter as *gems* e em seguida a declaração das dependências que usaremos no projeto.

```
source "http://rubygems.org"
```

```
gem "rails"  
gem "devise"
```

Note que a declaração *source* indica de onde o Bundler obterá as *gems* (no caso mais comum, de <http://rubygems.org>) e a declaração *gem* indica a *gem* declarada como dependência.

Podemos então rodar o comando `bundle` (atalho para o comando `bundle install`) para obter as *gems* necessárias para o nosso projeto. Note que ao estruturar a declaração das *gems* dessa forma não declaramos versões específicas para as *gems*, portanto a versão mais recente de cada uma delas será baixada para nosso repositório de *gems* local. Poderíamos fazer o seguinte:

```
gem "rails", "3.1.0"
```

Dessa forma explicitaríamos o uso da versão 3.1.0 da *gem rails*. Uma outra forma válida seria:

```
gem "rails", "~> 3.1.0"
```

Assim obteríamos a gem em uma versão maior ou igual à versão 3.1.0, mas menor que a versão 3.2.0.

Ao rodar o comando `bundle` será gerado um novo arquivo chamado *Gemfile.lock*, que especifica todas as gems obtidas para aquele Gemfile e sua respectiva versão baixada. O *Gemfile.lock* é uma boa alternativa para congelar as versões das gems a serem utilizadas, uma vez que ao rodarmos o comando `bundle` sobre a presença de um *Gemfile.lock*, as versões presentes nesse arquivo serão utilizadas para especificar as gems a serem baixadas.

2.5 OUTRAS IMPLEMENTAÇÕES DE INTERPRETADORES RUBY

Com a popularização da linguagem Ruby, principalmente após o surgimento do Ruby on Rails, implementações alternativas da linguagem começaram a surgir. A maioria delas segue uma tendência natural de serem baseados em uma Máquina Virtual ao invés de serem interpretadores simples. Algumas implementações possuem até compiladores completos, que transformam o código Ruby em alguma linguagem intermediária a ser interpretada por uma máquina virtual.

A principal vantagem das máquinas virtuais é facilitar o suporte em diferentes plataformas. Além disso, ter código intermediário permite otimização do código em tempo de execução, feito através da **JIT**.

JRUBY

JRuby foi a primeira implementação alternativa completa da versão 1.8.6 do Ruby e é a principal implementação da linguagem Java para a JVM. Com o tempo ganhou compatibilidade com as versões 1.8.7 e 1.9.2 na mesma implementação.

Como roda na JVM, não é um simples interpretador, já que também opera nos modos de compilação AOT (*Ahead Of Time*) e JIT (*Just In Time*), além do modo interpretador tradicional *Tree Walker*.

Uma de suas principais vantagens é a interoperabilidade com código Java existente, além de aproveitar todas as vantagens de uma das plataformas de execução de código mais maduras (Garbage Collector, Threads nativas, entre outras).

Muitas empresas apoiam o projeto, além da própria Oracle (após ter adquirido a Sun Microsystems), outras empresas de expressão como IBM, Borland e ThoughtWorks mantém alguns projetos na plataforma e algumas delas têm funcionários dedicados ao projeto.

<http://jruby.org>

IRONRUBY

A comunidade .Net também não ignora o sucesso da linguagem e patrocina o projeto **IronRuby**, mantido pela própria *Microsoft*. IronRuby foi um dos primeiros projetos verdadeiramente de código aberto dentro da

Microsoft.

<http://ironruby.net>

RUBINIUS

Criada por Evan Phoenix, **Rubinius** é um dos projetos que tem recebido mais atenção da comunidade Ruby, por ter o objetivo de criar a implementação de Ruby com a maior parte possível do código em Ruby. Além disso, trouxe ideias de máquinas virtuais do SmallTalk, possuindo um conjunto de instruções (bytecode) próprio e implementada em C/C++.

<http://rubini.us>

RUBYSPEC

O projeto Rubinius possui uma quantidade de testes enorme, escritos em Ruby, o que incentivou a iniciativa de especificar a linguagem Ruby. O projeto RubySpec (<http://rubyspec.org/>) é um acordo entre os vários implementadores da linguagem Ruby para especificar as características da linguagem Ruby e seu comportamento, através de código executável, que funciona como um **TCK** (*Test Compatibility Kit*).

RubySpec tem origem na suíte de testes de unidade do projeto Rubinius, escritos com uma versão mínima do RSpec, conhecida como **MSpec**. O RSpec é uma ferramenta para descrição de especificações no estilo pregado pelo *Behavior Driven Development*.

Avi Bryant durante seu *keynote* na **RailsConf de 2007** lançou um desafio à comunidade:

*"I'm from the future, I know how this story ends. All the people who are saying you can't implement Ruby on a fast virtual machine are wrong. That machine already exists today, it's called **Gemstone**, and it could certainly be adapted to Ruby. It runs Smalltalk, and Ruby essentially is Smalltalk. So adapting it to run Ruby is absolutely within the realm of the possible."*

Ruby e Smalltalk são parecidos, então Avi basicamente pergunta: por que não criar máquinas virtuais para Ruby aproveitando toda a tecnologia de máquinas virtuais para SmallTalk, que já têm bastante maturidade e estão no mercado a tantos anos?

Integrantes da empresa Gemstone, que possui uma das máquinas virtuais para SmallTalk mais famosas (Gemstone/S), estavam na plateia e chamaram o Avi Bryant para provar que isto era possível.

Na RailsConf de 2008, o resultado foi que a Gemstone apresentou o produto que estão desenvolvendo, conhecido como **Maglev**. É uma máquina virtual para Ruby, baseada na existente para Smalltalk. As linguagens são tão parecidas que apenas poucas instruções novas tiveram de ser inseridas na nova máquina virtual.

Os números apresentados são surpreendentes. Com tão pouco tempo de desenvolvimento, conseguiram apresentar um ganho de até 30x de performance em alguns micro benchmarks. Um dos testes mais conhecidos sobre a performance de interpretadores e máquinas virtuais Ruby feito por Antonio Cangiano em 2010 - **The Great Ruby Shootout** (<http://programmingzen.com/2010/07/19/the-great-ruby-shootout-july-2010/>)

- mostra que apesar de destacar-se em alguns testes, o Maglev mostra-se muito mais lento em alguns outros, além do alto consumo de memória.

RUBY ENTERPRISE EDITION

Para melhorar a performance de aplicações Rails e diminuir a quantidade de memória utilizada, **Ninh Bui**, **Hongli Lai** e **Tinco Andringa** (da Phusion) modificaram o interpretador Ruby e lançaram com o nome de **Ruby Enterprise Edition**.

As principais modificações no REE foram no comportamento do *Garbage Collector*, fazendo com que funcione com o recurso de *Copy on Write* disponível na maioria dos sistemas operacionais baseados em UNIX (Linux, Solaris, etc.).

Outra importante modificação foi na alocação de memória do interpretador, com o uso de bibliotecas famosas como `tcmalloc`. Os desenvolvedores da Phusion ofereceram as modificações (*patches*) para entrar na implementação oficial do Ruby.

A implementação oficial do Ruby versão 1.9, codinome *YARV*, adicionou algumas novas construções a linguagem em si, mas também resolveu muitos dos problemas antes endereçados pela *REE*. Além disso, o suporte ao Ruby 1.8 começou a se tornar mais escasso, o que é um movimento natural relacionado a evolução da linguagem.

Como consequência de tudo isso, a própria *Phusion*, mantenedora do *REE*, anunciou o fim da manutenção do projeto. Caso queira ler mais a respeito do assunto, visite o anúncio oficial em: <http://blog.phusion.nl/2012/02/21/ruby-enterprise-edition-1-8-7-2012-02-released-end-of-life-imminent/>.

CAPÍTULO 3

Ruby básico

“Pensa como pensam os sábios, mas fala como falam as pessoas simples”

– Aristóteles

3.1 APRENDER RUBY?

Ruby on Rails é escrito em Ruby, e embora seja possível fazer aplicações inteiras sem ter um conhecimento razoável sobre Ruby, com o tempo surge a necessidade de entender mais profundamente determinado comportamento da linguagem e qual sua influência na maneira como o Rails trata determinada parte das nossas aplicações.

Conhecer bem a linguagem determinará a diferença entre *usar o Rails* e *conhecer o Rails*.

3.2 EXECUTANDO CÓDIGO RUBY NO TERMINAL: IRB E ARQUIVOS .RB

O **IRB** é um dos principais recursos disponíveis aos programadores Ruby. Funciona como um console/terminal, e os comandos vão sendo interpretados ao mesmo tempo em que vão sendo inseridos, de forma interativa. O `irb` avalia cada linha inserida e já mostra o resultado imediatamente.

Todos os comandos apresentados neste capítulo podem ser executados dentro do `irb`, ou em arquivos `.rb`. Por exemplo, poderíamos criar um arquivo chamado **teste.rb** e executá-lo com o seguinte comando:

```
ruby teste.rb
```

3.3 VARIÁVEIS, STRINGS E COMENTÁRIOS

Ao aprender uma nova linguagem de programação sempre nos deparamos com o famoso “Olá Mundo”. Podemos imprimir uma mensagem com ruby utilizando pelo menos 3 opções: **puts**, **print** e **p**.

```
puts "Olá Mundo"  
print "Olá Mundo"  
p "Olá Mundo"
```

Mas qual a diferença entre eles? O `puts` imprime o conteúdo e pula uma linha, o `print` imprime, mas não pula linha, já o `p` chama o método `inspect` do elemento.

COMENTÁRIOS

Comentários em Ruby podem ser de uma linha apenas:

```
# Imprime uma mensagem  
puts "Oi mundo"
```

Ou comentários de blocos:

```
=begin  
  Imprime uma mensagem  
=end  
puts "Oi mundo"
```

COMENTÁRIOS DE BLOCO

Nos comentários de bloco, tanto o “`=begin`” quanto o “`=end`” devem ficar no início da linha, na coluna 0 do arquivo. Os comentários de blocos não costumam ser muito usados Ruby, a própria documentação do Rails usa apenas comentários de linha.

RESULTADO DA EXECUÇÃO

Atenção para o modo de saída utilizado pelo Ruby e como poderemos identificar uma saída correta em nosso código:

- **Arquivo .rb:** Ao executarmos um código em um arquivo .rb, somente serão exibidas no terminal as saídas que nós especificamos com puts ou outro comando específico que veremos nos exemplos a seguir.
- **IRB:** Todo comando executado no IRB exibe ao menos uma linha que inicia com os caracteres =>. Essa linha indica o **retorno** da execução, como veremos nos exemplos a seguir. Caso utilizemos o puts no terminal, a execução resultará em mais de uma linha, sendo que a última, iniciando com => ainda representa o retorno da operação. A linha indicando o retorno nil significa que o código executado não tem um valor de retorno definido. Nesses casos sempre obteremos o valor nil que é um valor equivalente ao null do Java ou outras linguagens.

PALAVRAS RESERVADAS DO RUBY

alias and BEGIN begin break case class def defined do else elsif END end ensure false for if in module next nil not or redo rescue retry return self super then true undef unless until when while yield

3.4 VARIÁVEIS E ATRIBUIÇÕES

Um dos mais conceitos básicos em linguagens de programação é a declaração de variáveis, que é apenas uma associação entre um nome e um valor. Em Ruby, basta definirmos o nome da variável e atribuir um valor usando o sinal =:

```
ano = 1950
```

Mas qual o tipo dessa variável? A variável ano é do tipo Fixnum, um tipo do Ruby que representa números inteiros. Mas não informamos isso na declaração da variável ano, isso porque na linguagem Ruby não é necessária esse tipo de informação, já que o interpretador da linguagem infere o tipo da variável automaticamente durante a execução do código. Esta característica é conhecida como inferência de tipos.

GETS

O Ruby nos permite atribuir numa variável um valor digitado no console através do teclado, isso é feito através do método `%gets%`, mas com ele só poderemos atribuir Strings atribuir outros tipos adiante. Exemplo:

```
irb(main):001:0> nome = gets
José
=> "José\n"
```

O `\n` representa um caracter de nova linha. Esse é um caracter que seu teclado manda quando é pressionada a tecla Enter.

3.5 TIPAGEM

Por não ser necessária a declaração do tipo, muitos pensam que a linguagem Ruby é fracamente tipada e com isso o tipo não importaria para o interpretador. No Ruby os tipos são importantes sim para a linguagem, tornando-a fortemente tipada, além disso, o Ruby também é implicitamente e dinamicamente tipada, implicitamente tipada pois os tipos são inferidos pelo interpretador, não precisam ser declarados e dinamicamente tipada porque o Ruby permite que o tipo da variável possa ser alterado durante a execução do programa, exemplo:

```
idade = 27
idade = "27"
```

O .CLASS

Em Ruby temos a possibilidade de descobrir o tipo de uma variável utilizando o `.class`:

```
irb(main):001:0> num = 87
=> 87
irb(main):002:0> puts num.class
Fixnum
=> nil
```

3.6 EXERCÍCIOS - VARIÁVEIS E ATRIBUIÇÕES

- 1) Vamos fazer alguns testes com o Ruby, escrevendo o código e executando no terminal. O arquivo utilizado se chamará **restaurante_basico.rb**, pois nossa futura aplicação web será um sistema de qualificação de restaurantes.

- Crie um diretório no Desktop com o nome **ruby**
- Dentro deste diretório crie o arquivo **restaurante_basico.rb**

Ao final de cada exercício você pode rodar o arquivo no terminal com o comando **ruby restaurante_basico.rb**.

- 2) Vamos imprimir uma mensagem pedindo para o usuário digitar o nome do restaurante.

```
puts "Digite o nome do restaurante: "
```

- 3) Agora iremos guardar o nome do nosso restaurante em uma variável nome. Para isso usaremos o método **gets** para capturar o valor digitado no console.

```
puts "Digite o nome do restaurante"
nome = gets
```

- 4) Por fim vamos imprimir o nome do nosso restaurante.

```
puts "Digite o nome do restaurante"
nome = gets
print "Nome do restaurante: "
puts nome
```

3.7 STRING

Um tipo muito importante nos programas Ruby são os objetos do tipo String. As Strings literais em Ruby podem ser delimitadas por aspas simples ou aspas duplas além de outras formas especiais que veremos mais adiante.

```
irb(main):001:0> mensagem = "Olá Mundo"
=> "Olá Mundo"
irb(main):002:0> mensagem = 'Olá Mundo'
=> "Olá Mundo"
```

MUTABILIDADE

A principal característica das Strings em Ruby é que são mutáveis, diferente do Java, por exemplo.

```
irb(main):001:0> mensagem = "Bom dia, "
=> "Bom dia,"
irb(main):002:0> mensagem << " tudo bem?"
=> "Bom dia, tudo bem?"
irb(main):003:0> puts mensagem
Bom dia, tudo bem?
=> nil
```

O operador << é utilizado para a operação append de Strings, ou seja, para a concatenação de Strings em uma mesma instância. Já o operador + também concatena Strings mas não na mesma instância, isso quer dizer que o + gera novas Strings.

INTERPOLAÇÃO DE STRINGS

Lembra da nossa mensagem que fizemos no último exercício?

```
#Declaração da variável %%nome%%  
print "Nome do restaurante: "  
puts nome
```

Nós podemos simplificá-la, uma alternativa mais interessante para criar Strings com valor dinâmico é a interpolação:

```
#Declaração da variável %%nome%%  
print "Nome do restaurante: #{nome}"
```

Basta colocar dentro da String a variável entre as chaves, em #{ }. Mas fique atento, com Strings definidas com aspas simples não é possível fazer uso da interpolação, por isso prefira sempre o uso de String com aspas duplas.

Prefira sempre a interpolação ao invés da concatenação (+) ou do append (<<). É mais elegante e mais rápido.

O MÉTODO CAPITALIZE

Repare que no caso do restaurante o usuário pode digitar qualquer nome, começando com ou sem letra maiúscula. Mas seria elegante um restaurante com nome que comece com letra minúscula? Para isso temos um método chamado **capitalize**, sua função nada mais é do que colocar a primeira letra de uma String em caixa alta.

```
irb(main):001:0> nome = "fasano"  
=> "fasano"  
irb(main):002:0> puts nome.capitalize  
Fasano  
=> nil
```

A chamada do método **capitalize** ao invés de alterar a variável para guardar o valor “Fasano”, retorna uma nova String com o valor com a primeira letra em caixa alta. Você pode confirmar este comportamento imprimindo novamente o nome, logo após a chamada do método **capitalize**, repare que foi impresso exatamente o mesmo valor que definimos na declaração da variável.


```
nome = "fasano"  
puts nome.capitalize #Fasano  
puts nome #fasano
```

O método **capitalize** é uma função pura, porque não importa quantas vezes seja invocado, retornará sempre o mesmo valor e também não causa efeitos colaterais, ou seja, não altera um valor já calculado anteriormente.

O OPERADOR BANG (!)

Mas e se quisermos que a mudança realizada pelo `capitalize` seja definitiva? Para isso acrescentamos o caracter **!** no final do método.

```
nome = "fasano"  
puts nome.capitalize! #Fasano  
puts nome #Fasano
```

O caracter **!** é chamado de bang e deve ser usado com moderação, já mudar definitivamente o valor onde foi utilizado.

3.8 EXERCÍCIOS - STRINGS

- 1) Primeiramente, vamos alterar o modo como o nome do nosso restaurante é impresso. Abra o arquivo **restaurante_basico.rb** e passe a utilizar a concatenação.

```
print "Nome do restaurante: " + nome
```

- 2) Já deu uma melhorada, só que aprendemos uma maneira mais elegante de fazer concatenação, que é a interpolação, vamos passar a utilizá-la.

```
print "Nome do restaurante: #{nome}"
```

DEFAULT ENCODING

A partir de sua versão 1.9 o Ruby tem a capacidade de tratar de modo separado o encoding de arquivo (external encoding), encoding de conteúdo de arquivo (internal encoding), além do encoding aceito no IRB.

O encoding padrão é o US-ASCII ou ISO-8859-1, dependendo da plataforma, e nenhum dos dois tem suporte a caracteres acentuados e outros símbolos úteis. Outro encoding aceito é o UTF-8 (Unicode) que aceita os caracteres acentuados que precisamos para nossas mensagens em português.

Para configurar o internal encoding de nossos arquivos, sempre devemos adicionar uma linha de comentário especial na **primeira linha** dos arquivos **.rb**:

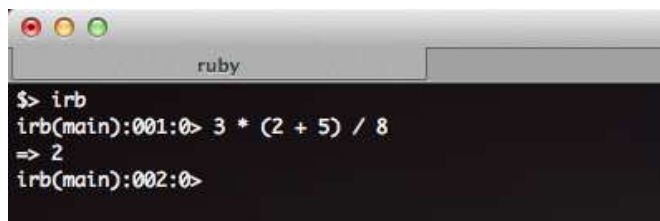
```
#coding:utf-8  
puts "Olá, mundo!"
```

Para o IRB podemos configurar o encoding no comando `irb` com a opção de encoding. O comando completo é `irb -E UTF-8:UTF-8`. Em algumas plataformas é possível utilizar a forma abreviada `irb -U`.

3.9 TIPOS E OPERAÇÕES BÁSICAS

Ruby permite avaliar expressões aritméticas tradicionais:

```
irb(main):001:0> 3*(2+5)/8  
=> 2
```



```
ruby  
$> irb  
irb(main):001:0> 3 * (2 + 5) / 8  
=> 2  
irb(main):002:0>
```

Estão disponíveis os operadores tradicionais:

- + - soma
- - - subtração
- / - divisão

- * - multiplicação
- ** - potência
- % - resto da divisão inteira

Os valores podem ainda ser atribuídos a variáveis:

```
irb(main):001:0> resultado = 4 * 4
=> 16
irb(main):002:0> puts resultado
16
=> nil
```

Ruby fornece uma maneira de trabalharmos com sequências de uma forma bem simples: (1..3) # range representando números de 1 a 3. ('a'..'z') # range representando letras minúsculas do alfabeto (0...5) # range representando números de 0 a 4.

Símbolos também são texto, como as Strings. Só que devem ser precedidos do carácter ': ', ao invés de aspas e pertencem à classe Symbol:

```
>> puts :simbolo
simbolo
=> nil
>> :simbolo.class
=> Symbol
```

As principais diferenças são:

- São imutáveis. Uma vez criado, um símbolo não pode ser alterado. Se precisarmos de um novo valor, precisa criar um novo objeto símbolo.
- São compartilhados. Símbolos com o mesmo valor, mesmo que em locais diferentes do código, compartilham do mesmo objeto. Isto é, qualquer lugar da aplicação que contenha, por exemplo, :writable, se referem a um mesmo objeto do tipo Symbol.

Por causa destas características, símbolos são análogos às Strings do Java. As Strings do Ruby estão mais para o StringBuilder do Java. Por serem imutáveis e compartilhados, objetos Symbol geralmente são usados como identificadores e para nomenclatura (labels). Durante o curso usaremos muito este recurso.

3.10 EXERCÍCIOS - TIPOS

1) Em Ruby tudo é objeto. Vamos fazer alguns testes curiosos e descobrir os tipos:

```
# Tipos
3.times{
  puts "0 3 é um objeto!"
}
puts 3.class
puts 33333333333333333333333333333333.class
```

2) Podemos fazer as operações básicas em ruby de forma simples:

```
# imprime conta
puts 3*(2+5)/8
```

3.11 ESTRUTURAS DE CONTROLE

Para utilizar estruturas de controle em ruby, precisamos antes conhecer os operadores booleanos, true e false. Os operadores booleanos aceitam quaisquer expressões aritméticas:

```
>> 3 > 2
=> true
>> 3+4-2 <= 3*2/4
=> false
```

Os operadores booleanos são: ==, >, <, >= e <=. Expressões booleanas podem ainda ser combinadas com os operadores && (and) e || (or).

O if do ruby aceita qualquer expressão booleana, no entanto, cada objeto em Ruby possui um “valor booleano”. Os únicos objetos de valor booleano false são o próprio false e o nil. Portanto, qualquer valor pode ser usado como argumento do if:

```
>> variavel = nil
=> nil
>> if(variavel)
>>   puts("so iria imprimir se variavel != null")
>> end
=> nil
>> if(3 == 3)
>>   puts("3 é 3")
>> end
3 é 3
=> nil
```

Teste também o switch:

```
def procura_sede_copa_do_mundo( ano )
  case ano
  when 1895..1993
    "Não lembro... :)"
  when 1994
    "Estados Unidos"
  when 1998
    "França"
  end
end

puts procura_sede_copa_do_mundo(1994)
```

O código acima funciona como uma série de if/elsif :

```
if 1994 == ano
  "Estados Unidos"
elsif 1998 == ano
  "França"
elsif 1895..1993 == ano
  "Não lembro... :)"
end
```

Utilizar um laço de repetições pode poupar muito trabalho. Em ruby o código é bem direto:

```
for i in (1..3)
  x = i
end
```

Ruby possui bom suporte a expressões regulares, fortemente influenciado pelo Perl. Expressões regulares literais são delimitadas por / (barra).

```
>> /rio/ =~ "são paulo"
=> nil
>> /paulo/ =~ "são paulo"
=> 4
```

O operador =~ faz a função de match e retorna a posição da String onde o padrão foi encontrado, ou nil caso a String não bata com a expressão regular. Expressões regulares aparecem com uma frequência maior em linguagens dinâmicas, e, também por sua sintaxe facilitada no Ruby, utilizaremos bastante.

RUBULAR.COM

Um site muito interessante para testar Expressões Regulares é o <http://rubular.com/>

MATCHDATA

Há também o método `match`, que retorna um objeto do tipo `MatchData`, ao invés da posição do match. O objeto retornado pelo método `match` contém diversas informações úteis sobre o resultado da expressão regular, como o valor de agrupamentos (captures) e posições (offset) em que a expressão regular bateu.

OPERADOR OU IGUAL

O operador `||=` atribui um valor apenas a variável esteja vazia. é muito utilizado para carregar valores de maneira “lazy”.

```
nome ||= "anonimo"
```

Nesse caso, se `nome` é nulo, ele será preenchido com `"anonimo"`.

3.12 EXERCÍCIOS - ESTRUTURAS DE CONTROLE E REGEXP

- 1) Nosso restaurante pode ter notas boas se a mesma for superior a 7, ou ruim caso contrário.

```
# estruturas de controle: if
nota = 10

if nota > 7
  puts "Boa nota!"
else
  puts "Nota ruim!"
end
```

Teste com notas diferentes e verifique as saídas no terminal.

- 2) Implemente um for em ruby:

```
# estruturas de controle: for
for i in (1..3)
  x = i
end
puts x
```

- 3) Teste o operado de expressões regulares “”

```
puts /rio/ =~ "são paulo" # nil
puts /paulo/ =~ "são paulo" # 4
```

Abra o site <http://rubular.com/> e teste outras expressões regulares!

- 4) O operador “||=” é considerado uma forma elegante de fazer um if. Verifique se uma variável já foi preenchida, caso não estiver, vamos atribuir um valor a ela.

```
restaurante ||= "Fogo de Chao"
puts restaurante
```

3.13 DESAFIOS

- 1) Sem tentar executar o código abaixo, responda: Ele funciona? Por que?

```
resultado = 10 + 4
texto = "O valor é " + resultado
puts(texto)
```

- 2) E o código abaixo, deveria funcionar? Por que?

```
puts(1+2)
```

- 3) Baseado na sua resposta da primeira questão, por que o código abaixo funciona?

```
resultado = 10 + 3
texto = "O valor é: #{resultado}"
```

- 4) Qual a saída deste código?

```
resultado = 10 ** 2
puts('o resultado é: #{resultado}')
```

- 5) **(Para Casa)** Pesquise sobre outras maneiras de criar Strings literais em Ruby.

- 6) Por que a comparação entre símbolos é muito mais rápida que entre Strings?

```
simbolo1 = :abc
simbolo2 = :abc
simbolo1 == simbolo2
# => true
```

```
texto1 = "abc"
texto2 = "abc"
texto1 == texto2
# => true
```

Mais Ruby: classes, objetos e métodos

“Uma imagem vale mais que mil palavras”

– Napoleão Bonaparte

ORIENTAÇÃO A OBJETOS PURA

Entre as linguagens de programação orientada a objetos, muito se discute se são puramente orientadas a objeto ou não, já que grande parte possui recursos que não se comportam como objetos.

Os tipos primitivos de Java são um exemplo desta contradição, já que não são objetos de verdade. Ruby é considerada uma linguagem puramente orientada a objetos, já que **tudo** em Ruby é um objeto (inclusive as classes, como veremos).

4.1 MUNDO ORIENTADO A OBJETOS

Ruby é uma linguagem puramente orientada a objetos, bastante influenciada pelo Smalltalk. Desta forma, **tudo** em Ruby é um objeto, até mesmo os tipos básicos que vimos até agora.

Uma maneira simples de visualizar isso é através da chamada de um método em qualquer um dos objetos:

```
"strings são objetos".upcase()  
:um_simbolo.object_id()
```

Até os números inteiros são objetos, da classe Fixnum:

```
10.class()
```


4.2 MÉTODOS COMUNS

Uma das funcionalidades comuns a diversas linguagens orientadas a objeto está na capacidade de, dado um objeto, descobrir de que tipo ele é. No ruby, existe um método chamado `class()`, que retorna o tipo do objeto, enquanto `object_id()`, retorna o número da referência, ou identificador único do objeto dentro da memória heap.

Outro método comum a essas linguagens, é aquele que “transforma” um objeto em uma `String`, geralmente usado para log. O Ruby também disponibiliza esse método, através da chamada ao `to_s()`.

Adicionalmente aos tipos básicos, podemos criar nossos próprios objetos, que já vem com esses métodos que todo objeto possui (`class`, `object_id`).

Para criar um objeto em Ruby, basta invocar o método `new` na classe que desejamos instanciar. O exemplo a seguir mostra como instanciar um objeto:

```
# criando um objeto
objeto = Object.new()
```

4.3 DEFINIÇÃO DE MÉTODOS

`def` é uma palavra chave do Ruby para a **definição** (criação) de métodos, que podem, claro, receber parâmetros:

```
def pessoa.vai(lugar)
  puts "indo para " + lugar
end
```

Mas, e o retorno de um método? Como funciona? Para diminuir o excesso de código que as linguagens costumam introduzir (chamado de ruído sintático), o Ruby optou por retornar o resultado da execução da última instrução executada no método. O exemplo a seguir mostra um método que devolve uma `String`:

```
def pessoa.vai(lugar)
  "indo para " + lugar
end
```

Para visualizar esse retorno funcionando, podemos acessar o método e imprimir o retorno do mesmo:

```
puts pessoa.vai("casa")
```

Podemos ainda refatorar o nosso método para usar interpolação:

```
def pessoa.vai(lugar)
  "indo para #{lugar}"
end
```

Para receber vários argumentos em um método, basta separá-los por vírgula:

```
def pessoa.troca(roupa, lugar)
  "trocando de #{roupa} no #{lugar}"
end
```

A invocação desses métodos é feita da maneira tradicional:

```
pessoa.troca('camiseta', 'banheiro')
```

Alguns podem até ter um valor padrão, fazendo com que sejam opcionais:

```
def pessoa.troca(roupa, lugar='banheiro')
  "trocando de #{roupa} no #{lugar}"
end
```

```
# invocação sem o parametro:
```

```
pessoa.troca("camiseta")
```

```
# invocação com o parametro:
```

```
pessoa.troca("camiseta", "sala")
```

4.4 EXERCÍCIOS - MÉTODOS

1) Métodos são uma das formas com que os objetos se comunicam. Vamos utilizar os conceitos vistos no capítulo anterior porém indo um pouco mais além na orientação a objetos agora.

- Crie mais um arquivo no seu diretório **ruby** chamado **restaurante_avancado**.

Ao final de cada exercício você pode rodar o arquivo no terminal com o comando **ruby restaurante_avancado.rb**.

2) Vamos simular um método que atribui uma nota a um restaurante.

```
# declaração do método
def qualifica(nota)
  puts "A nota do restaurante foi #{nota}"
end

# chamada do método
qualifica(10)
```

- 3) Podemos ter parâmetros opcionais em ruby utilizando um valor padrão.

```
def qualifica(nota, msg="Obrigado")
  puts "A nota do restaurante foi #{nota}. #{msg}"
end

# chamadas com parâmetros opcionais
qualifica(10)
qualifica(1, "Comida ruim.")
```

4.5 DISCUSSÃO: ENVIANDO MENSAGENS AOS OBJETOS

- 1) Na orientação a objetos a chamada de um método é análoga ao envio de uma mensagem ao objeto. Cada objeto pode reagir de uma forma diferente à mesma mensagem, ao mesmo estímulo. Isso é o polimorfismo.

Seguindo a ideia de envio de mensagens, uma maneira alternativa de chamar um método é usar o método `send()`, que todo objeto em Ruby possui.

```
pessoa.send(:fala)
```

O método `send` recebe como argumento o nome do método a ser invocado, que pode ser um símbolo ou uma string. De acordo com a orientação a objetos é como se estivéssemos enviando a mensagem **"fala"** ao objeto `pessoa`.

Além da motivação teórica, você consegue enxergar um outro grande benefício dessa forma de invocar métodos, através do `send()`? Qual?

4.6 CLASSES

Para não precisar adicionar sempre todos os métodos em todo objeto que criamos, Ruby possui classes, que atuam como fábricas (molde) de objetos. Classes possibilitam a criação de objetos já incluindo alguns métodos.

```
class Pessoa
  def fala
    puts "Sei Falar"
  end

  def troca(roupa, lugar="banheiro")
    "trocando de #{roupa} no #{lugar}"
  end
end
```

```
p = Pessoa.new
# o objeto apontado por p já nasce com os métodos fala e troca.
```

Todo objeto em Ruby possui o método `class`, que retorna a classe que originou este objeto (note que os parênteses podem ser omitidos na chamada e declaração de métodos):

```
p.class
# => Pessoa
```

O diferencial de classes em Ruby é que são abertas. Ou seja, **qualquer classe** pode ser alterada a qualquer momento na aplicação. Basta “reabrir” a classe e fazer as mudanças:

```
class Pessoa
  def novo_metodo
    # ...
  end
end
```

Caso a classe `Pessoa` já exista estamos apenas reabrindo sua definição para **adicionar** mais código. Não será criada uma nova classe e nem haverá um erro dizendo que a classe já existe.

4.7 EXERCÍCIOS - CLASSES

- 1) Nosso método `qualifica` não pertence a nenhuma classe atualmente. Crie a classe **Restaurante** e instancie o objeto.

```
class Restaurante
  def qualifica(nota, msg="Obrigado")
    puts "A nota do restaurante foi #{nota}. #{msg}"
  end
end
```

- 2) Crie dois objetos, com chamadas diferentes.

```
restaurante_um = Restaurante.new
restaurante_dois = Restaurante.new

restaurante_um.qualifica(10)
restaurante_dois.qualifica(1, "Ruim!")
```

4.8 DESAFIO: CLASSES ABERTAS

- 1) Qualquer classe em Ruby pode ser reaberta e qualquer método redefinido. Inclusive classes e métodos da biblioteca padrão, como `Object` e `Fixnum`.

Podemos redefinir a soma de números reabrindo a classe Fixnum? Isto seria útil?

```
class Fixnum
  def +(outro)
    self - outro # fazendo a soma subtrair
  end
end
```

4.9 SELF

Um método pode invocar outro método do próprio objeto. Para isto, basta usar a referência especial `self`, que aponta para o para o próprio objeto. É análogo ao `this` de outras linguagens como Java e C#.

Todo método em Ruby é chamado em algum objeto, ou seja, um método é sempre uma mensagem enviada a um objeto. Quando não especificado, o destino da mensagem é sempre `self`:

```
class Conta
  def transfere_para(destino, quantia)
    debita quantia
    # mesmo que self.debita(quantia)

    destino.deposita quantia
  end
end
```

4.10 DESAFIO: SELF E O MÉTODO PUTS

- 1) Vimos que todo método é sempre chamado em um objeto. Quando não especificamos o objeto em que o método está sendo chamado, Ruby sempre assume que seja em `self`.

Como tudo em Ruby é um objeto, todas as operações devem ser métodos. Em especial, `puts` não é uma operação, muito menos uma palavra reservada da linguagem.

```
puts "ola!"
```

Em qual objeto é chamado o método `puts`? Por que podemos chamar `puts` em qualquer lugar do programa? (dentro de classes, dentro de métodos, fora de tudo, ...)

- 2) Se podemos chamar `puts` em qualquer `self`, por que o código abaixo não funciona? (Teste!)

```
obj = "uma string"
obj.puts "todos os objetos possuem o método puts?"
```

- 3) (opcional) Pesquise onde (em que classe ou algo parecido) está definido o método `puts`. Uma boa dica é usar a documentação oficial da biblioteca padrão:

<http://ruby-doc.org>

4.11 ATRIBUTOS E PROPRIEDADES: ACESSORES E MODIFICADORES

Atributos, também conhecidos como variáveis de instância, em Ruby são sempre privados e começam com `@`. Não há como alterá-los de fora da classe; apenas os métodos de um objeto podem alterar os seus atributos (**encapsulamento!**).

```
class Pessoa
  def muda_nome(novo_nome)
    @nome = novo_nome
  end

  def diz_nome
    "meu nome é #{@nome}"
  end
end

p = Pessoa.new
p.muda_nome "João"
p.diz_nome

# => "João"
```

Podemos fazer com que algum código seja executado na criação de um objeto. Para isso, todo objeto pode ter um método especial, chamado de `initialize`:

```
class Pessoa
  def initialize
    puts "Criando nova Pessoa"
  end
end

Pessoa.new

# => "Criando nova Pessoa"
```

Os inicializadores são métodos privados (não podem ser chamados de fora da classe) e podem receber parâmetros. Veremos mais sobre métodos privados adiante.

```
class Pessoa
  def initialize(nome)
    @nome = nome
  end
end
```

```
joao = Pessoa.new("João")
```

Métodos acessores e modificadores são muito comuns e dão a ideia de propriedades. Existe uma convenção para a definição destes métodos, que a maioria dos desenvolvedores Ruby segue (assim como Java tem a convenção para *getters* e *setters*):

```
class Pessoa
  def nome # acessor
    @nome
  end

  def nome=(novo_nome)
    @nome = novo_nome
  end
end
```

```
pessoa = Pessoa.new
pessoa.nome="José"
puts pessoa.nome
# => "José"
```

4.12 SYNTAX SUGAR: FACILITANDO A SINTAXE

Desde o início, Matz teve como objetivo claro fazer com que Ruby fosse uma linguagem extremamente legível. Portanto, sempre que houver oportunidade de deixar determinado código mais legível, Ruby o fará.

Um exemplo importante é o modificador que acabamos de ver (`nome=`). Os parenteses na chamada de métodos são quase sempre opcionais, desde que não haja ambiguidades:

```
pessoa.nome= "José"
```

Para ficar bastante parecido com uma simples atribuição, bastaria colocar um espaço antes do '='. Priorizando a legibilidade, Ruby abre mão da rigidez sintática em alguns casos, como este:

```
pessoa.nome = "José"
```

Apesar de parecer, a linha acima **não é uma simples atribuição**, já que na verdade o método `nome=` está sendo chamado. Este recurso é conhecido como *Syntax Sugar*, já que o Ruby aceita algumas exceções na sintaxe para que o código fique mais legível.

A mesma regra se aplica às operações aritméticas que havíamos visto. Os números em Ruby também são objetos! Experimente:

```
10.class  
# => Fixnum
```

Os operadores em Ruby são métodos comuns. Tudo em Ruby é um objeto e todas as operações funcionam como envio de mensagens.

```
10.+(3)
```

Ruby tem *Syntax Sugar* para estes métodos operadores matemáticos. Para este conjunto especial de métodos, podemos omitir o ponto e trocar por um espaço. Como com qualquer outro método, os parênteses são opcionais.

4.13 EXERCÍCIOS - ATRIBUTOS E PROPRIEDADES

- 1) Crie um **initialize** no Restaurante.

```
class Restaurante  
  def initialize  
    puts "criando um novo restaurante"  
  end  
  # não apague o método 'qualifica'  
end
```

Rode o exemplo e verifique a chamada ao initialize.

- 2) Cada restaurante da nossa aplicação precisa de um nome diferente. Vamos criar o atributo e suas propriedades de acesso.

```
class Restaurante  
  def initialize(nome)  
    puts "criando um novo restaurante: #{nome}"  
  end  
  # não apague o método 'qualifica'  
end  
  
# crie dois nomes diferentes  
restaurante_um = Restaurante.new("Fasano")  
restaurante_dois = Restaurante.new("Fogo de Chao")
```

- 3) Embora nossos objetos tenham agora recebido um nome, esse nome não foi guardado em nenhuma variável.

```
class Restaurante  
  def initialize(nome)  
    puts "criando um novo restaurante: #{nome}"
```



```
@nome = nome
end

# altere o método qualifica
def qualifica(nota, msg="Obrigado")
  puts "A nota do #{@nome} foi #{nota}. #{msg}"
end
end
```

- 4) Repare que a variável **nome** é uma variável de instância de Restaurante. Em ruby, o “@” definiu esse comportamento. Agora vamos fazer algo parecido com o atributo **nota** e suas propriedades de acesso.

```
class Restaurante
  def initialize(nome)
    puts "criando um novo restaurante: #{nome}"
    @nome = nome
  end

  # altere o método qualifica
  def qualifica(msg="Obrigado")
    puts "A nota do #{@nome} foi #{@nota}. #{msg}"
  end

  # propriedades
  def nota=(nota)
    @nota = nota
  end

  def nota
    @nota
  end
end
```

```
restaurante_um = Restaurante.new("Fasano")
restaurante_dois = Restaurante.new("Fogo de Chao")
```

```
restaurante_um.nota = 10
restaurante_dois.nota = 1
```

```
restaurante_um.qualifica
restaurante_dois.qualifica("Comida ruim.")
```

- 5) Seria muito trabalhoso definir todas as propriedades de acesso a nossa variáveis. Refatore a classe Restaurante para utilizar o **attr_accessor :nota** Seu arquivo final deve ficar assim:

```
class Restaurante
```

```
attr_accessor :nota

def initialize(nome)
  puts "criando um novo restaurante: #{nome}"
  @nome = nome
end

def qualifica(msg="Obrigado")
  puts "A nota do #{@nome} foi #{@nota}. #{msg}"
end

end

restaurante_um = Restaurante.new("Fasano")
restaurante_dois = Restaurante.new("Fogo de Chao")

restaurante_um.nota = 10
restaurante_dois.nota = 1

restaurante_um.qualifica
restaurante_dois.qualifica("Comida ruim.")
```

4.14 COLEÇÕES

Arrays em Ruby são instâncias da classe Array, não sendo simplesmente uma estrutura de dados, mas possuindo diversos métodos auxiliares que nos ajudam no dia-a-dia.

Um exemplo simples e encontrado em objetos de tipo Array em várias linguagens é o que permite a inclusão de elementos e size que nos retorna a quantidade de elementos lá dentro.

```
lista = Array.new
lista << "RR-71"
lista << "RR-75"
lista << "FJ-91"

puts lista.size
# => 3
```

A tarefa mais comum ao interagir com array é a de resgatar os elementos e para isso usamos [] passando um índice como parametro:

```
puts lista[1]
# => "RR-75"
```

```
puts lista[0]
# => "RR-71"
```

Muitas vezes já sabemos de antemão o que desejamos colocar dentro da nossa array e o Ruby fornece uma maneira literal de declará-la:

```
lista = [1, 2, "string", :simbolo, /$regex~/]
puts lista[2]
# => string
```

Um exemplo que demonstra uma aparição de arrays é a chamada ao método `methods`, que retorna uma array com os nomes de todos os métodos que o objeto sabe responder naquele instante. Esse método é definido na classe `Object` então todos os objetos o possuem:

```
cliente = "Petrobras"

puts cliente.methods
```

4.15 EXEMPLO: MÚLTIPLOS PARÂMETROS

Em alguns instantes desejamos receber um número arbitrário de argumentos em um método, por exemplo a lista de produtos que estamos comprando em um serviço:

```
def compra(produto1, produto2, produto3, produtoN)
end
```

Para receber um número qualquer de parâmetros usamos a sintaxe `*` do Ruby:

```
def compra(*produtos)
  # produtos é uma array
  puts produtos.size
end
```

Note que nesse caso, em Ruby receber uma array ou um número indeterminado de argumentos resulta no mesmo código interno ao método pois a variável `produtos` funciona como uma array.

A mudança ocorre na sintaxe de chamada a esse método. Na versão mostrada acima, invocamos o método `compra` com diversos parâmetros:

```
compra("Notebook", "Pendrive", "Cafeteira")
```

Já no caso de definir o método recebendo uma array, somos obrigados a definir uma array no momento de invocá-lo:

```
def compra(produtos)
  # produtos é uma array
  puts produtos.size
end
compra( ["Notebook", "Pendrive", "Cafeteira"] )
```

O operador `*` é chamado de *splat*.

4.16 HASHES

Mas nem sempre desejamos trabalhar com arrays cuja maneira de encontrar o que está lá dentro é através de um número. Por exemplo, ao ler um arquivo de configuração de um servidor desejamos saber qual a porta e se ele deve ativar o suporte ssh:

```
porta = 80
ssh = false
nome = Caelum.com.br
```

Nesse caso não desejamos acessar uma array através de inteiros (Fixnum), mas sim o valor correspondente as chaves “porta”, “ssh” e “nome”.

Ruby também tem uma estrutura indexada por qualquer objeto, onde as chaves podem ser de qualquer tipo, o que permite atingir nosso objetivo. A classe Hash é quem dá suporte a essa funcionalidade, sendo análoga aos objetos HashMap, HashTable, arrays indexados por String e dicionários de outras linguagens.

```
config = Hash.new
config["porta"] = 80
config["ssh"] = false
config["nome"] = "Caelum.com.br"

puts config.size
# => 3

puts config["ssh"]
# => false
```

Por serem únicos e imutáveis, símbolos são ótimos candidatos a serem chaves em Hashes, portanto poderíamos trabalhar com:

```
config = Hash.new
config[:porta] = 80
```

Ao invocar um método com um número maior de parâmetros, o código fica pouco legível, já que Ruby não possui tipagem explícita, apesar de esta não ser a única causa.

Imagine que tenho uma conta bancária em minhas mãos e desejo invocar o método de transferência, que requer a conta destino, a data na qual o valor será transferido e o valor. Então tento invocar:

```
aluno = Conta.new
escola = Conta.new

# Time.now gera um objeto Time que representa "agora"
aluno.transfere(escola, Time.now, 50.00)
```

No momento de executar o método descobrimos que a ordem dos parâmetros era incorreta, o valor deveria vir antes da data, de acordo com a definição do método:

```
class Conta

  def transfere(destino, valor, data)
    # executa a transferência
  end

end
```

Mas só descobrimos esse erro ao ler a definição do método na classe original, e para contornar esse problema, existe um movimento que se tornou comum com a popularização do Rails 2, passando parâmetro através de hash:

```
aluno.transfere( {:destino => escola, :data => Time.now,
                  :valor => 50.00} )
```

Note que o uso do Hash implicou em uma legibilidade maior apesar de uma proliferação de palavras:

```
def transfere(argumentos)
  destino = argumentos[:destino]
  data = argumentos[:data]
  valor = argumentos[:valor]
  # executa a transferência
end
```

Isso acontece pois a semântica de cada parâmetro fica explícita para quem lê o código, sem precisar olhar a definição do método para lembrar o que eram cada um dos parâmetros.

COMBINANDO HASHES E NÃO HASHES

Variações nos símbolos permitem melhorar ainda mais a legibilidade, por exemplo:

```
class Conta
  def transfere(valor, argumentos)
    destino = argumentos[:para]
    data = argumentos[:em]
    # executa a transferência
  end
end

aluno.transfere(50.00, {:para => escola, :em => Time.now})
```

Para quem utiliza Ruby 1.9, ao criar um Hash onde a chave é um símbolo, podemos utilizar uma outra sintaxe que por vezes se torna mais legível:

```
aluno.transfere(50.00, {para: escola, em: Time.now})
```

Além dos parênteses serem sempre opcionais, quando um Hash é o último parâmetro de um método, as chaves podem ser omitidas (*Syntax Sugar*).

```
aluno.transfere :destino => escola, :valor => 50.0, :data => Time.now
```

4.17 EXERCÍCIOS - ARRAYS E HASHES

- 1) No seu diretório **ruby** crie mais um arquivo chamado **listas.rb**. Teste um array adicionando dois elementos.

```
nomes = []

nomes[0] = "Fasano"
nomes << "Fogo de Chao"

for nome in nomes
  puts nome
end
```

- 2) Vamos juntar os conhecimentos de classes e métodos para criar uma franquia onde podemos adicionar uma lista de restaurantes. Para isso teremos que criar na inicialização um array de restaurantes.

```
class Franquia

  def initialize
    @restaurantes = []
  end

  def adiciona(restaurante)
    @restaurantes << restaurante
  end

  def mostra
    for restaurante in @restaurantes
      puts restaurante.nome
    end
  end

end

class Restaurante
  attr_accessor :nome
end

restaurante_um = Restaurante.new
restaurante_um.nome = "Fasano"

restaurante_dois = Restaurante.new
restaurante_dois.nome = "Fogo de Chao"

franquia = Franquia.new
franquia.adiciona restaurante_um
franquia.adiciona restaurante_dois

franquia.mostra
```

- 3) O método adiciona recebe apenas um restaurante. Podemos usar a syntax com * e refatorar o método para permitir múltiplos restaurantes como parâmetro.

```
# refatore o método adiciona
def adiciona(*restaurantes)
  for restaurante in restaurantes
    @restaurantes << restaurante
  end
end

# adicione ambos de uma só vez
franquia.adiciona restaurante_um, restaurante_dois
```

- 4) Hashes são muito úteis para passar valores identificados nos parâmetros. Na classe **restaurante** adicione o método **fechar_conta** e faça a chamada.

```
def fechar_conta(dados)
  puts "Conta fechado no valor de #{dados[:valor]}
  e com nota #{dados[:nota]}. Comentário: #{dados[:comentario]}"
end

restaurante_um.fechar_conta :valor => 50, :nota => 9,
  :comentario => 'Gostei!'
```

4.18 BLOCOS E PROGRAMAÇÃO FUNCIONAL

Imagine o exemplo a seguir que soma o saldo das contas de um banco:

```
class Banco

  def initialize(contas)
    @contas = contas
  end

  def status
    saldo = 0
    for conta in @contas
      saldo += conta
    end
    saldo
  end

end

banco = Banco.new([200, 300, 400])
banco.status
```

Esse processo é executado em diversos pontos da nossa aplicação e todos eles precisam exatamente desse comportamento.

Em um dia ensolarado, um ponto de nossa aplicação passa a necessitar da impressão dos saldos parciais, isso é que cada soma seja impressa. Como fazer isso sem alterar os outros tantos pontos de execução e sem duplicação de código?

O primeiro passo é perceber a necessidade que temos de inserir um código novo, desejamos incluir o seguinte código:


```
for conta in @contas
  saldo += conta
  puts saldo # essa linha é nova
end
```

Então na chamada específica desse método, passemos esse código como “parâmetro”:

```
banco.status do |saldo|
  puts saldo
end
```

Isso não é um parâmetro, e sim um bloco de código, o código que desejamos executar. Note que esse bloco recebe um parâmetro chamado **saldo** e que esse parâmetro é o saldo parcial que desejamos imprimir. Para facilitar a leitura podemos renomeá-lo para **saldo_parcial**:

```
banco.status do |saldo_parcial|
  puts saldo_parcial
end
```

Imagine que o bloco funciona exatamente como a definição de uma função em ruby: ele pode receber parâmetros e ser invocado. Falta invocá-lo no código do banco, para dar a chance de execução a cada chamada do laço:

```
class Banco
  # initialize...
  def status(&block)
    saldo = 0
    for conta in @contas
      saldo += conta
      block.call(saldo)
    end
    saldo
  end
end
```

Note que block é um objeto que ao ter o método call invocado, chamará o bloco que foi passado, concluindo nosso primeiro objetivo: dar a chance de quem se interessar no saldo parcial, fazer algo com ele.

Qualquer outro tipo de execução, como outros cálculos, que eu desejar fazer para cada saldo, posso fazê-lo passando blocos distintos.

Ainda faltou manter compatibilidade com aquelas chamadas que não possuem bloco. Para isso, basta verificarmos se foi passado algum bloco como parâmetro para nossa função, e somente nesse caso invocá-lo.

Isto é, se o bloco foi dado (block_given?), então invoque o bloco:

```
for conta in @contas
  saldo += conta

  if block_given?
    block.call(saldo)
  end
end
```

OUTRA SINTAXE DE BLOCO

Existe uma outra sintaxe que podemos utilizar para passar blocos que envolve o uso de chaves:

```
banco.status { |saldo_parcial| puts saldo_parcial }
```

Como vimos até aqui, o método que recebe um bloco pode decidir se deve ou não chamá-lo. Para chamar o bloco associado, existe uma outra abordagem com a palavra `yield`:

```
if block_given?
  yield(saldo)
end
```

Nessa abordagem, não se faz necessário receber o argumento `&block` portanto o código do Banco seria:

```
class Banco
  # initialize...
  def status
    saldo = 0
    for conta in @contas
      saldo += conta
      if block_given?
        yield(saldo)
      end
    end
    saldo
  end
end
```

Entender quando usar blocos costuma parecer complicado no início, não se preocupe . Você pode enxergá-los como uma oportunidade do método delegar parte da responsabilidade a quem o chama, permitindo customizações em cada uma de suas chamadas (estratégias diferentes, ou callbacks).

A biblioteca padrão do Ruby faz alguns usos muito interessantes de blocos. Podemos analisar a forma de iterar em coleções, que é bastante influenciada por técnicas de programação funcional.

Dizer que estamos passando uma função (pedaço de código) como parâmetro a outra função é o mesmo que passar blocos na chamada de métodos.

Para iterar em uma Array possuímos o método `each`, que chama o bloco de código associado para cada um dos seus items, passando o item como parâmetro ao bloco:

```
lista = ["rails", "rake", "ruby", "rvm"]
lista.each do |programa|
  puts programa
end
```

A construção acima não parece trazer muitos ganhos se comparada a forma tradicional e imperativa de iterar em um array (`for`).

Imagine agora uma situação onde queremos colocar o nome de todos os funcionários em maiúsculo, isto é, aplicar uma função para todos os elementos de uma array, construindo uma array nova.

```
funcionarios = ["Guilherme", "Sergio", "David"]
nomes_maiusculos = []

for nome in funcionarios
  nomes_maiusculos << nome.upcase
end
```

Poderíamos usar o método `each`:

```
funcionarios = ["Guilherme", "Sergio", "David"]
nomes_maiusculos = []

funcionarios.each do |nome|
  nomes_maiusculos << nome.upcase
end
```

Mas as duas abordagens envolvem o conceito de dizer a linguagem que queremos adicionar um elemento a uma lista existente: o trabalho é imperativo.

Lembrando que o bloco, assim como uma função qualquer, possui retorno, seria muito mais compacto se pudéssemos exprimir o desejo de criar a array diretamente:

```
funcionarios = ["Guilherme", "Sergio", "David"]

nomes_maiusculos = funcionarios.cria_uma_array
```

O código dessa função deve iterar por cada elemento, adicionando eles dentro da nossa array, **exatamente** como havíamos feito antes:

```
class Array
  def cria_uma_array
    array = []
    self.each do |elemento|
      array << elemento.upcase
    end
    array
  end
end
```

Mas podemos reclamar que o método `upcase` nem sempre vai funcionar: a cada chamada de `cria_uma_array` queremos executar um comportamento diferente. E essa é a dica para utilização de blocos: passe um bloco que customiza o comportamento do método `cria_uma_array`:

```
nomes_maiusculos = funcionarios.cria_uma_array do |nome|
  nome.upcase
end
```

E faça o `cria_uma_array` invocar esse bloco:

```
class Array
  def cria_uma_array
    array = []
    self.each do |elemento|
      array << yield(elemento)
    end
  end
end
```

Esse método que criamos já existe e se chama `map` (ou `collect`), que coleta os retornos de todas as chamadas do bloco associado:

```
funcionarios = ["Guilherme", "Sergio", "David"]

nomes_maiusculos = funcionarios.map do |nome|
  nome.upcase
end
```

Na programação imperativa tradicional precisamos de no mínimo mais duas linhas, para o array auxiliar e para adicionar os itens maiúsculos no novo array.

Diversos outros métodos do módulo `Enumerable` seguem a mesma ideia: `find`, `find_all`, `grep`, `sort`, `inject`. Não deixe de consultar a documentação, que pode ajudar a criar código mais compacto.

Só tome cuidado pois código mais compacto nem sempre é mais legível e fácil de manter.

4.19 EXERCÍCIOS - BLOCOS

- 1) Refatore o método `mostra` de **Franquia** para iterar sobre os elementos usando blocos.

```
def mostra
  @restaurantes.each do |r|
    puts r.nome
  end
end
```

- 2) Crie um método **relatorio** que envia a lista de restaurantes da franquía. Repare que esse método não são o que ocorrerá com cada item da lista. Ele disponibiliza os itens e é o bloco passado que define o que fazer com eles.

```
def relatorio
  @restaurantes.each do |r|
    yield r
  end
end

# chamada com blocos
franquia.relatorio do |r|
  puts "Restaurante cadastrado: #{r.nome}"
end
```

4.20 PARA SABER MAIS: MAIS SOBRE BLOCOS

Analise e rode o código abaixo, olhe na documentação o método `sort_by` e teste o `next`.

```
caelum = [
  { :ruby => 'rr-71', :java => 'fj-11' },
  { :ruby => 'rr-75', :java => 'fj-21' }
]

caelum.sort_by { |curso| curso[:ruby] }.each do |curso|
  puts "Curso de RoR na Caelum: #{ curso[:ruby] }"
end

caelum.sort_by { |curso| curso[:ruby] }.each do |curso|
```

```
next if curso[:ruby] == 'rr-71'
puts "Curso de RoR na Caelum: #{ curso[:ruby] }"
end
```

4.21 DESAFIO: USANDO BLOCOS

1) Queremos imprimir o nome de todos os alunos de uma turma com o código a seguir:

```
fj91 = Turma.new("Guilherme", "Paulo", "Paniz")

fj91.each do |nome|
  puts nome
end
```

Crie a sua classe **Turma** que durante a invocação do método `each`, itera por todos os nomes passados em seu construtor.

Tente lembrar dos conceitos de blocos e programação funcional para resolver.

A biblioteca padrão do Ruby faz usos bastante interessante de módulos como mixins. Os principais exemplos são os módulos `Enumerable` e `Comparable`.

4.22 MANIPULANDO ERROS E EXCEPTIONS

Uma *exception* é um tipo especial de objeto que estende ou é uma instância da classe `Exception`. Lançar uma exception significa que algo não esperado ou errado ocorreu no fluxo do programa. *Raising* é a palavra usada em ruby para lançamento de exceptions. Para tratar uma exception é necessário criar um código a ser executado caso o programa receba o erro. Para isso existe a palavra `rescue`.

EXCEPTIONS COMUNS

A lista abaixo mostra as exceptions mais comuns em ruby e quando são lançadas, todas são filhas de `Exception`. Nos testes seguintes você pode usar essas exceptions.

- **RuntimeError** : É a exception padrão lançada pelo método `raise`.
- **NoMethodError** : Quando um objeto recebe como parametro de uma mensagem um nome de método que não pode ser encontrado.
- **NameError** : O interpretador não encontra uma variável ou método com o nome passado.
- **IOError** : Causada ao ler um stream que foi fechado, tentar escrever em algo *read-only* e situações similares.
- **Errno::error** : É a família dos erros de entrada e saída (IO).

- **TypeError** : Um método recebe como argumento algo que não pode tratar.
- **ArgumentError** : Causada por número incorreto de argumentos.

4.23 EXERCÍCIO: MANIPULANDO EXCEPTIONS

- 1) Em um arquivo ruby crie o código abaixo para testar exceptions. O método `gets` recupera o valor digitado. Teste também outros tipos de exceptions e digite um número inválido para testar a exception.

```
print "Digite um número:"
numero = gets.to_i

begin
  resultado = 100 / numero
rescue
  puts "Número digitado inválido!"
  exit
end

puts "100/#{numero} é #{resultado} "
```

- 2) (opcional) Exceptions podem ser lançadas com o comando `raise`. Crie um método que lança uma exception do tipo `ArgumentError` e capture-a com `rescue`.

```
def verifica_idade(idade)
  unless idade > 18
    raise ArgumentError,
      "Você precisa ser maior de idade para jogar jogos violentos."
  end
end

verifica_idade(17)
```

- 3) (opcional) É possível utilizar sua própria exception criando uma classe e estendendo de `Exception`.

```
class IdadeInsuficienteException < Exception
end

def verifica_idade(idade)
  raise IdadeInsuficienteException,
    "Você precisa ser maior de idade..." unless idade > 18
end
```

Para testar o `rescue` dessa exception invoque o método com um valor inválido:

```
begin
  verifica_idade(13)
```

```
rescue IdadeInsuficienteException => e
  puts "Foi lançada a exception: #{e}"
end
```

PARA SABER MAIS: THROW E CATCH

Ruby possui também throw e catch que podem ser utilizados com símbolos e a sintaxe lembra a de Erlang, onde catch é uma função que, se ocorrer algum throw com aquele label, retorna o valor do throw atrelado:

```
def pesquisa_banco(nome)
  if nome.size < 10
    throw :nome_invalido, "Nome invalido, digite novamente"
  end
  # executa a pesquisa
  "cliente #{nome}"
end

def executa_pesquisa(nome)
  catch :nome_invalido do
    cliente = pesquisa_banco(nome)
    return cliente
  end
end

puts executa_pesquisa("ana")
# => "Nome invalido, digite novamente"

puts executa_pesquisa("guilherme silveira")
# => cliente guilherme silveira
```

4.24 ARQUIVOS COM CÓDIGO FONTE RUBY

Todos os arquivos fonte, contendo código Ruby devem ter a extensão `.rb` para que o interpretador seja capaz de carregá-lo.

Existem exceções onde a leitura dos arquivos é feita por outras bibliotecas e, nesses casos, o arquivo possui outras extensões.

Para organizar seu código, é natural dividi-lo em vários arquivos e diretórios, bastando usar o método `require` para incluir o fonte de outro arquivo.

Imagine o arquivo `conta.rb`:

```
class Conta
```



```
attr_reader :saldo

def initialize(saldo)
  @saldo = saldo
end
end
```

Agora podemos acessar uma conta bastando primeiro importar o arquivo que a define:

```
require 'conta'

puts Conta.new(500).saldo
```

Caso a extensão *.rb* seja omitida, a extensão adequada será usada (*.rb*, *.so*, *.class*, *.dll*, etc). O Ruby procura pelo arquivo em alguns diretórios predefinidos (*Ruby Load Path*), incluindo o diretório atual.

Assim como qualquer outra linguagem isso resulta em um possível Load Hell, onde não sabemos exatamente de onde nossos arquivos estão sendo carregados. Tome bastante cuidado para a configuração de seu ambiente.

Caminhos relativos ou absolutos podem ser usados para incluir arquivos em outros diretórios, sendo que o absoluto não é recomendado devido ao atrelamento claro com uma estrutura fixa que pode não ser encontrada ao portar o código para outra máquina:

```
require 'modulo/funcionalidades/coisa_importante'
require '/usr/local/lib/my/libs/ultra_parser'
```

CONSTANTES DO SISTEMA

A constante `$:`, ou `$LOAD_PATH` contém diretórios do **Load Path**:

```
$:
# => ["/Library/Ruby/Site/1.8", ..., "."]
```

Existem diversas outras constantes que começam com `$`, todas elas que são resquícios de PERL e que, em sua grande maioria dentro do ambiente Rails, possuem alternativas mais compreensíveis como `LOAD_PATH`.

O comando `require` carrega o arquivo apenas uma vez. Para executar a interpretação do conteúdo do arquivo diversas vezes, utilize o método `load`.

```
load 'conta.rb'
load 'conta.rb'
# executado duas vezes!
```

Portanto no irb (e em Ruby em geral) para recarregar um arquivo que foi alterado é necessário executar um load e não um require - que não faria nada.

4.25 PARA SABER MAIS: UM POUCO DE IO

Para manipular arquivos de texto existe a classe `File`, que permite manipulá-los de maneira bastante simples, utilizando blocos:

```
print "Escreva um texto: "  
texto = gets  
File.open( "caelum.txt", "w" ) do |f|  
  f << texto  
end
```

E para imprimir seu conteúdo:

```
Dir.entries('caelum').each do |file_name|  
  idea = File.read( file_name )  
  puts idea  
end
```

Podemos lidar de maneira similar com requisições HTTP utilizando o código abaixo e imprimir o conteúdo do resultado de uma requisição:

```
require 'net/http'  
Net::HTTP.start( 'www.caelum.com.br', 80 ) do |http|  
  print( http.get( '/' ).body )  
end
```

Metaprogramação e Padrões de Projeto

“A ciência nunca resolve um problema sem criar mais dez”

– George Bernard Shaw

5.1 MÉTODOS DE CLASSE

Classes em Ruby **também são objetos**:

```
Pessoa.class
# => Class

c = Class.new
instancia = c.new
```

Variáveis com letra maiúscula representam constantes em Ruby, que até podem ser modificadas, mas o interpretador gera um *warning*. Portanto, `Pessoa` é apenas uma constante que aponta para um objeto do tipo `Class`.

Se classes são objetos, podemos definir métodos de classe como em qualquer outro objeto:

```
class Pessoa
  # ...
end

def Pessoa.pessoas_no_mundo
  100
end
```

```
Pessoa.pessoas_no_mundo  
# => 100
```

Há um *idiomismo* para definir os métodos de classe dentro da própria definição da classe, onde `self` aponta para o próprio objeto classe.

```
class Pessoa  
  def self.pessoas_no_mundo  
    100  
  end  
  
  # ...  
end
```

5.2 PARA SABER MAIS: SINGLETON CLASSES

A definição `class << object` define as chamadas singleton classes em ruby. Por exemplo, uma classe normal em ruby poderia ser:

```
class Pessoa  
  def fala  
    puts 'oi'  
  end  
end
```

Podemos instanciar e invocar o método normalmente:

```
p = Pessoa.new  
p.fala # imprime 'oi'
```

Entretanto, também é possível definir métodos apenas para esse objeto “p”, pois tudo em ruby, até mesmo as classes, são objetos, fazendo:

```
def p.anda  
  puts 'andando'  
end
```

O método “anda” é chamado de singleton method do objeto “p”.

Um singleton method “vive” em uma singleton class. Todo objeto em ruby possui 2 classes:

- a classe a qual foi instanciado

- sua singleton class

A singleton class é exclusiva para guardar os métodos desse objeto, sem compartilhar com outras instâncias da mesma classe.

Existe uma notação especial para definir uma singleton class:

```
class << Pessoa
  def anda
    puts 'andando'
  end
end
```

Definindo o código dessa forma temos o mesmo que no exemplo anterior, porém definindo o método anda explicitamente na singleton class. É possível ainda definir tudo na mesma classe:

```
class Pessoa
  class << self
    def anda
      puts 'andando'
    end
  end
end
```

Mais uma vez o método foi definido apenas para um objeto, no caso, o objeto “Pessoa”, podendo ser executado com:

```
Pessoa.anda
```

5.3 EXERCÍCIOS - RUBY OBJECT MODEL

- 1) Crie um novo arquivo chamado **metaprogramacao.rb**. Independente da instância seria interessante saber o número total de restaurantes criados. Na mesma classe restaurante, crie uma variável de classe chamada **total**.

```
class Restaurante
  def initialize(nome)
    puts "criando um novo restaurante: #{nome}"
    @@total ||= 0
    @@total = @@total + 1
    puts "Restaurantes criados: #{@@total}"
    @nome = nome
  end
end
```

Execute novamente e analise o resultado.

- 2) Crie um método **self** chamado **relatorio** que mostra a quantidade total de restaurantes instanciados. (Opcional: Com esse método, você pode limpar um pouco o initialize).

```
def self.relatorio
  puts "Foram criados #{@total} restaurantes"
end
```

```
# Faça mais uma chamada
Restaurante.relatorio
```

MÉTODO DE CLASSE É DIFERENTE DE STATIC DO JAVA

Se você trabalha com **java** pode confundir o **self** com o **static**. Cuidado! O método definido como **self** roda apenas na classe, não funciona nas instâncias. Você pode testar fazendo:

```
Restaurante.relatorio
restaurante_um.relatorio
```

A invocação na instância dará um: `NoMethodError: undefined method 'relatorio' for #<Restaurante:0x100137b48 @nome="Fasano", @nota=10>`

- 3) Caso sua classe possua muitos métodos de classe, é recomendado agrupá-los. Muitos códigos ruby utilizaram essa técnica, inclusive no próprio Rails Sua classe ficará então assim:

```
class Restaurante
  # initialize, qualifica ...

  class << self
    def relatorio
      puts "Foram criados #{@total} restaurantes"
    end
  end
end
```

Execute e veja que o comportamento é o mesmo.

5.4 DESIGN PATTERNS: SINGLETON

Como criar uma classe com a garantia de ser instanciada apenas uma vez ? Essa classe deveria ser capaz de verificar se alguma vez já foi instanciada e saber devolver sempre a mesma referência.

Como visto anteriormente, em ruby, variáveis com @ são de instância e variáveis com @@ são variáveis de classe.

Utilizando a ideia acima podemos criar uma classe simples de relatório, onde desejamos que apenas uma seja de fato criada.

```
class Relatorio
  @@instance = Relatorio.new

  def self.instance
    return @@instance
  end
end
```

Dessa forma conseguimos criar o relatório apenas uma vez. Mas a implementação ainda apresenta problemas. Ainda é possível instanciar o Relatório mais de uma vez. Para resolver esse problema e implementar a classe Relatório como um Singleton realmente, precisamos tornar privado o **new** da nossa classe. Dessa forma, apenas será possível acessar o Relatorio a partir do método **instance**.

```
class Relatorio
  @@instance = Relatorio.new

  def self.instance
    return @@instance
  end

  private_class_method :new
end

# ambos relatórios referenciam o mesmo objeto
relatorio1 = Relatorio.instance
relatorio2 = Relatorio.instance
```

Existe ainda uma maneira mais otimizada e produtiva de chegar no mesmo resultado. O ruby já vem com um módulo chamado **Singleton**. Basta inclui-lo na classe para ter o mesmo resultado. Você verá mais sobre módulos no decorrer do curso.

```
require 'singleton'
class Relatorio
  include Singleton
end
```

5.5 EXERCÍCIO: DESIGN PATTERN - SINGLETON

1) Crie o Relatório de forma a retornar sempre a mesma instância:

```
class Relatorio
  @@instance = Relatorio.new

  def self.instance
    return @@instance
  end

  private_class_method :new
end

# ambos relatórios referenciam o mesmo objeto
relatorio1 = Relatorio.instance
relatorio2 = Relatorio.instance

puts relatorio1 == relatorio2
```

2) Faça o mesmo teste, mas conhecendo agora o módulo Singleton do ruby:

```
require 'singleton'
class Relatorio
  include Singleton
end

# ambos relatórios referenciam o mesmo objeto
relatorio1 = Relatorio.instance
relatorio2 = Relatorio.instance

puts relatorio1 == relatorio2
```

5.6 CONVENÇÕES

Métodos que retornam booleanos costumam terminar com ?, para que pareçam perguntas aos objetos:

```
texto = "nao sou vazio"
texto.empty? # => false
```

Já vimos esta convenção no método respond_to?.

Métodos que tem efeito colateral (alteram o estado do objeto, ou que costumem lançar exceções) geralmente terminam com ! (bang):

```
conta.cancela!
```


A comparação entre objetos é feita através do método `==` (sim, é um método!). A versão original do método apenas verifica se as referências são iguais, ou seja, se apontam para os mesmos objetos. Podemos reescrever este comportamento e dizer como comparar dois objetos:

```
class Pessoa
  def ==(outra)
    self.cpf == outra.cpf
  end
end
```

Na definição de métodos, procure sempre usar os parênteses. Para a chamada de métodos, não há convenção. Prefira o que for mais legível.

Nomes de variável e métodos em Ruby são sempre minúsculos e separados por `'_'` (underscore). Variáveis com nomes maiúsculo são sempre constantes. Para nomes de classes, utilize as regras de **CamelCase**, afinal nomes de classes são apenas constantes.

5.7 POLIMORFISMO

Ruby também tem suporte a herança simples de classes:

```
class Animal
  def come
    "comendo"
  end
end

class Pato < Animal
  def quack
    "Quack!"
  end
end

pato = Pato.new
pato.come # => "comendo"
```

Classes filhas herdam todos os métodos definidos na classe mãe.

A tipagem em Ruby não é explícita, por isso não precisamos declarar quais são os tipos dos atributos. Veja este exemplo:

```
class PatoNormal
  def faz_quack
```

```
"Quack!"
end
end

class PatoEstranho
  def faz_quack
    "Queck!"
  end
end

class CriadorDePatos
  def castiga(pato)
    pato.faz_quack
  end
end

pato1 = PatoNormal.new
pato2 = PatoEstranho.new
c = CriadorDePatos.new
c.castiga(pato1) # => "Quack!"
c.castiga(pato2) # => "Queck!"
```

Para o criador de patos, não interessa que objeto será passado como parâmetro. Para ele basta que o objeto saiba fazer *quack*. Esta característica da linguagem Ruby é conhecida como *Duck Typing*.

"If it walks like a duck and quacks like a duck, I would call it a duck."

5.8 EXERCÍCIOS - DUCK TYPING

- 1) Para fazer alguns testes de herança e polimorfismo abra o diretório **ruby** e crie um novo arquivo chamado **duck_typing.rb**. Nesse arquivo faça o teste do restaurante herdando um método da classe **Franquia**.

```
class Franquia
  def info
    puts "Restaurante faz parte da franquia"
  end
end

class Restaurante < Franquia
end

restaurante = Restaurante.new
restaurante.info
```

Execute o arquivo no terminal e verifique a utilização do método herdado.

- 2) Podemos em ruby fazer a sobrescrita do método e invocar o método da classe mãe.

```
class Franquia
  def info
    puts "Restaurante faz parte da franquia"
  end
end
```

```
class Restaurante < Franquia
  def info
    super
    puts "Restaurante Fasano"
  end
end
```

```
restaurante = Restaurante.new
restaurante.info
```

- 3) Como a linguagem ruby é implicitamente tipada, não temos a garantia de receber um objeto do tipo que esperamos. Isso faz com que acreditemos que o objeto é de um tipo específico caso ele possua um método esperado. Faça o teste abaixo:

```
class Franquia
  def info
    puts "Restaurante faz parte da franquia"
  end
end
```

```
class Restaurante < Franquia
  def info
    super
    puts "Restaurante Fasano"
  end
end
```

```
# metodo importante
# recebe franquia e invoca o método info
def informa(franquia)
  franquia.info
end
```

```
restaurante = Restaurante.new
informa restaurante
```

5.9 DESIGN PATTERNS: TEMPLATE METHOD

As operações de um sistema podem ser de diversos tipos e o que determina qual operação será realizada pode ser um simples variável. Dependendo da variável uma coisa pode ocorrer enquanto outra vez é necessário fazer outra. Para exemplificar melhor, vamos usar a classe **Relatorio**. Nosso relatório mostrará um conteúdo inicialmente em HTML.

```
class Relatorio
  def imprime
    puts "<html>Dados do restaurante</html>"
  end
end
```

O que acontece caso um parâmetro defina o formato dos dados que o relatório precisa ser criado?

```
class Relatorio
  def imprime(formato)
    if formato == :texto
      puts "*** Dados do restaurante ***"
    elsif formato == :html
      puts "<html>Dados do restaurante</html>"
    else
      puts "formato desconhecido!"
    end
  end
end
```

Para solucionar esse problema de forma orientada a objetos poderíamos criar uma classe abstrata que define o comportamento de um relatório, ou seja, que define que um relatório de ter um head, um body, um footer, etc. Mas como criar uma classe abstrata em Ruby? Embora não exista a palavra chave reservada “abstract” o conceito permanece presente na linguagem. Vejamos:

```
class Relatorio
  def imprime
    imprime_cabecalho
    imprime_conteudo
  end
end
```

A classe Relatorio agora possui os métodos que definem um relatório. Obviamente esses métodos podem ser invocados. A solução para isso normalmente é lançar uma exception nesses métodos não implementados. Agora que temos nossa classe abstrata, podemos criar subclasses de Relatorio que contém a implementação de cada um dos tipos, por exemplo para HTML:

```
class HTMLRelatorio < Relatorio
  def imprime_cabecalho
    puts "<html>"
  end

  def imprime_conteudo
    puts "Dados do relatorio"
  end
end

class TextoRelatorio < Relatorio
  def imprime_cabecalho
    puts "***"
  end

  def imprime_conteudo
    puts "Dados do relatorio"
  end
end
```

Agora para usar nossos relatórios podemos fazer:

```
relatorio = HTMLRelatorio.new
relatorio.imprime

relatorio = TextoRelatorio.new
relatorio.imprime
```

5.10 EXERCÍCIO OPCIONAL: DESIGN PATTERN - TEMPLATE METHOD

1) Crie a classe que define as obrigações de um relatório:

```
class Relatorio
  def imprime
    imprime_cabecalho
    imprime_conteudo
  end
end
```

2) Para termos implementações diferentes, crie um relatório HTML e um de texto puro:

```
class HTMLRelatorio < Relatorio
  def imprime_cabecalho
    puts "<html>"
  end
end
```

```
    def imprime_conteudo
      puts "Dados do relatorio"
    end
  end

  class TextoRelatorio < Relatorio
    def imprime_cabecalho
      puts "***"
    end

    def imprime_conteudo
      puts "Dados do relatorio"
    end
  end
end
```

3) Utilize os relatórios com a chamada ao método imprime:

```
relatorio = HTMLRelatorio.new
relatorio.imprime

relatorio = TextoRelatorio.new
relatorio.imprime
```

5.11 MODULOS

Modulos podem ser usados como namespaces:

```
module Caelum
  module Validadores

    class ValidadorDeCpf
      # ...
    end

    class ValidadorDeRg
      # ...
    end

  end
end

validador = Caelum::Validadores::ValidadorDeCpf.new
```

Ou como **mixins**, conjunto de métodos a ser incluso em outras classes:

```
module Comentavel
  def comentarios
    @comentarios ||= []
  end

  def recebe_comentario(comentario)
    self.comentarios << comentario
  end
end

class Revista
  include Comentavel
  # ...
end

revista = Revista.new
revista.recebe_comentario("muito ruim!")
puts revista.comentarios
```

5.12 DESIGN PATTERNS: OBSERVER

Como executar operações caso algo ocorra no sistema. Temos uma classe Restaurante e queremos manter avisados todos os objetos do sistema que se interessem em modificações nele. Nossa classe Franquia precisa executar o método **alerta** no caso

```
class Franquia

  def alerta
    puts "Um restaurante foi qualificado"
  end

end

class Restaurante

  def qualifica(nota)
    puts "Restaurante recebeu nota #{nota}"
  end

end

restaurante = Restaurante.new
restaurante.qualifica(10)
```

Poderíamos chamar o método **alerta** direto ao final do método **qualifica** ou poderíamos passar como parâmetro do método um observer para rodar. Porém essas maneiras acoplariam muito nosso código, por exemplo, não funcionariam para alertar mais de um observer.

Uma saída é criar uma lista de observadores e executá-los ao final da operação. Para isso podemos ter um método que permite adicionar quantos objetos forem necessários serem alertados.

```
class Restaurante

  def initialize
    @observers = []
  end

  def adiciona_observer(observer)
    @observer << observer
  end

  def notifica
    # percorre todos os observers chamando o método alerta
  end

  # qualifica

end

restaurante = Restaurante.new
restaurante.qualifica(10)
```

Poder ser observado não é uma característica única de um objeto. Podemos utilizar essa estratégia de avisar componentes por todo o software. Por que não colocar o comportamento em uma classe responsável por isso. Utilizando herança tornaríamos nosso código **observável**.

Ao invés de utilizar a herança novamente podemos utilizar os módulos para isso. Todo comportamento que faz do objeto ser um **observer** será colocado nesse módulo.

```
module Observer
  def initialize
    @observers = []
  end

  def adiciona_observer(observer)
    @observer << observer
  end

  def notifica
```



```
# percorre todos os observers chamando o método alerta
end
end

class Restaurante
  include Observer
  def qualifica(nota)
    puts "Restaurante recebeu nota #{nota}"
    notifica
  end
end
```

MODULO OBSERVER

O módulo acima já existe em ruby e é chamado de Observer. <http://ruby-doc.org/core/classes/Observerable.html>

5.13 DESAFIO: DESIGN PATTERN - OBSERVER

- 1) Crie um novo arquivo e implemente o Design Pattern Observer para Restaurante e Franquia.

5.14 METAPROGRAMAÇÃO

Por ser uma linguagem dinâmica, Ruby permite adicionar outros métodos e operações aos objetos em tempo de execução.

Imagine que tenho uma pessoa:

```
pessoa = Object.new()
```

O que aconteceria, se eu tentasse invocar um método inexistente nesse objeto? Por exemplo, se eu tentar executar

```
pessoa.fala()
```

O interpretador retornaria com uma mensagem de erro uma vez que o método não existe.

Mas e se eu desejasse, **em tempo de execução**, adicionar o comportamento (ou método) fala para essa pessoa. Para isso, tenho que **definir** que uma **pessoa** possui o método **fala**:

```
pessoa = Object.new()
```

```
def pessoa.fala()
  puts "Sei falar"
end
```

Agora que tenho uma pessoa com o método fala, posso invocá-lo:

```
pessoa = Object.new()

def pessoa.fala()
  puts "Sei falar"
end

pessoa.fala()
```

Tudo isso é chamado **meta-programação**, um recurso muito comum de linguagens dinâmicas. Meta-programação é a capacidade de gerar/alterar código em tempo de execução. Note que isso é muito diferente de um gerador de código comum, onde geráramos um código fixo, que deveria ser editado na mão e a aplicação só rodaria esse código posteriormente.

Levando o dinamismo de Ruby ao extremo, podemos criar métodos que definem métodos em outros objetos:

```
class Aluno
  # nao sabe nada
end

class Professor
  def ensina(aluno)
    def aluno.escreve
      "sei escrever!"
    end
  end
end

juca = Aluno.new
juca.respond_to? :escreve
# => false

professor = Professor.new
professor.ensina juca
juca.escreve
# => "sei escrever!"
```

A criação de métodos acessores é uma tarefa muito comum no desenvolvimento orientado a objetos. Os métodos são sempre muito parecidos e os desenvolvedores costumam usar recursos de geração de códigos das IDEs para automatizar esta tarefa.

Já vimos que podemos criar código Ruby que escreve código Ruby (métodos). Aproveitando essa possibilidade do Ruby, existem alguns métodos de classe importantes que servem apenas para criar alguns outros métodos nos seus objetos.

```
class Pessoa
  attr_accessor :nome
end

p = Pessoa.new
p.nome = "Joaquim"
puts p.nome
# => "Joaquim"
```

A chamada do método de classe `attr_accessor`, define os métodos `nome` e `nome=` na classe `Pessoa`.

A técnica de *código gerando código* é conhecida como **metaprogramação**, ou **metaprogramming**, como já definimos.

Outro exemplo interessante de metaprogramação é como definimos a visibilidade dos métodos em Ruby. Por padrão, todos os métodos definidos em uma classe são públicos, ou seja, podem ser chamados por qualquer um.

Não existe nenhuma palavra reservada (*keyword*) da linguagem para mudar a visibilidade. Isto é feito com um método de classe. Toda classe possui os métodos `private`, `public` e `protected`, que são métodos que alteram outros métodos, mudando a sua visibilidade (código alterando código == **metaprogramação**).

Como visto, por padrão todos os métodos são públicos. O método de classe `private` altera a visibilidade de todos os métodos definidos após ter sido chamado:

```
class Pessoa

  private

  def vai_ao_banheiro
    # ...
  end
end
```

Todos os métodos após a chamada de `private` são privados. Isso pode lembrar um pouco C++, que define regiões de visibilidade dentro de uma classe (seção pública, privada, ...). Um método privado em Ruby **só pode ser chamado em self** e o `self` deve ser **implícito**. Em outras palavras, não podemos colocar o `self` explicitamente para métodos privados, como em `self.vai_ao_banheiro`.

Caso seja necessário, o método `public` faz com que os métodos em seguida voltem a ser públicos:

```
class Pessoa

  private
  def vai_ao_banheiro
    # ...
  end

  public
  def sou_um_metodo_publico
    # ...
  end
end
```

O último modificador de visibilidade é o `protected`. Métodos `protected` só podem ser chamados em `self` (implícito ou explícito). Por isso, o `protected` do Ruby acaba sendo semelhante ao `protected` do Java e C++, que permitem a chamada do método na própria classe e em classes filhas.

5.15 EXERCÍCIOS - METAPROGRAMAÇÃO

- 1) Alguns restaurantes poderão receber um método a mais chamado “`cadastrar_vips`”. Para isso precisaremos abrir em tempo de execução a classe `Restaurante`.

```
# faça a chamada e verifique a exception NoMethodError
restaurante_um.cadastrar_vips

# adicione esse método na classe Franquia
def expandir(restaurante)
  def restaurante.cadastrar_vips
    puts "Restaurante #{self.nome} agora com área VIP!"
  end
end

# faça a franquía abrir a classe e adicionar o método
franquia.expandir restaurante_um
restaurante_um.cadastrar_vips
```

- 2) Um método útil para nossa franquía seria a verificação de nome de restaurantes cadastrados. vamos fazer isso usando **`method_missing`**.

```
# Adicione na classe Franquia
def method_missing(name, *args)
  @restaurantes.each do |r|
    return "O restaurante #{r.nome} já foi cadastrado!"
    if r.nome.eql? *args
  end
end
```

```
    return "O restaurante #{args[0]} não foi cadastrado ainda."  
end
```

```
    # Faça as chamadas e analise os resultados  
    puts franquia.já_cadastrado?("Fasano")  
    puts franquia.já_cadastrado?("Boteco")
```

CAPÍTULO 6

Ruby on Rails

“A libertação do desejo conduz à paz interior”

– Lao-Tsé

Aqui faremos um mergulho no Rails e, em um único capítulo, teremos uma pequena aplicação pronta: com banco de dados, interface web, lógica de negócio e todo o necessário para um CRUD, para que nos capítulos posteriores possamos ver cada parte do Rails com detalhes e profundidade, criando uma aplicação bem mais completa.

6.1 RUBY ON RAILS

David Heinemeier Hansson criou o Ruby on Rails para usar em um de seus projetos na *37signals*, o Basecamp. Desde então, passou a divulgar o código e incentivar o uso do mesmo, e em 2006 começou a ganhar muita atenção da comunidade de desenvolvimento Web.

O Rails foi criado pensando na praticidade que ele proporcionaria na hora de escrever os aplicativos para Web. No Brasil a Caelum vem utilizando o framework desde 2007, e grandes empresas como Abril e Locaweb adotaram o framework em uma grande quantidade de projetos.

Outro atrativo do framework é que, comparado a outros, ele permite que as funcionalidades de um sistema possam ser implementadas de maneira incremental por conta de alguns padrões e conceitos adotados. Isso tornou o Rails uma das escolhas óbvias para projetos e empresas que adotam metodologias ágeis de desenvolvimento e gerenciamento de projeto.

Para saber mais sobre metodologias ágeis, a Caelum oferece os cursos *Gerenciamento ágil de projetos de Software com Scrum* (PM-83) e *Práticas ágeis de desenvolvimento de Software* (PM-87).

Como pilares do Rails estão os conceitos de *Don't Repeat Yourself* (DRY), e *Convention over Configuration* (CoC).

O primeiro conceito nos incentiva a fazer bom uso da **reutilização de código**, que é também uma das principais vantagens da orientação a objetos. Além de podermos aproveitar as características de OO do Ruby, o próprio framework nos incentiva a adotar padrões de projeto mais adequados para essa finalidade.

O segundo conceito nos traz o benefício de poder escrever muito menos código para implementar uma determinada funcionalidade em nossa aplicação, desde que respeitemos alguns padrões de nome e localização de arquivos, nome de classes e métodos, entre outras regras simples e fáceis de serem memorizadas e seguidas.

É possível, porém, contornar as exceções com algumas linhas de código a mais. No geral nossas aplicações apresentam um código bem simples e enxuto por conta desse conceito.

MVC

A arquitetura principal do Rails é baseada no conceito de separar tudo em três camadas: o **MVC** (*Model, View, Controller*). MVC é um padrão arquitetural onde os limites entre seus modelos, suas lógicas e suas visualizações são bem definidos, sendo muito mais simples fazer um reparo, uma mudança ou uma manutenção, já que essas três partes se comunicam de maneira bem desacoplada.

META-FRAMEWORK

Rails não é baseado num único padrão, mas sim um conjunto de padrões, alguns dos quais discutiremos durante o curso.

Outros frameworks que faziam parte do núcleo do Rails antigamente foram removidos desse núcleo para diminuir o acoplamento com ele e permitir que vocês os substituam sem dificuldade, mas continuam funcionando e sendo usados em conjunto. Aqui estão alguns deles:

- ActionMailer
- ActionPack
 - Action View
 - Action Controller
- ActiveRecord
- ActiveSupport

PROJETOS QUE USAM O RUBY ON RAILS

Há uma extensa lista de aplicações que usam o Ruby on Rails e, entre elas estão o Twitter, YellowPages, Groupon, Typo (blog open source) e o Spokeo (ferramenta de agrupamento de sites de relacionamentos).

Uma lista de sites usando Ruby on Rails com sucesso pode ser encontrada nesses endereços:

- <http://rubyonrails.org/applications>
- <http://www.rubyonrailsgallery.com>
- <http://www.opensourcerails.com>

6.2 AMBIENTE DE DESENVOLVIMENTO

Por ser uma aplicação em Ruby -- contendo arquivos .rb e alguns arquivos de configuração diversos, todos baseados em texto puro -- o Ruby on Rails não requer nenhuma ferramenta avançada para que possamos criar aplicações. Utilizar um editor de textos simples ou uma IDE (*Integrated Development Environment*) cheia de recursos e atalhos é uma decisão de cada desenvolvedor.

Algumas IDEs podem trazer algumas facilidades como execução automática de testes, *syntax highlighting*, integração com diversas ferramentas, wizards, servidores, autocomplete, logs, perspectivas do banco de dados etc. Existem diversas IDEs para trabalhar com Rails, sendo as mais famosas:

- RubyMine - baseada no IntelliJ IDEA
- Aptana Studio 3 - antes conhecida como RadRails, disponível no modelo *stand-alone* ou como plug-in para Eclipse
- Ruby in Steel - para Visual Studio
- NetBeans

Segundo enquetes em listas de discussões, a maioria dos desenvolvedores Ruby on Rails não utiliza nenhuma IDE, apenas um bom **editor de texto** e um pouco de treino para deixá-lo confortável no uso do **terminal de comando** do sistema operacional é suficiente para ser bem produtivo e ainda por cima não depender de um Software grande e complexo para que você possa desenvolver.

Durante nosso curso iremos utilizar o editor de textos padrão do Ubuntu, o GEdit. Alguns outros editores são bem conhecidos na comunidade:

- TextMate - o editor preferido da comunidade Rails, somente para Mac;
- SublimeText 2 - poderoso e flexível, é compatível com plugins do TextMate. Versões para Windows, Mac OS e Linux;

- NotePad++ - famoso entre usuários de Windows, traz alguns recursos interessantes e é bem flexível;
- SciTE - editor simples, compatível com Windows, Mac e Linux;
- Vi / Vim - editor de textos poderoso, pode ser bem difícil para iniciantes. Compatível com Windows, Mac e Linux;
- Emacs - o todo poderoso editor do mundo Linux (também com versões para Windows e Mac);

6.3 CRIANDO UM NOVO PROJETO RAILS

Quando instalamos a gem **rails** em nossa máquina, ganhamos o comando que iremos utilizar para gerar os arquivos iniciais de nossa aplicação. Como a maioria dos comandos de terminal, podemos passar uma série de informações para que o projeto gerado seja customizado.

Para conhecer mais sobre as opções disponíveis, podemos imprimir a ajuda no console. Para isso basta executar o seguinte comando:

```
rails --help
```

Existem algumas opções de ambiente que são muito úteis quando temos diversas versões de Ruby instaladas na mesma máquina e queremos utilizar uma versão específica em nossa aplicação. Também temos uma série de opções para gerar nossa aplicação sem as ferramentas padrão (ou com ferramentas alternativas) para JavaScript, testes, pré-processamento de CSS e etc.

Como o Rails por padrão já traz um *set* de ferramentas bem pensadas para nossa produtividade, podemos nos ater ao comando básico para criação de uma aplicação:

```
rails new [caminho-da-app]
```

Caso a pasta apontada como caminho da aplicação não exista, ela será criada pelo gerador do Rails. Caso a pasta exista e contenha arquivos que possam conflitar com os gerados pelo Rails será necessário informar a ação adequada (sobrescrever, não sobrescrever) para cada um deles durante o processo de geração de arquivos.

```
rails new meu_projeto
```

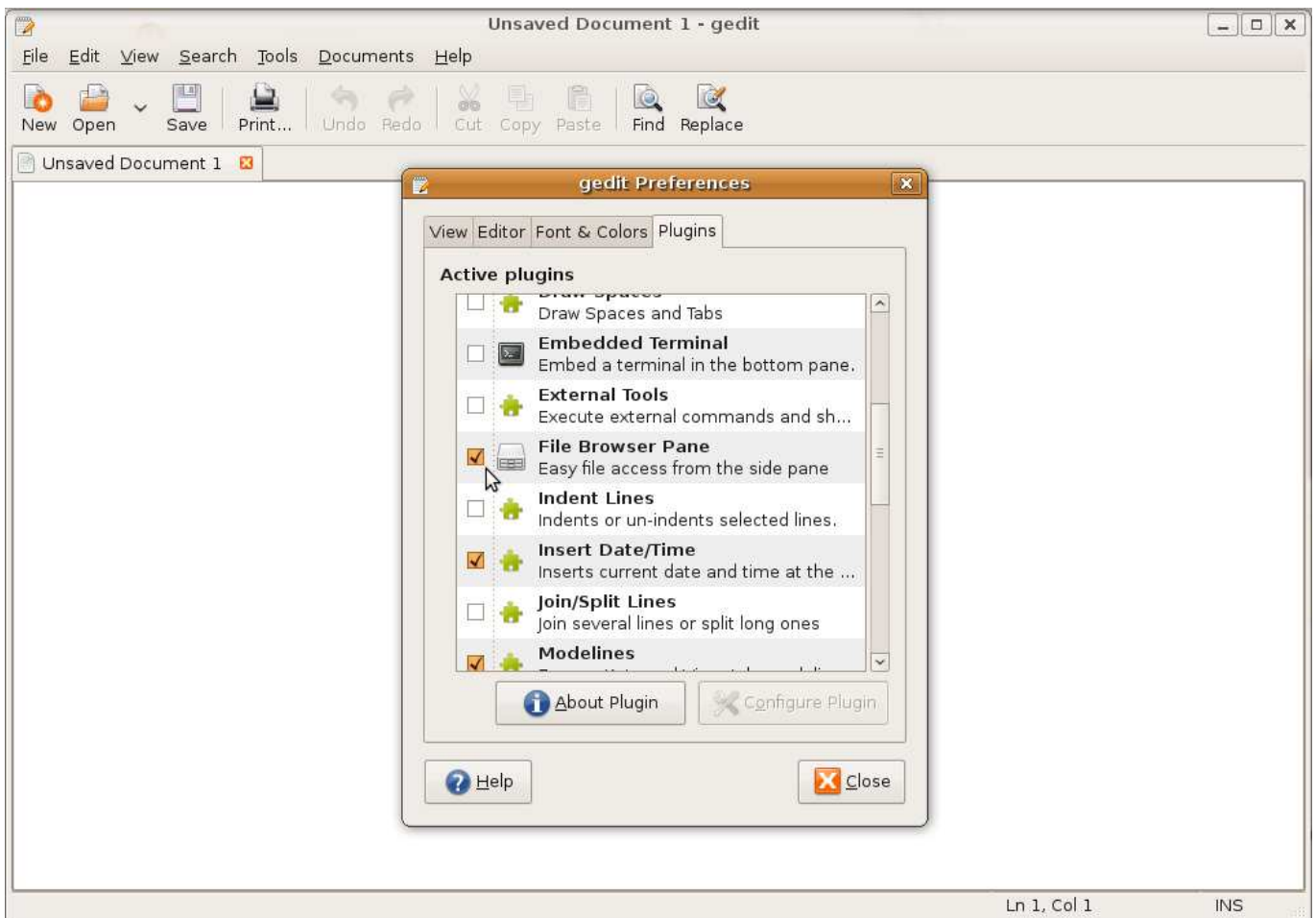
Dentre todas as opções, uma das mais úteis é a que prepara nossa aplicação para conexão com um banco de dados. Como o Rails é compatível com uma grande quantidade de bancos de dados podemos escolher aquele que temos mais familiaridade, ou um que já estiver instalado em nossa máquina. Por padrão o Rails usa o SQLite3 pois é bem simples, as informações podem ser salvas em qualquer local do sistema de arquivos (por padrão dentro da aplicação) e está disponível para Windows, Mac e Linux.

Caso queira utilizar, por exemplo, o MySQL, basta passar a opção:

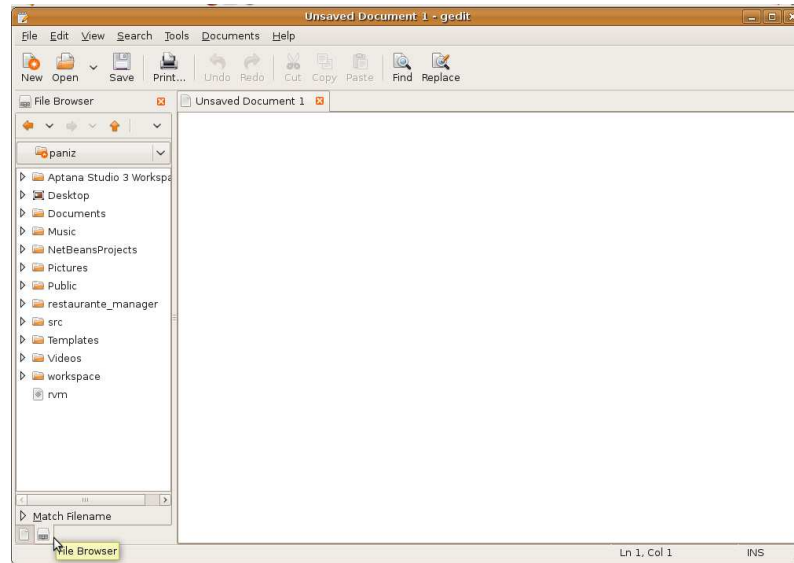
```
rails new meu_projeto -d mysql
```

6.4 EXERCÍCIOS - INICIANDO O PROJETO

- 1) Inicie o GEdit:
 - a) Abra o GEdit através do link “Text Editor” no seu desktop ou barra de ferramentas
 - b) Clique no menu **Edit** e escolher a opção **Preferences**
 - c) Na nova janela clicar na aba **Plugins** e selecionar **File Browser Pane**



- d) Clicar no menu View e então selecionar **Side Pane**



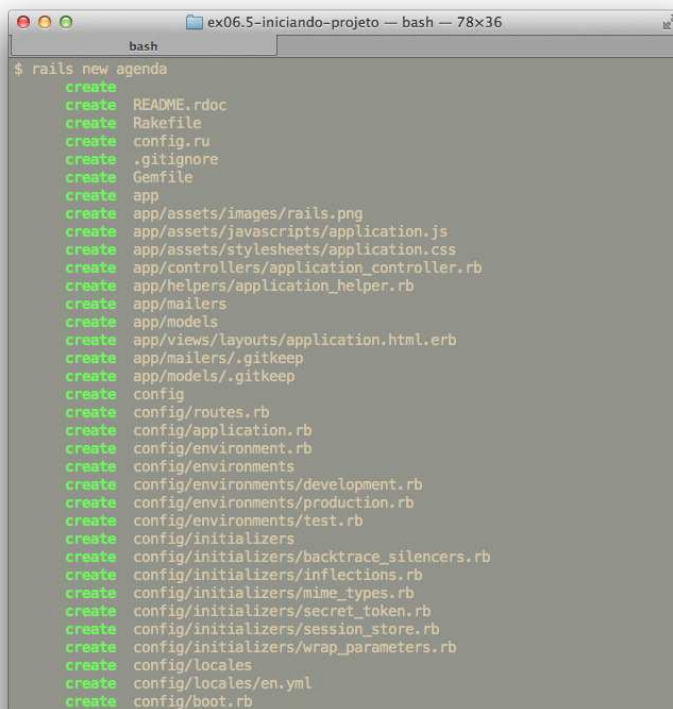
2) Inicie um novo projeto:

a) Abra o terminal

b) Execute o seguinte comando

```
rails new agenda
```

O script informa no terminal quais arquivos foram criados para nossa aplicação e logo na sequência executa o comando para instalação das dependências (bundle install).



```
bash
ex06.5-iniciando-projeto — bash — 78x36

$ rails new agenda
create
create  README.rdoc
create  Rakefile
create  config.ru
create  .gitignore
create  Gemfile
create  app
create  app/assets/images/rails.png
create  app/assets/javascripts/application.js
create  app/assets/stylesheets/application.css
create  app/controllers/application_controller.rb
create  app/helpers/application_helper.rb
create  app/mailers
create  app/models
create  app/views/layouts/application.html.erb
create  app/mailers/.gitkeep
create  app/models/.gitkeep
create  config
create  config/routes.rb
create  config/application.rb
create  config/environment.rb
create  config/environments
create  config/environments/development.rb
create  config/environments/production.rb
create  config/environments/test.rb
create  config/initializers
create  config/initializers/backtrace_silencers.rb
create  config/initializers/inflexions.rb
create  config/initializers/mime_types.rb
create  config/initializers/secret_token.rb
create  config/initializers/session_store.rb
create  config/initializers/wrap_parameters.rb
create  config/locales
create  config/locales/en.yml
create  config/boot.rb
```

6.5 ESTRUTURA DOS DIRETÓRIOS

Cada diretório tem uma função específica e bem clara na aplicação:



- **app** - A maioria dos arquivos específicos da nossa aplicação ficam aqui (inclusive todo o MVC, dividido em diretórios);
- **config** - Configurações da aplicação;

- **db** - Migrações, esquema e outros arquivos relacionados ao banco de dados;
- **doc** - Documentação do sistema;
- **lib** - Bibliotecas auxiliares;
- **log** - Informações de log;
- **public** - Arquivos estáticos que serão servidos pela WEB;
- **test** - Testes da nossa aplicação;
- **tmp** - Arquivos temporários como cache e informações de sessões;
- **vendor** - Dependências e bibliotecas de terceiros.

6.6 O BANCO DE DADOS

Uma das características do Rails é a facilidade de se comunicar com o banco de dados de modo transparente ao programador.

Um exemplo dessa facilidade é que ao gerar a aplicação, ele criou também um arquivo de configuração do banco, pronto para se conectar. O arquivo está localizado em **config/database.yml**. Como o banco de dados padrão é o SQLite3 -- por ser pequeno, versátil e comportar bem um ambiente de desenvolvimento -- a configuração foi gerada para ele. Nada impede, porém, a alteração manual dessa configuração para outros tipos de bancos.

Nesse arquivo, são configuradas três conexões diferentes: *development*, *test* e *production*, associados respectivamente aos bancos *agenda_development*, *agenda_test* e *agenda_production*.

O banco *agenda_development* é usado na fase de desenvolvimento da aplicação e *agenda_production* em produção. A configuração para *agenda_test* se refere ao banco utilizado para os testes que serão executados, e deve ser mantido separado dos outros pois o framework apagará dados e tabelas constantemente.

Apesar de já estarem configuradas as conexões, o gerador da aplicação não cria os bancos de dados automaticamente. Existe um comando para isso:

```
rake db:create
```

A geração não é automática pois em alguns casos é possível que algum detalhe ou outro de configuração no arquivo **config/database.yml** seja necessário.

6.7 EXERCÍCIOS - CRIANDO O BANCO DE DADOS

- 1)
 - Pelo terminal entre no diretório do projeto.

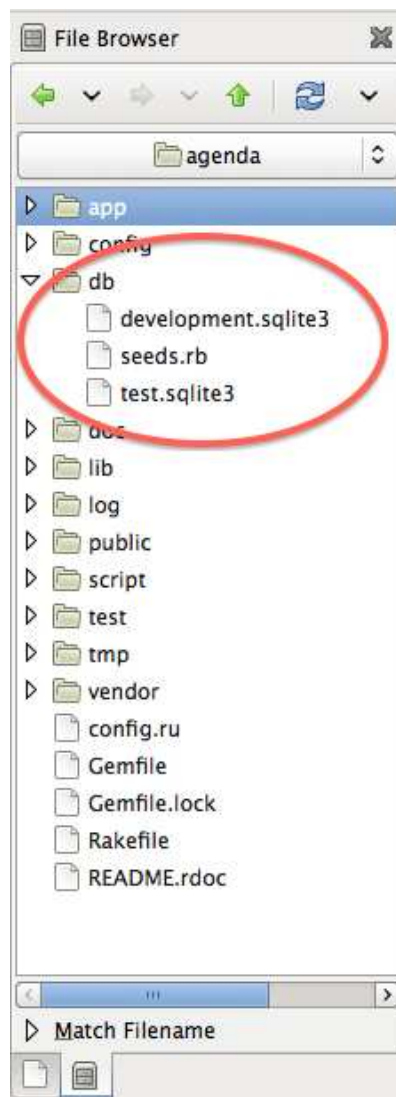
```
cd agenda
```

- Execute o script para criação dos bancos de dados.

```
rake db:create
```

Esse comando não deve exibir nenhuma saída no console, não se preocupe. Porém, se alguma mensagem for exibida, verifique se o comando foi digitado exatamente como demonstrado acima.

- 2) Verifique que as bases de desenvolvimento e teste foram criadas. Pelo explorador de arquivos do GEdit, verifique a pasta **db** da nossa aplicação. O *Rails* cria arquivos com a extensão **.sqlite3** para serem utilizados como base de dados pelo SQLite3, isso permite que você copie o banco de dados para outra máquina sem a necessidade de processos complexos de configuração e importação/exportação de dados.



6.8 A BASE DA CONSTRUÇÃO: SCAFFOLD

Pensando no propósito de facilitar a execução de tarefas repetitivas, o Rails traz um comando que é muito útil nos casos em que precisamos de um cadastro básico de alguma entidade em nossa aplicação -- um CRUD -- que se chama **scaffold**. *Scaffold* pode ser traduzido para o português como “andaime”, ou seja, é uma base, um apoio que facilita a construção de nossa aplicação.

Com esse comando é gerado todo o código necessário para que possamos **listar todos os registros, exibir um registro, criar um novo registro, atualizar um registro e excluir um registro** de determinada entidade em nossa aplicação.

6.9 EXERCÍCIOS - SCAFFOLD

1) Execute o scaffold do seu modelo “Evento”:

a) No terminal, na pasta da aplicação, execute o comando:

```
rails generate scaffold Evento nome:string local:string inicio:datetime  
termino:datetime
```

Veja os diversos arquivos que são criados.

b) Note que também são gerados arquivos para criação da tabela no banco de dados, onde serão armazenados nossos “eventos”, e um arquivo chamado **routes.rb** foi modificado.

c) (opcional) Criamos atributos dos tipos *string*, *datetime*. Quais outros tipos são suportados? Abra a documentação do Rails (<http://api.rubyonrails.org>) e procure pela classe **ActiveRecord::ConnectionAdapters::Table**.

6.10 GERAR AS TABELAS

Juntamente com nossos arquivos foi gerado um arquivo de migração, dentro da pasta **db/migrate**. No cenário mais comum, para cada entidade que criamos é necessário que exista uma tabela no banco de dados, para garantir esse padrão o Rails gera automaticamente um script em Ruby para que as informações no banco de dados sejam consistentes com o que declaramos na aplicação.

Quando executamos o script de *scaffold*, implicitamente declaramos que nossa aplicação tem uma entidade **Evento**, então o Rails gerou o arquivo **db/migrate/<timestamp>_create_eventos.rb** (<timestamp> corresponde ao momento da criação do arquivo) com a definição da nossa tabela eventos e dos campos necessários.

Agora precisamos executar o script de geração das tabelas. Assim como temos um comando *rake* que cria o banco de dados, também temos um comando para criar as tabelas necessárias para nossa aplicação:

```
rake db:migrate
```

O Rails gera para todas as tabelas por padrão o campo `id` como chave primária. Toda a lógica de utilização dessa chave primária é tratada automaticamente para que o desenvolvedor possa se preocupar cada vez menos com o banco de dados e mais com os modelos.

VERSÃO DO BANCO DE DADOS

Ao executarmos a migração do banco de dados pela primeira vez, será criada uma outra tabela chamada **schema_migrations**. Essa tabela contém uma única coluna (`version`), que indica quais foram as migrações executadas até agora, utilizando para isso o timestamp da última migration executada.

DATABASE EVOLUTION

O Rails adota uma estratégia de evolução para o banco de dados, dessa maneira é possível a aplicação evolua gradativamente e o esquema do banco de dados seja incrementado com tabelas e campos em tabelas já existentes conforme o necessário.

Essa característica vai de encontro com as necessidades das metodologias ágeis de desenvolvimento de Software.

6.11 EXERCÍCIOS - MIGRAR TABELA

1) Vamos executar o script de migração de banco de dados para que a tabela “eventos” seja criada:

a) Execute o comando pelo terminal, na pasta da nossa aplicação:

```
rake db:migrate
```

b) O Rails inclui uma maneira facilitada de conectarmos ao console interativo do banco de dados para que possamos inspecionar as tabelas e registros:

```
rails dbconsole
```

c) O comando anterior inicia o console interativo, portanto agora será necessário utilizar a sintaxe do SQLite3 para verificar nossas tabelas. Vamos primeiro listar todas as tabelas do nosso banco de dados de desenvolvimento:

```
sqlite> .tables
```

d) Agora vamos solicitar mais informações sobre a tabela *eventos*:

```
sqlite> PRAGMA table_info(eventos);
```

e) Para finalizar a sessão de console de banco de dados, execute o comando a seguir:

```
sqlite> .quit
```


A sintaxe do console interativo do banco de dados depende do tipo de banco utilizado. Se você escolher, por exemplo, utilizar o MySQL os comandos para listar as tabelas do banco de dados é `show tables;`, e para obter detalhes sobre a tabela é `describe eventos;`.

6.12 SERVER

Agora que já geramos o código necessário em nossa aplicação, criamos o banco de dados e as tabelas necessárias, precisamos colocar nossa aplicação em um servidor de aplicações Web que suporte aplicações feitas com o Ruby on Rails. Só assim os usuários poderão acessar nossa aplicação.

O próprio conjunto de ferramentas embutidas em uma instalação padrão de um ambiente Rails já inclui um servidor simples, suficiente para que o desenvolvedor possa acessar sua aplicação através do *browser*. Esse servidor, o **WEBrick**, não é indicado para aplicações em produção por ser muito simples e não contar com recursos mais avançados necessários para colocar uma aplicação “no ar” para um grande número de usuários.

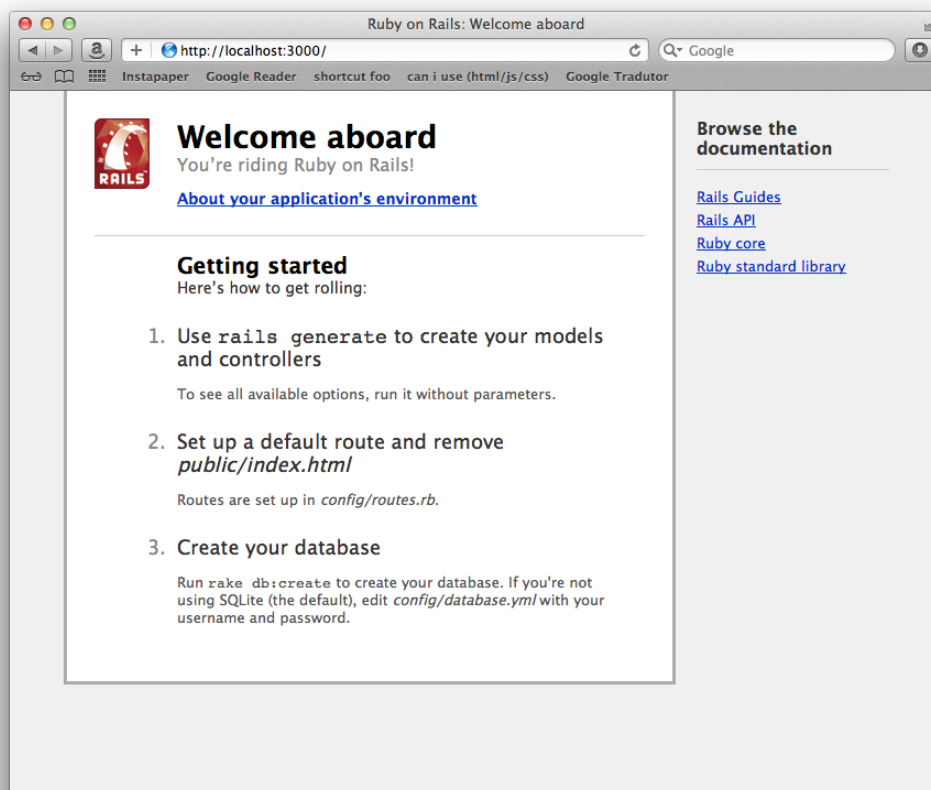
Podemos iniciar o servidor através do comando *rails server* no terminal:

```
rails server
```



```
paniz@caelum131-03: ~/agenda
paniz@caelum131-03:~/agenda$ rails server
=> Booting WEBrick
=> Rails 3.0.0.beta4 application starting in development on http://0.0.0.0:3000
=> Call with -d to detach
=> Ctrl-C to shutdown server
[2010-06-23 05:33:41] INFO  WEBrick 1.3.1
[2010-06-23 05:33:41] INFO  ruby 1.9.1 (2009-01-30) [i686-linux]
[2010-06-23 05:33:47] INFO  WEBrick::HTTPServer#start: pid=11691 port=3000
```

Após o processo de inicialização, podemos acessar nossa aplicação no navegador. Por padrão o servidor disponibiliza nossa aplicação na porta 3000, podemos visitar nossa aplicação em `http://localhost:3000`.



Quando geramos o *scaffold* de eventos, foram geradas telas simples para listar todos registros de eventos, exibir um evento e os formulários para criar um novo evento e editar um evento existente. As telas são simples, mas são totalmente funcionais.

Ao visitar “<http://localhost:3000/eventos>”, você é redirecionado para a listagem de eventos.

6.13 EXERCÍCIOS - INICIANDO O SERVIDOR

- 1) a) Inicie o servidor com o comando:

```
rails server
```

- b) Acesse a aplicação pela url: <http://localhost:3000/eventos>

6.14 DOCUMENTAÇÃO DO RAILS

O Ruby tem uma ferramenta capaz de gerar documentação do nosso código a partir dos comentários que podemos fazer dentro de uma classe ou imediatamente antes das declarações de métodos, essa ferramenta é o RDoc.

A documentação da versão atual do Rails está disponível em **<http://api.rubyonrails.org>**.

Há diversos outros sites que fornecem outras versões da documentação, ou modos alternativos de exibição e organização.

- <http://www.railsapi.com>
- <http://www.gotapi.com>
- <http://apidock.com/rails>

O RubyGems também oferece a documentação para cada uma das gems que estão instaladas, para isso basta iniciar o servidor de documentação embutido:

```
gem server
```

Este comando inicia um servidor WebRick na porta 8808. Basta acessar pelo browser para ver o RDoc de todos os gems instalados.

A última alternativa é gerar a documentação através do rake:

```
rails docs  
rake doc:rails
```

Depois de executar a task `doc:rails`, a documentação estará disponível no diretório `docs/api/`. A documentação do Ruby (bem como a biblioteca padrão) pode ser encontrada em **<http://ruby-doc.org>**.

Existem ainda excelentes guias e tutoriais oficiais disponíveis. É **obrigatório** para todo desenvolvedor Rails passar por estes guias: **<http://guides.rubyonrails.org>**.

Além desses guias, existe um site que lista as principais gems que podem ser usadas para cada tarefa: **<http://ruby-toolbox.com/>**.

A comunidade Ruby e Rails também costuma publicar excelentes *screencasts* (vídeos) com aulas e/ou palestras sobre diversos assuntos relacionados a Ruby e Rails. A lista está sempre crescendo. Aqui estão alguns dos principais:

- **<http://railscasts.com>** - Vídeos pequenos abordando algumas práticas e ferramentas interessantes, mantidos por Ryan Bates. Alguns vídeos são gratuitos.
- **<http://peepcode.com>** - Verdadeiras vídeo-aulas, algumas chegam a 2h de duração. São pagos, cerca de US\$ 9,00 por vídeo.
- **<http://www.confreaks.com>** - Famosos por gravar diversas conferências sobre Ruby e Rails.

6.15 EXERCÍCIO OPCIONAL - UTILIZANDO A DOCUMENTAÇÃO

No formulário de cadastro de eventos podemos verificar que, apesar de já criar os formulários com os componentes adequados para que o usuário possa escolher uma data correta, a lista de anos disponíveis exibe somente alguns anos recentes.

Um dos requisitos possíveis para nossa aplicação é que o ano inicial do evento seja no mínimo o ano atual, para evitar que algum usuário cadastre um evento com uma data de fim no passado.

Para fazer essa alteração na aplicação será preciso editar o código gerado pelo *Rails* responsável por exibir a lista de anos, comece a listar a partir do ano atual:

- 1) Verifique no arquivo `app/views/eventos/_form.html.erb` um método chamado `datetime_select`:

```
<%= f.datetime_select :termino %>
```

Encontre a documentação desse método (em **ActionViewHelpersDateHelper**). Como podemos indicar que queremos exibir anos a partir do ano atual?

```
<%= f.datetime_select :termino, start_year: 2012 %>
```

CAPÍTULO 7

Active Record

“Não se deseja aquilo que não se conhece”

– Ovídio

Nesse capítulo começaremos a desenvolver um sistema utilizando Ruby on Rails com recursos mais avançados.

7.1 MOTIVAÇÃO

Queremos criar um sistema de qualificação de restaurantes. Esse sistema terá clientes que qualificam os restaurantes visitados com uma nota, além de informar quanto dinheiro gastaram. Os clientes terão a possibilidade de deixar comentários para as qualificações feitas por eles mesmos ou a restaurantes ainda não visitados. Além disso, os restaurantes terão pratos, e cada prato a sua receita.

O site <http://www.tripadvisor.com> possui um sistema similar para viagens, onde cada cliente coloca comentários sobre hotéis e suas visitas feitas no mundo inteiro.

7.2 EXERCÍCIOS: CONTROLE DE RESTAURANTES

- 1) Crie um novo projeto chamado `vota_prato`:
 - a) No terminal, garanta que não está no diretoria do projeto anterior.
 - b) Digite o comando `rails new vota_prato -d mysql`
 - c) Observe o log de criação do projeto:



```
rr71@caelum131-03:~$ rails new vota_prato -d mysql
create
create  README
create  Rakefile
create  config.ru
create  .gitignore
create  Gemfile
create  app
create  app/views/layouts/application.html.erb
create  app/controllers/application_controller.rb
create  app/helpers/application_helper.rb
create  app/models
create  config
create  config/routes.rb
create  config/application.rb
create  config/environment.rb
create  config/environments
create  config/environments/production.rb
create  config/environments/test.rb
create  config/environments/development.rb
create  config/initializers
create  config/initializers/session_store.rb
create  config/initializers/backtrace_silencers.rb
create  config/initializers/secret_token.rb
create  config/initializers/mime_types.rb
create  config/initializers/inflections.rb
create  config/locales
create  config/locales/en.yml
create  config/boot.rb
create  config/database.yml
create  db
create  db/seeds.rb
create  doc
create  doc/README_FOR_APP
create  lib
create  lib/tasks
create  lib/tasks/.gitkeep
create  log
create  log/server.log
create  log/production.log
create  log/development.log
```

7.3 MODELO - O “M” DO MVC

Models são os modelos que serão usados nos sistemas: são as entidades que serão armazenadas em um banco. No nosso sistema teremos modelos para representar um **Cliente**, um **Restaurante** e uma **Qualificação**, por exemplo.

O componente de Modelo do Rails é um conjunto de classes que usam o ActiveRecord, uma classe ORM que mapeia objetos em tabelas do banco de dados. O ActiveRecord usa convenções de nome para determinar os mapeamentos, utilizando uma série de regras que devem ser seguidas para que a configuração seja a mínima possível.

ORM

ORM (*Object-Relational Mapping*) é um conjunto de técnicas para a transformação entre os modelos orientado a objetos e relacional.

7.4 ACTIVERECORD

É um framework que implementa o acesso ao banco de dados de forma transparente ao usuário, funcionando como um Wrapper para seu modelo. Utilizando o conceito de *Conventions over Configuration*, o ActiveRecord adiciona aos seus modelos as funções necessárias para acessar o banco.

`ActiveRecord::Base` é a classe que você deve estender para associar seu modelo com a tabela no Banco de Dados.

7.5 RAKE

Rake é uma ferramenta de build, escrita em Ruby, e semelhante ao **make** e ao **ant**, em escopo e propósito.

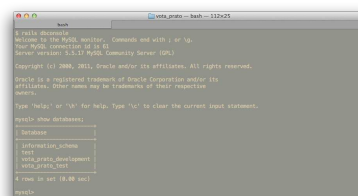
Rake tem as seguintes funcionalidades:

- Rakefiles (versão do rake para os Makefiles) são completamente definidas em sintaxe Ruby. Não existem arquivos XML para editar, nem sintaxe rebuscada como a do Makefile para se preocupar.
- É possível especificar tarefas com pré-requisitos.
- Listas de arquivos flexíveis que agem como arrays, mas sabem como manipular nomes de arquivos e caminhos (paths).
- Uma biblioteca de tarefas pré-compactadas para construir rakefiles mais facilmente.

Para criar nossas bases de dados, podemos utilizar a rake task **db:create**. Para isso, vá ao terminal e dentro do diretório do projeto digite:

```
$ rake db:create
```

O *Rails* criará dois databases no mysql: `vota_prato_development`, `vota_prato_test`.

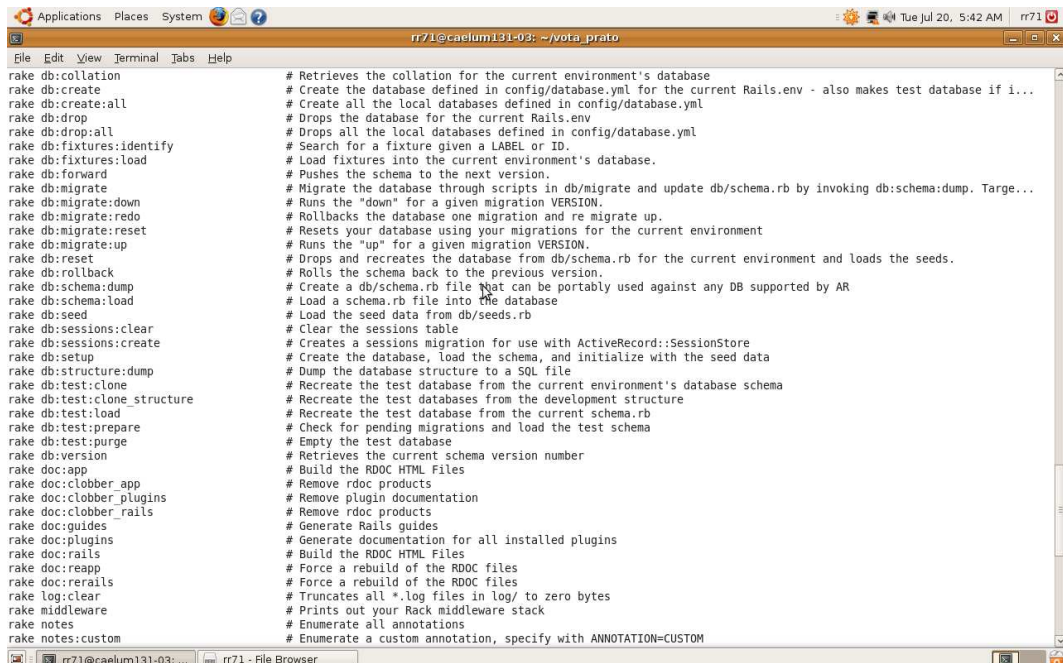


RAKE DB:CREATE:ALL

Uma outra tarefa disponível em uma aplicação *Rails* é a rake `db:create:all`. Além de criar os dois databases já citados, ela também é responsável por criar o database `vota_prato_production`. No ambiente de desenvolvimento, é bem comum trabalharmos apenas com os databases de teste e desenvolvimento.

Para ver todas as tasks rake disponíveis no seu projeto podemos usar o comando (na raiz do seu projeto):

```
$ rake -T
```



```
File Edit View Terminal Tabs Help
rr71@caelum131-03: ~/vota_prato

rake db:collation      # Retrieves the collation for the current environment's database
rake db:create         # Create the database defined in config/database.yml for the current Rails.env - also makes test database if i...
rake db:create:all     # Create all the local databases defined in config/database.yml
rake db:drop           # Drops the database for the current Rails.env
rake db:drop:all       # Drops all the local databases defined in config/database.yml
rake db:fixtures:identify # Search for a fixture given a LABEL or ID.
rake db:fixtures:load  # Load fixtures into the current environment's database.
rake db:forward        # Pushes the schema to the next version.
rake db:migrate        # Migrate the database through scripts in db/migrate and update db/schema.rb by invoking db:schema:dump. Targe...
rake db:migrate:down   # Runs the "down" for a given migration VERSION.
rake db:migrate:redo   # Rollbacks the database one migration and re migrate up.
rake db:migrate:reset  # Resets your database using your migrations for the current environment
rake db:migrate:up     # Runs the "up" for a given migration VERSION.
rake db:reset         # Drops and recreates the database from db/schema.rb for the current environment and loads the seeds.
rake db:rollback       # Rolls the schema back to the previous version.
rake db:schema:dump    # Create a db/schema.rb file that can be portably used against any DB supported by AR
rake db:schema:load    # Load a schema.rb file into the database
rake db:seed          # Load the seed data from db/seeds.rb
rake db:sessions:clear # Clear the sessions table
rake db:sessions:create # Creates a sessions migration for use with ActiveRecord::SessionStore
rake db:setup         # Create the database, load the schema, and initialize with the seed data
rake db:structure:dump # Dump the database structure to a SQL file
rake db:test:clone     # Recreate the test database from the current environment's database schema
rake db:test:clone_structure # Recreate the test databases from the development structure
rake db:test:load      # Recreate the test database from the current schema.rb
rake db:test:prepare   # Check for pending migrations and load the test schema
rake db:test:purge     # Empty the test database
rake db:version        # Retrieves the current schema version number
rake doc:app           # Build the RDOC HTML Files
rake doc:clobber_app   # Remove rdoc products
rake doc:clobber_plugins # Remove plugin documentation
rake doc:clobber_rails # Remove rdoc products
rake doc:guides        # Generate Rails guides
rake doc:plugins       # Generate documentation for all installed plugins
rake doc:rails         # Build the RDOC HTML Files
rake doc:reapp         # Force a rebuild of the RDOC files
rake doc:reraails      # Force a rebuild of the RDOC files
rake log:clear         # Truncates all *.log files in log/ to zero bytes
rake notes             # Prints out your Rack middleware stack
rake notes:custom      # Enumerate all annotations
rake notes:custom      # Enumerate a custom annotation, specify with ANNOTATION=CUSTOM
```

7.6 CRIANDO MODELOS

Agora vamos criar o modelo do Restaurante. Para isso, temos um gerador específico para **model** através do comando `rails generate model restaurante`.

Repare que o Rails gerou uma série de arquivos para nós.


```
rr71@c
File Edit View Terminal Tabs Help
rr71@caelum131-03:~/vota_prato$ rails generate model restaurante
  invoke  active_record
  create  db/migrate/20100720084332_create_restaurantes.rb
  create  app/models/restaurante.rb
  invoke  test_unit
  create  test/unit/restaurante_test.rb
  create  test/fixtures/restaurantes.yml
rr71@caelum131-03:~/vota_prato$ rake db:create:all
(in /home/rr71/vota_prato)
vota_prato_development already exists
vota_prato_test already exists
vota_prato_production already exists
rr71@caelum131-03:~/vota_prato$
```

7.7 MIGRATIONS

Migrations ajudam a gerenciar a evolução de um esquema utilizado por diversos bancos de dados. Foi a solução encontrada para o problema de como adicionar uma coluna no banco de dados local e propagar essa mudança para os demais desenvolvedores de um projeto e para o servidor de produção.

Com as migrations, podemos descrever essas transformações em classes que podem ser controladas por sistemas de controle de versão (por exemplo, git) e executá-las em diversos bancos de dados.

Sempre que executarmos a tarefa *Generator -> model*, o Rails se encarrega de criar uma migration inicial, localizado em **db/migrate**.

`ActiveRecord::Migration` é a classe que você deve estender ao criar uma migration.

Quando geramos nosso modelo na seção anterior, Rails gerou para nós uma migration (**db/migrate/<timestamp>_create_restaurantes.rb**). Vamos agora editar nossa migration com as informações que queremos no banco de dados.

Queremos que nosso restaurante tenha um nome e um endereço. Para isso, devemos acrescentar as chamadas de método abaixo:

```
t.string :nome, limit: 80
t.string :endereco
```

Faça isso dentro do método `change` da classe `CreateRestaurantes`. Sua migration deverá ficar como a seguir:

```
class CreateRestaurantes < ActiveRecord::Migration
  def change
    create_table :restaurantes do |t|
      t.string :nome, limit: 80
      t.string :endereco
      t.timestamps
    end
  end
end
```

```
end  
end
```

Supondo que agora lembramos de adicionar a especialidade do restaurante. Como fazer? Basta usar o outro gerador (*Generator*) do rails que cria migration. Por exemplo:

```
$ rails generate migration add_column_especialidade_to_restaurante especialidade
```

Um novo arquivo chamado <timestamp>_add_column_especialidade_to_restaurante.rb será criado pelo *Rails*, e o código da migração gerada será como a seguir:

```
class AddColumnEspecialidadeToRestaurante < ActiveRecord::Migration  
  def change  
    add_column :restaurantes, :especialidade, :string  
  end  
end
```

Note que através do nome da migração e do parâmetro que passamos no gerador, o nome da coluna que queremos adicionar, nesse caso **especialidade**, o *Rails* deduziu que a migração é para adicionar a coluna e já fez o trabalho braçal para você!

Vamos apenas adicionar mais alguns detalhes a essa migração, vamos limitar o tamanho da string que será armazenada nesse campo para 40 caracteres:

```
class AddColumnEspecialidadeToRestaurante < ActiveRecord::Migration  
  def change  
    add_column :restaurantes, :especialidade, :string, limit: 40  
  end  
end
```

7.8 EXERCÍCIOS: CRIANDO OS MODELOS

1) Crie nosso banco de dados

a) Entre no terminal, no diretório do projeto

b) execute o comando:

```
$ rake db:create
```

No terminal, acesse o mysql e verifique que os databases foram criados:

```
$ rails dbconsole  
mysql> show databases;  
mysql> quit
```

2) Crie o modelo do Restaurante

a) Novamente no Terminal, no diretório do projeto

b) execute **rails generate model restaurante**

3) Edite seu script de migração do modelo “restaurante” para criar os campos nome e endereço:

a) Abra o arquivo db/migrate/<timestamp>_create_restaurantes.rb

b) Adicione as linhas:

```
t.string :nome, limit: 80
t.string :endereço
```

O código final de sua migration deverá ser como o que segue:

```
class CreateRestaurantes < ActiveRecord::Migration
  def change
    create_table :restaurantes do |t|
      t.string :nome, limit: 80
      t.string :endereço
      t.timestamps
    end
  end
end
```

4) Migre as tabelas para o banco de dados:

a) Vá ao Terminal

b) Execute a tarefa db:migrate:

```
$ rake db:migrate
```

Verifique no banco de dados se as tabelas foram criadas:

```
$ rails dbconsole
mysql> use vota_prato_development;
mysql> desc restaurantes;
mysql> quit
```

5) Adicione a coluna especialidade ao nosso modelo “restaurante”:

a) Vá novamente ao Terminal e digite:

```
$ rails generate migration add_column_especialidade_to_restaurante especialidade
```

b) Abra o arquivo db/migrate/<timestamp>_add_column_especialidade_to_restaurante.rb

c) Altere o limite de caracteres da coluna especialidade como a seguir:

```
add_column :restaurantes, :especialidade, :string, limit: 40
```

Seu arquivo ficará assim:

```
class AddColumnEspecialidadeToRestaurante < ActiveRecord::Migration
  def change
    add_column :restaurantes, :especialidade, :string, limit: 40
  end
end
```

d) Para efetivar a mudança no banco de dados execute a seguinte tarefa:

```
rake db:migrate
```

A saída do comando será semelhante a seguinte:

```
== CreateRestaurantes: migrating =====
-- create_table(:restaurantes)
   -> 0.7023s
== CreateRestaurantes: migrated (0.7025s) =====

== AddColumnEspecialidadeToRestaurante: migrating =====
-- add_column(:restaurantes, :especialidade, :string, {:limit=>40})
   -> 0.9402s
== AddColumnEspecialidadeToRestaurante: migrated (0.9404s) =====
```

e) Olhe novamente no banco de dados, veja que as tabelas foram realmente criadas.

```
$ rails dbconsole
mysql> use vota_prato_development;
mysql> show tables;
```

E o resultado será semelhante ao seguinte:

```
| Tables_in_vota_prato_development |
+-----+
| restaurantes                      |
| schema_migrations                 |
+-----+

2 rows in set (0.00 sec)
```

6) (Opcional) Utilizamos o método **add_column** na nossa *migration* para adicionar uma nova coluna. O que mais poderia ser feito? Abra a documentação e procure pelo módulo **ActiveRecordConnectionAdaptersSchemaStatements**.

7.9 MANIPULANDO NOSSOS MODELOS PELO CONSOLE

Podemos utilizar o console para escrever comandos Ruby, e testar nosso modelo. A grande vantagem disso, é que não precisamos de controladores ou de uma view para testar se nosso modelo funciona de acordo com

o esperado e se nossas regras de validação estão funcionando. Outra grande vantagem está no fato de que se precisarmos manipular nosso banco de dados, ao invés de termos de conhecer a sintaxe sql e digitar a query manualmente, podemos utilizar código ruby e manipular através do nosso console.

Para criar um novo restaurante, podemos utilizar qualquer um dos jeitos abaixo:

```
r = Restaurante.new
r.nome = "Fasano"
r.endereco = "Av. dos Restaurantes, 126"
r.especialidade = "Comida Italiana"
r.save

r = Restaurante.new do |r|
  r.nome = "Fasano"
  r.endereco = "Av. dos Restaurantes, 126"
  r.especialidade = "Comida Italiana"
end
r.save
```

Uma outra forma possível para a criação de objetos do Active Record é usando um *hash* como parâmetro no construtor. As chaves do hash precisam coincidir com nomes das propriedades as quais queremos atribuir os valores. Veja o exemplo a seguir:

```
r = Restaurante.new nome: "Fasano",
                    endereco: "Av. dos Restaurantes, 126",
                    especialidade: "Comida Italiana"
r.save
```

Porém o Active Record tem uma restrição quanto ao uso dessa funcionalidade. É preciso que você especifique exatamente quais são as propriedades que podem ter seu valor atribuído a partir do *hash*.

É preciso dizer isso explicitamente através da invocação do método `attr_accessible`. Vamos editar a classe `Restaurante` para que o código fique como o seguinte:

```
class Restaurante < ActiveRecord::Base
  attr_accessible :nome, :endereco, :especialidade
end
```

Pronto! Agora você já pode criar objetos com as propriedades populadas através de um *hash*. Além disso, também pode invocar um método de classe em `Restaurante` que cria um objeto e já armazena os dados no banco, sem precisar da invocação ao método `save`, veja:

```
Restaurante.create nome: "Fasano",
                  endereco: "Av. dos Restaurantes, 126",
                  especialidade: "Comida Italiana"
```

MASS ASSIGNMENT

Essa ideia de precisar especificar quais campos podem ser atribuídos através de um *hash* é referida por muitos como “mass assignment whitelist”. A ideia é que o programador deve decidir quais propriedades de um objeto podem ser atribuídas diretamente.

Em versões anteriores do framework, toda propriedade era passível de atribuição, a não ser que o programador se lembrasse de adicionar uma invocação ao método `attr_accessible`. O problema disso é que, se por algum descuido, o programador esquecer de estabelecer exatamente quais propriedades estão abertas a atribuição, algumas falhas de segurança podem ocorrer.

Atualmente o padrão é justamente o inverso. Nenhuma propriedade pode ser atribuída diretamente a partir dos valores de um *hash*, caso queira isso, é preciso especificar através da invocação do `attr_accessible`.

Você pode ler mais a respeito nesse post: <http://blog.caelum.com.br/seguranca-de-sua-aplicacao-e-os-frameworks-ataque-ao-github/>

Note que o comando `save` efetua a seguinte ação: se o registro não existe no banco de dados, cria um novo registro; se já existe, atualiza o registro existente.

Existe também o comando `save!`, que tenta salvar o registro, mas ao invés de apenas retornar “**false**” se não conseguir, lança a exceção `RecordNotSaved`.

Para atualizar um registro diretamente no banco de dados, podemos fazer:

```
Restaurante.update(1, {nome: "1900"})
```

Para atualizar múltiplos registros no banco de dados:

```
Restaurante.update_all("especialidade = 'Massas'")
```

Ou, dado um objeto `r` do tipo `Restaurante`, podemos utilizar:

```
r.update_attribute(:nome, "1900")
```

```
r.update_attributes nome: "1900", especialidade: "Pizzas"
```

Existe ainda o comando `update_attributes!`, que chama o comando `save!` ao invés do comando `save` na hora de salvar a alteração.

Para remover um restaurante, também existem algumas opções. Todo `ActiveRecord` possui o método `destroy`:

```
restaurante = Restaurante.first  
restaurante.destroy
```

Para remover o restaurante de id **1**:

```
Restaurante.destroy(1)
```

Para remover os restaurantes de ids **1, 2 e 3**:

```
restaurantes = [1,2,3]  
Restaurante.destroy(restaurantes)
```

Para remover **todos** os restaurantes:

```
Restaurante.destroy_all
```

Podemos ainda remover todos os restaurantes que obedeçam determinada condição, por exemplo:

```
Restaurante.destroy_all(especialidade: "italiana")
```

Os métodos destroy sempre fazem primeiro o find(id) para depois fazer o destroy(). Se for necessário evitar o SELECT antes do DELETE, podemos usar o método delete():

```
Restaurante.delete(1)
```

7.10 EXERCÍCIOS: MANIPULANDO REGISTROS

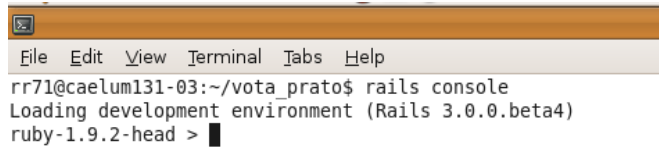
Teste a manipulação de registros pelo console.

- 1) Edite o arquivo localizado em app/models/restaurante.rb. Na declaração da classe invoque o método attr_accessible como no exemplo abaixo:

```
class Restaurante < ActiveRecord::Base  
  attr_accessible :nome, :endereco, :especialidade  
end
```

- 2) Insira um novo restaurante

- a) Para ter acesso ao Console, basta digitar **rails console** no Terminal



```
File Edit View Terminal Tabs Help
rr71@caelum131-03:~/vota_prato$ rails console
Loading development environment (Rails 3.0.0.beta4)
ruby-1.9.2-head > █
```

b) Digite:

```
r = Restaurante.new nome: "Fasano",
                    endereco: "Av. dos Restaurantes, 126",
                    especialidade: "Comida Italiana"
```

c) Olhe seu banco de dados:

```
$ rails dbconsole
mysql> select * from restaurantes;
```

d) Volte para o Console e digite:

```
r.save
```

e) Olhe seu banco de dados novamente:

```
$ rails dbconsole
mysql> select * from restaurantes;
```

3) Atualize seu restaurante

a) Digite:

```
r.update_attributes nome: "1900"
```

b) Olhe seu banco de dados novamente:

```
$ rails dbconsole
mysql> select * from restaurantes;
```

4) Vamos remover o restaurante criado:

a) Digite

```
Restaurante.destroy(1)
```

b) Olhe seu banco de dados e veja que o restaurante foi removido

```
$ rails dbconsole
mysql> select * from restaurantes;
```


7.11 EXERCÍCIOS OPCIONAIS

- 1) Teste outras maneiras de efetuar as operações do exercício anterior.

7.12 FINDERS

O ActiveRecord possui o método "**find**" para realizar buscas. Esse método, aceita os seguintes parâmetros:

```
Restaurante.all    # retorna todos os registros
Restaurante.first  # retorna o primeiro registro
Restaurante.last   # retorna o último registro
```

Ainda podemos passar para o método find uma lista com os id's dos registros que desejamos:

```
r = Restaurante.find(1)
varios = Restaurante.find(1,2,3)
```

Além desses, podemos definir condições para nossa busca (como o SELECT do MySQL). Existem diversas formas de declararmos essas condições:

```
Restaurante.where("nome = 'Fasano' and especialidade = 'massa'")
Restaurante.where(["nome = ? and especialidade = ?",
  'Fasano', 'massa'])
Restaurante.where(["nome = :nome and especialidade = :especialidade", {
  nome: "Fasano", especialidade: "Massa"}])
Restaurante.where({nome: "Fasano", especialidade: "massa" })
```

Essas quatro formas fazem a mesma coisa. Procuram por registros com o campo nome = "Fasano" e o campo especialidade = "massa".

Existem ainda os chamados dynamic finders:

```
Restaurante.where(["nome = ? AND especialidade = ?",
  "Fasano", "italiana"])
```

poderia ser escrito como:

```
find_all_by_nome_and_especialidade("Fasano", "italiana")
```

Temos ainda o "**find_or_create_by**", que retorna um objeto se ele existir, caso contrário, cria o objeto no banco de dados e retorna-o:

```
Restaurante.find_or_create_by_nome("Fasano")
```

Para finalizar, podemos chamar outros métodos encadeados para fazer queries mais complexas:

- `.order` - define a ordenação. Ex: “created_at DESC, nome”.
- `.group` - nome do atributo pelo qual os resultados serão agrupados. Efeito idêntico ao do comando SQL GROUP BY.
- `.limit` - determina o limite do número de registros que devem ser retornados
- `.offset` - determina o ponto de início da busca. Ex: para `offset = 5`, iria pular os registros de 0 a 4.
- `.include` - permite carregar relacionamentos na mesma consulta usando LEFT OUTER JOINS.

Exemplo mais completo:

```
Restaurante.where('nome like :nome', {nome: '%teste%'}).  
  order('nome DESC').limit(20)
```

PARA SABE MAIS - OUTRAS OPÇÕES PARA OS FINDERS

Existem mais opções, como o `lock`, que podem ser utilizadas, mas não serão abordadas nesse curso. Você pode consultá-las na documentação da API do Ruby on Rails.

7.13 EXERCÍCIOS: BUSCAS DINÂMICAS

1) Vamos testar os métodos de busca:

a) Abra o console (`rails console` no Terminal)

b) Digite:

```
Restaurante.first
```

c) Aperte **enter**

d) Digite:

```
Restaurante.all
```

e) Aperte **enter**

f) Digite:

```
Restaurante.find(1)
```

g) Aperte **enter**

h) Digite:

```
Restaurante.where(["nome = ? AND especialidade = ?",  
                  "Fasano", "Comida Italiana"])
```

i) Aperte **enter**

j) Digite:

```
Restaurante.find_all_by_nome_and_especialidade("Fasano",  
                                                "Comida Italiana")
```

k) Aperte **enter**

l) Digite:

```
Restaurante.order("especialidade DESC").limit(1)
```

m) Aperte **enter**

7.14 VALIDAÇÕES

Ao inserir um registro no banco de dados é bem comum a entrada de dados inválidos.

Existem alguns campos de preenchimento obrigatório, outros que só aceitem números, que não podem conter dados já existentes, tamanho máximo e mínimo etc.

Para ter certeza que um campo foi preenchido antes de salvar no banco de dados, é necessário pensar em três coisas: “como validar a entrada?”, “qual o campo a ser validado?” e “o que acontece ao tentar salvar uma entrada inválida?”.

Para validar esses registros, podemos implementar o método `validate` em qualquer ActiveRecord, porém o Rails disponibiliza alguns comandos prontos para as validações mais comuns. São eles:

- `validates_presence_of`: verifica se um campo está preenchido;
- `validates_size_of`: verifica o comprimento do texto do campo;
- `validates_uniqueness_of`: verifica se não existe outro registro no banco de dados que tenha a mesma informação num determinado campo;
- `validates_numericality_of`: verifica se o preenchimento do campo é numérico;
- `validates_associated`: verifica se o relacionamento foi feito corretamente;
- etc...

Todos estes métodos disponibilizam uma opção (**:message**) para personalizar a mensagem de erro que será exibida caso a regra não seja cumprida. Caso essa opção não seja utilizada, será exibida uma mensagem padrão.

Toda mensagem de erro é gravada num hash chamado `errors`, presente em todo `ActiveRecord`.

Além dos validadores disponibilizados pelo rails, podemos utilizar um validador próprio:

```
validate :garante_alguma_coisa

def garante_alguma_coisa
  errors.add_to_base("Deve respeitar nossa regra") unless campo_valido?
end
```

Repare que aqui, temos que incluir manualmente a mensagem de erro padrão do nosso validador.

Se quisermos que o nome do nosso restaurante comece com letra maiúscula, poderíamos fazer:

```
validate :primeira_letra_deve_ser_maiuscula

private
def primeira_letra_deve_ser_maiuscula
  errors.add("nome",
    "primeira letra deve ser maiúscula") unless nome =~ /[A-Z].*/
end
```

Além dessas alternativas, o Rails 3 trouxe uma nova maneira de criar validações que facilita os casos onde temos vários validadores para o mesmo campo, como por exemplo:

```
validates :nome, presence: true, uniqueness: true, length: {maximum: 50}
```

equivalente a:

```
validates_presence_of :nome
validates_uniqueness_of :nome
validates_length_of :nome, :maximum => 50
```

MODIFICADORES DE ACESSO

Utilizamos aqui, o modificador de acesso **private**. A partir do ponto que ele é declarado, todos os métodos daquela classe serão privados, a menos que tenha um outro modificador de acesso que modifique o acesso a outros métodos.

VALIDADORES PRONTOS

Esse exemplo poderia ter sido reescrito utilizando o validador "**validates_format_of**", que verifica se um atributo confere com uma determinada expressão regular.

ENCODING DE ARQUIVOS NO RUBY E AS CLASSES DE MODEL

Devido ao encoding dos arquivos no Ruby 1.9 ser como padrão ASCII-8BIT, ao utilizarmos algum carácter acentuado no código fonte, é necessário indicarmos um encoding que suporte essa acentuação. Geralmente, esse encoding será o UTF-8. Portanto, temos que adicionar no começo do arquivo fonte a linha:

```
#encoding: utf-8
```

7.15 EXERCÍCIOS: VALIDAÇÕES

- 1) Para nosso restaurante implementaremos a validação para que o campo nome, endereço e especialidade não possam ficar vazios, nem que o sistema aceite dois restaurantes com o mesmo nome e endereço.

- a) Abra o modelo do restaurante (**app/models/restaurante.rb**)

- b) inclua as validações:

```
validates_presence_of :nome, message: "deve ser preenchido"  
validates_presence_of :endereco, message: "deve ser preenchido"  
validates_presence_of :especialidade, message: "deve ser preenchido"
```

```
validates_uniqueness_of :nome, message: "nome já cadastrado"  
validates_uniqueness_of :endereco, message: "endereço já cadastrado"
```

- c) Por utilizarmos acentuação em nosso arquivo de modelo, adicione a diretiva (comentário) que indica qual o encoding em que o Ruby deve interpretar **no início do seu arquivo**.

```
#encoding: utf-8
```

- 2) Inclua a validação da primeira letra maiúscula:

```
validate :primeira_letra_deve_ser_maiuscula
```

```
private  
def primeira_letra_deve_ser_maiuscula  
  errors.add(:nome,  
    "primeira letra deve ser maiúscula") unless nome =~ /[A-Z].*/  
end
```

- 3) Agora vamos testar nossos validadores:

- a) Abra o Terminal

b) Entre no Console (rails console)

c) Digite:

```
r = Restaurante.new nome: "fasano",  
  endereco: "Av. dos Restaurantes, 126"  
r.save
```

d) Verifique a lista de erros, digitando:

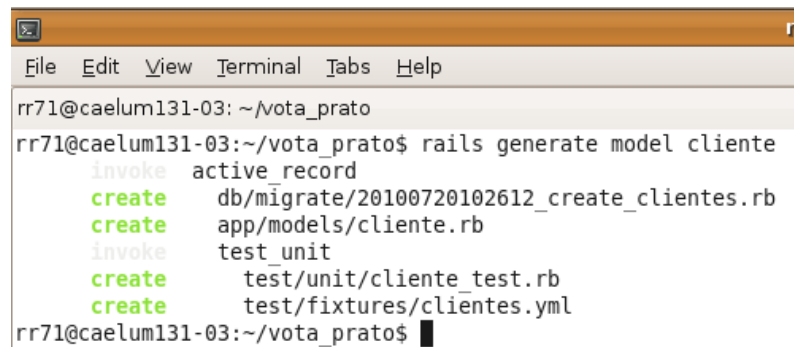
```
r.valid?           # verifica se objeto passa nas validações  
r.errors.empty?    # retorna true/false indicando se há erros ou não  
r.errors.count     # retorna o número de erros  
r.errors[:nome]    # retorna apenas o erro do atributo nome  
  
r.errors.each {|field, msg| puts "#{field} - #{msg}"}
```

7.16 EXERCÍCIOS - COMPLETANDO NOSSO MODELO

1) Vamos criar o nosso modelo Cliente, bem como sua migration:

a) No terminal, digite:

```
$ rails generate model cliente
```

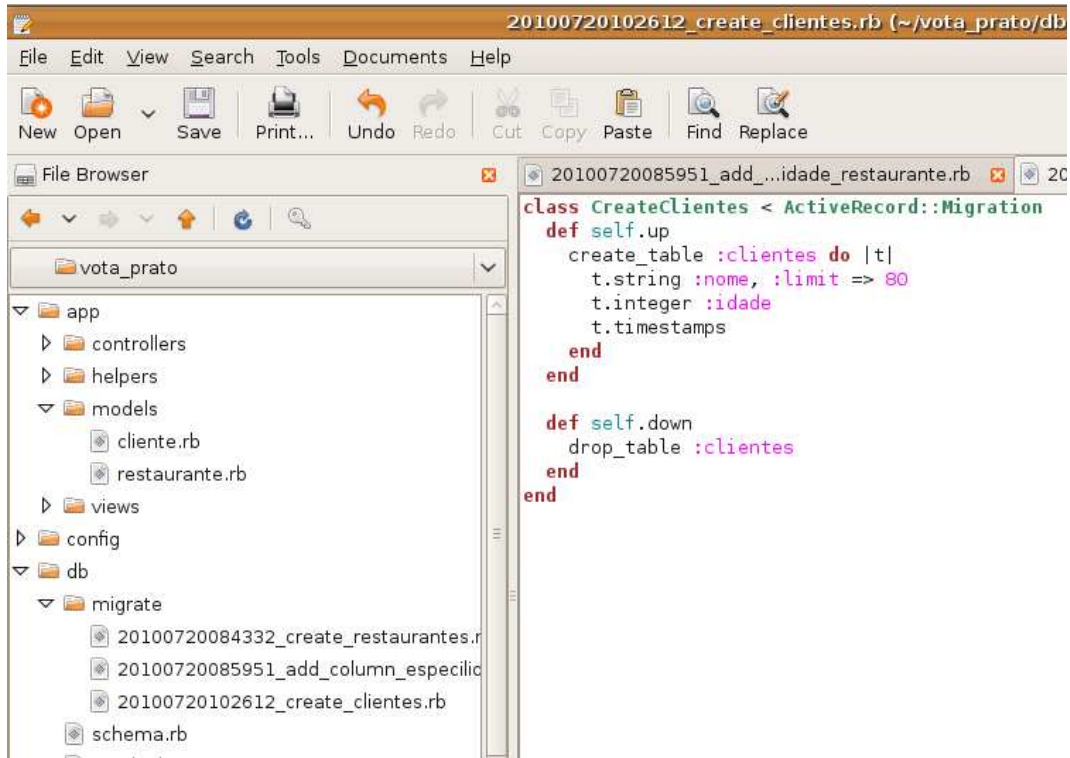


```
rr71@caelum131-03: ~/vota_prato  
rr71@caelum131-03:~/vota_prato$ rails generate model cliente  
  invoke  active_record  
  create  db/migrate/20100720102612_create_clientes.rb  
  create  app/models/cliente.rb  
  invoke  test_unit  
  create  test/unit/cliente_test.rb  
  create  test/fixtures/clientes.yml  
rr71@caelum131-03:~/vota_prato$
```

b) Abra o arquivo "**db/migrate/<timestamp>_create_clientes.rb**"

c) Edite método change para que ele fique como a seguir:

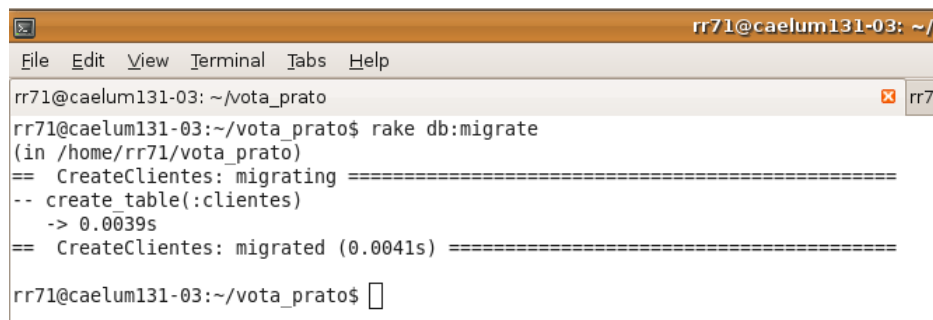
```
create_table :clientes do |t|  
  t.string :nome, limit: 80  
  t.integer :idade  
  t.timestamps  
end
```



d) Vamos efetivar a migration usando o comando:

```
$ rake db:migrate
```

e) Olhe no console o que foi feito



f) Olhe no banco de dados!

```
mysql> show tables;
+-----+
| Tables_in_vota_prato_development |
+-----+
| clientes                          |
| restaurantes                     |
| schema_migrations                |
+-----+
3 rows in set (0.01 sec)
```

- 2) Configure as propriedades adequadas para possibilitar o *mass assignment* na classe `Cliente`. Edite a definição criada no arquivo `app/models/cliente.rb` para adicionar a chamada ao `attr_accessible`:

```
class Cliente < ActiveRecord::Base
  attr_accessible :nome, :idade
end
```

- 3) Vamos agora fazer as validações no modelo "**cliente**":

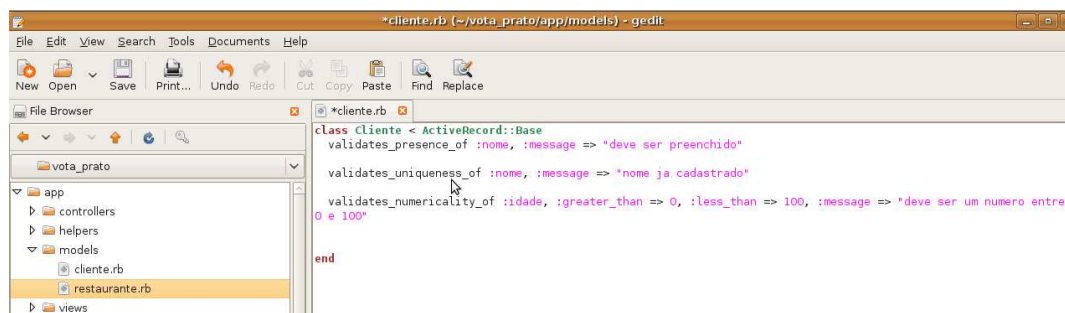
- a) Abra o arquivo "**app/models/cliente.rb**"

- b) Adicione as seguintes linhas:

```
validates_presence_of :nome, message: " - deve ser preenchido"
```

```
validates_uniqueness_of :nome, message: " - nome já cadastrado"
```

```
validates_numericality_of :idade, greater_than: 0,
                             less_than: 100,
                             message: " - deve ser um número entre 0 e 100"
```



- 4) Vamos criar o modelo Prato, bem como sua migration:

- a) Vamos executar o generator de model do rails novamente.
- b) `rails generate model prato` no Terminal
- c) Abra o arquivo "**db/migrate/<timestamp>_create_pratos.rb**"

d) Adicione as linhas:

```
t.string :nome, limit: 80
```

e) Volte para o Terminal

f) execute rake db:migrate

g) Olhe no console o que foi feito

h) Olhe no banco de dados!

- 5) Vamos definir que o nome de um **prato** que pode ser atribuído via hash como parâmetro no construtor, edite o arquivo `app/models/prato.rb` e adicione a invocação ao `attr_accessible`:

```
attr_accessible :nome
```

- 6) Vamos agora fazer as validações no modelo "**prato**". Ainda no arquivo `app/models/prato.rb` adicione o seguinte código:

```
validates_presence_of :nome, message: " - deve ser preenchido"
```

```
validates_uniqueness_of :nome, message: " - nome já cadastrado"
```

- 7) Vamos criar o modelo Receita, bem como sua migration:

a) Vá ao Terminal

b) Execute rails generate model receita

c) Abra o arquivo "**db/migrate/<timestamp>_create_receitas.rb**"

d) Adicione as linhas:

```
t.integer :prato_id  
t.text :conteudo
```

e) Volte ao Terminal

f) Execute rake db:migrate

g) Olhe no console o que foi feito

h) Olhe no banco de dados!

- 8) Complete a classe Receita definindo que `prato_id` e `conteudo` serão passíveis de atribuição via *mass assignment*, lembra como se faz? Edite a classe Receitas definida em `app/models/receita.rb` e adicione a seguinte linha:

```
class Receita  
  attr_accessible :prato_id, :conteudo  
end
```

9) Vamos agora fazer as validações no modelo "receita":

a) Abra o arquivo "app/models/receita.rb"

b) Adicione as seguintes linhas:

```
validates_presence_of :conteudo, message: " - deve ser preenchido"
```

7.17 O MODELO QUALIFICAÇÃO

Antes de criarmos nosso modelo de Qualificação, repare que o Rails pluraliza os nomes de nossos modelos de forma automática. Por exemplo, o nome da tabela do modelo Cliente ficou clientes. No entanto, qualificação é uma palavra que tem sua pluralização irregular (não basta adicionar um 's' no final da palavra), e o Rails deve gerar a seguinte palavra pluralizada: 'qualificacao', uma vez que estamos utilizando o português e ele possui regras de pluralização apenas para o inglês.

Para corrigir isso, vamos ter de editar o arquivo "config/initializers/inflections.rb" e inserir manualmente o plural da palavra 'qualificacao'. Repare que nesse mesmo arquivo temos diversos exemplos comentados de como podemos fazer isso.

Quando inserirmos as linhas abaixo no final do arquivo, o Rails passará a utilizar esse padrão para a pluralização:

```
ActiveSupport::Inflector.inflections do |inflect|  
  inflect.irregular 'qualificacao', 'qualificacoes'  
end
```

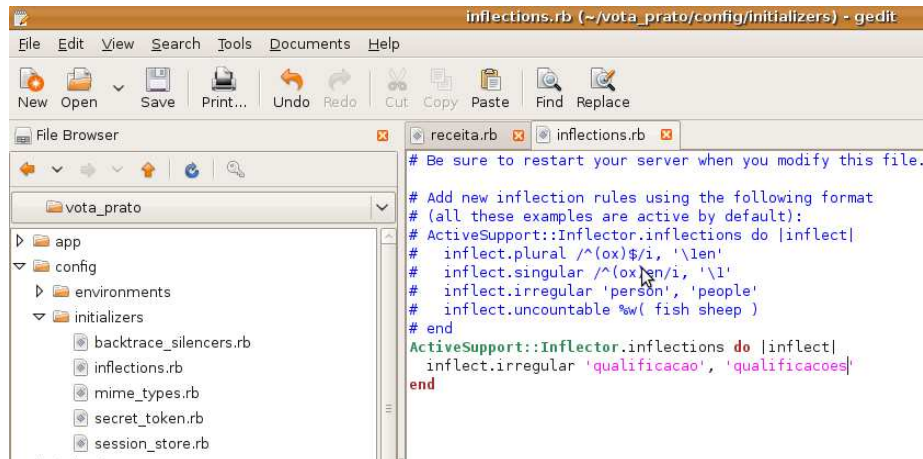
7.18 EXERCÍCIOS - CRIANDO O MODELO DE QUALIFICAÇÃO

1) Vamos corrigir a pluralização da palavra 'qualificacao'

a) Abra o arquivo "config/initializers/inflections.rb"

b) Adicione as seguintes linhas ao final do arquivo:

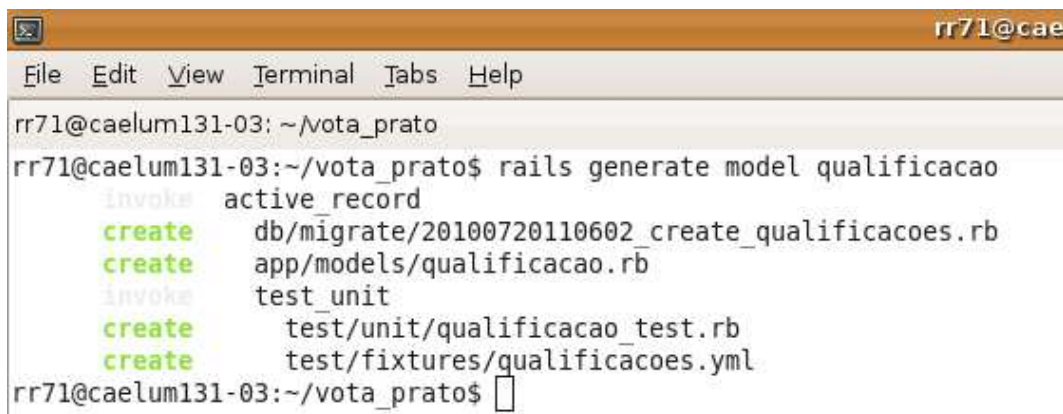
```
ActiveSupport::Inflector.inflections do |inflect|  
  inflect.irregular 'qualificacao', 'qualificacoes'  
end
```



BRAZILIAN-RAILS

Existe um plugin chamado **Brazilian Rails** que é um conjunto de gems para serem usadas com Ruby e com o Ruby on Rails e tem como objetivo unir alguns recursos úteis para os desenvolvedores brasileiros.

- 2) Vamos continuar com a criação do nosso modelo Qualificacao e sua migration.
 - a) No Terminal digite
 - b) rails generate model qualificacao

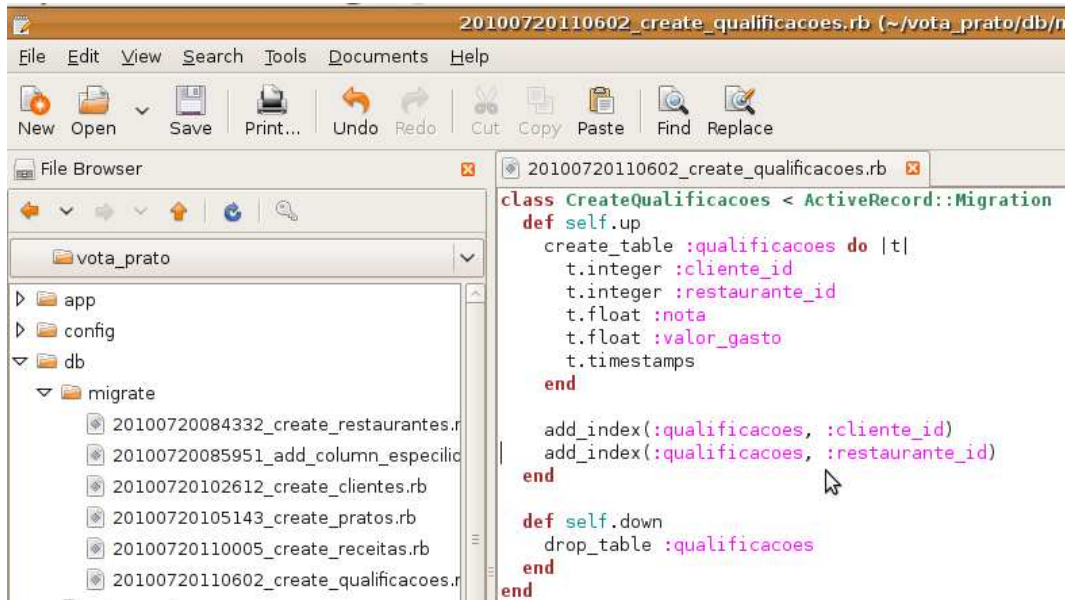


- c) Abra o arquivo "db/migrate/<timestamp>_create_qualificacoes.rb"
- d) Adicione as linhas:

```
t.integer :cliente_id
t.integer :restaurante_id
t.float :nota
t.float :valor_gasto
```

- e) Adicione ainda as seguintes linhas depois de "**create_table**": CUIDADO! Essas linhas não fazem parte do **create_table**, mas devem ficar dentro do método **self.up**.

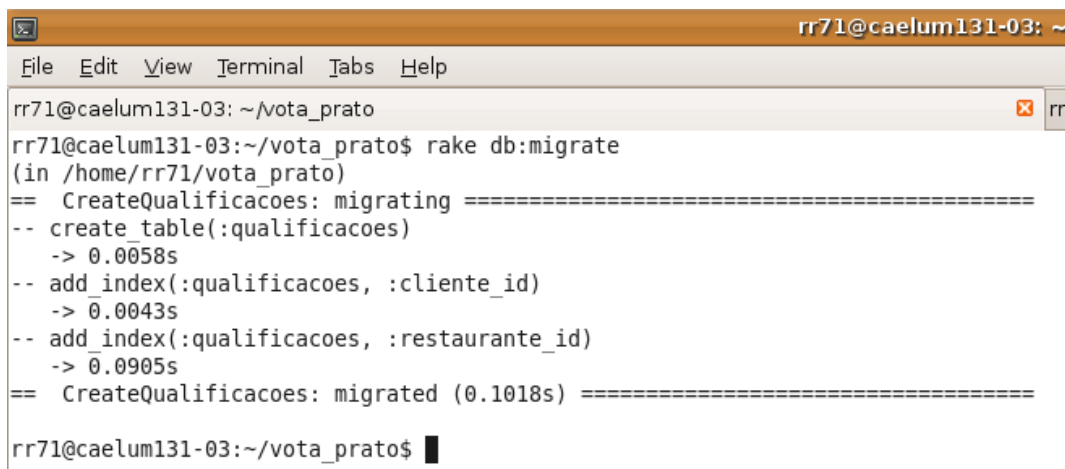
```
add_index(:qualificacoes, :cliente_id)
add_index(:qualificacoes, :restaurante_id)
```



- f) Volte ao Terminal
- g) Execute a task para rodar as migrações pendentes:

```
$ rake db:migrate
```

- h) Olhe no console o que foi feito



- i) Olhe no banco de dados

```
mysql> show tables;
+-----+
| Tables_in_vota_prato_development |
+-----+
| clientes                          |
| pratos                           |
| qualificacoes                    |
| receitas                         |
| restaurantes                     |
| schema_migrations                |
+-----+
6 rows in set (0.00 sec)
```

3) Configure as propriedades de uma qualificação utilizando o `attr_accessible`:

```
attr_accessible :nota, :valor_gasto, :cliente_id, :restaurante_id
```

4) Vamos agora fazer as validações no modelo "qualificacao":

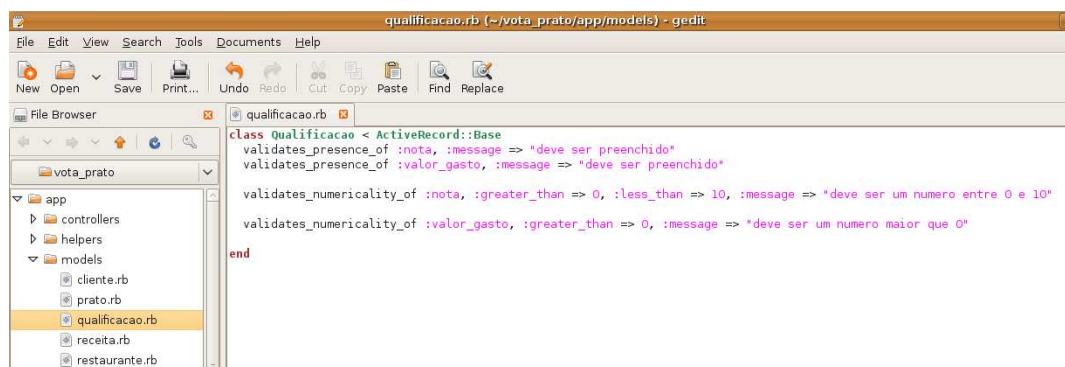
a) Abra o arquivo "app/models/qualificacao.rb"

b) Adicione as seguintes linhas:

```
validates_presence_of :nota, message: " - deve ser preenchido"
validates_presence_of :valor_gasto, message: " - deve ser preenchido"

validates_numericality_of :nota, greater_than_or_equal_to: 0,
                           less_than_or_equal_to: 10,
                           message: " - deve ser um número entre 0 e 10"

validates_numericality_of :valor_gasto, greater_than: 0,
                           message: " - deve ser um número maior que 0"
```



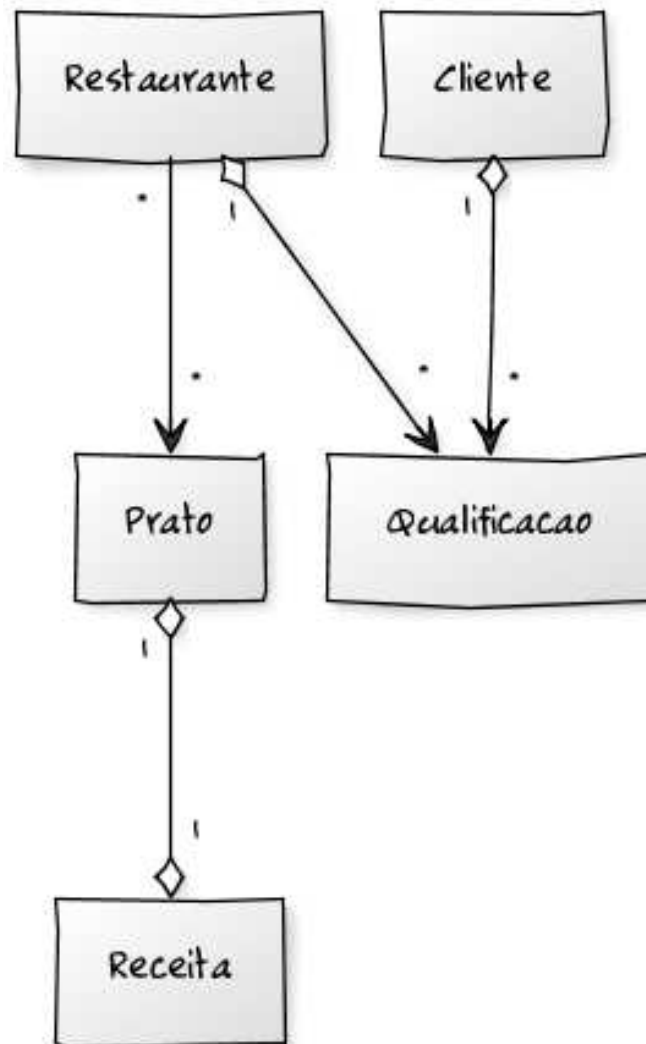
c) Execute o console do Rails:

```
$ rails console
```

No Console do Rails digite:

```
"qualificacao".pluralize
```

7.19 RELACIONAMENTOS



Para relacionar diversos modelos, precisamos informar ao *Rails* o tipo de relacionamento entre eles. Quando isso é feito, alguns métodos são criados para podermos manipular os elementos envolvidos nesse relacionamento. Os relacionamentos que Rails disponibiliza são os seguintes:

- **belongs_to** - usado quando um modelo tem como um de seus atributos o id de outro modelo (many-to-one ou one-to-one). Quando dissermos que uma qualificação **belongs_to** um restaurante, ainda ganharemos os seguintes métodos:
 - `Qualificacao.restaurante` (similar ao `Restaurante.find(restaurante_id)`)
 - `Qualificacao.restaurante=(restaurante)` (similar ao `qualificacao.restaurante_id = restaurante.id`)
 - `Qualificacao.restaurante?` (similar ao `qualificacao.restaurante == algum_restaurante`)

- `has_many` - associação que provê uma maneira de mapear uma relação one-to-many entre duas entidades. Quando dissermos que um restaurante **has_many** qualificações, ganharemos os seguintes métodos:

- `Restaurante.qualificacoes` (semelhante ao `Qualificacao.find :all, conditions: ["restaurante_id = ?", id]`)
- `Restaurante.qualificacoes<<`
- `Restaurante.qualificacoes.delete`
- `Restaurante.qualificacoes=`

- `has_and_belongs_to_many` - associação muitos-para-muitos, que é feita usando uma tabela de mapeamento. Quando dissermos que um prato **has_and_belongs_to_many** restaurantes, ganharemos os seguintes métodos:

- `Prato.restaurantes`
- `Prato.restaurantes<<`
- `Prato.restaurantes.delete`
- `Prato.restaurantes=`

Além disso, precisaremos criar a tabela **pratos_restaurantes**, com as colunas **prato_id** e **restaurante_id**. Por convenção, o nome da tabela é a concatenação do nome das duas outras tabelas, seguindo a ordem alfabética.

- `has_one` - lado bidirecional de uma relação um-para-um. Quando dissermos que um prato **has_one** receita, ganharemos os seguintes métodos:

- `Prato.receita`, (semelhante ao `Receita.find(:first, conditions: "prato_id = id")`)
- `Prato.receita=`

7.20 PARA SABER MAIS: AUTO-RELACIONAMENTO

Os relacionamentos vistos até agora foram sempre entre dois objetos de classes diferentes. Porém existem relacionamentos entre classes do mesmo tipo, por exemplo, uma Categoria de um site pode conter outras categorias, onde cada uma contém ainda suas categorias. Esse relacionamento é chamado *auto-relacionamento*.

No ActiveRecord temos uma forma simples para fazer essa associação.

```
class Category < ActiveRecord::Base
  has_many :children, class_name: "Category",
                    foreign_key: "father_id"
  belongs_to :father, class_name: "Category"
end
```


7.21 PARA SABER MAIS: CACHE

Todos os resultados de métodos acessados de um relacionamento são obtidos de um cache e não novamente do banco de dados. Após carregar as informações do banco, o ActiveRecord só volta se ocorrer um pedido explícito. Exemplos:

```
restaurante.qualificacoes      # busca no banco de dados
restaurante.qualificacoes.size # usa o cache
restaurante.qualificacoes.empty? # usa o cache
restaurante.qualificacoes(true).size # força a busca no banco de dados
restaurante.qualificacoes      # usa o cache
```

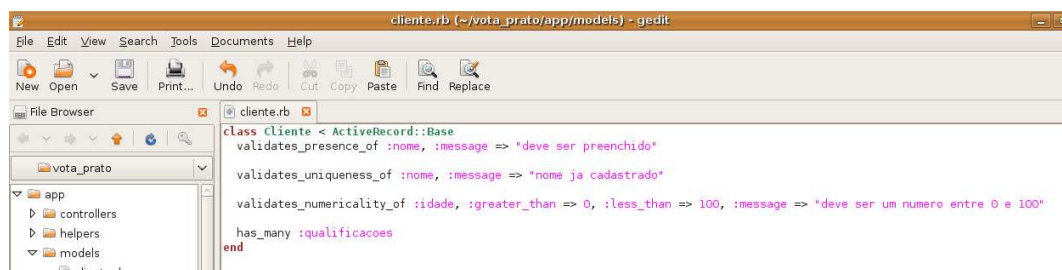
7.22 EXERCÍCIOS - RELACIONAMENTOS

1) Vamos incluir os relacionamentos necessários para nossos modelos:

a) Abra o arquivo "**app/models/cliente.rb**"

b) Adicione a seguinte linha:

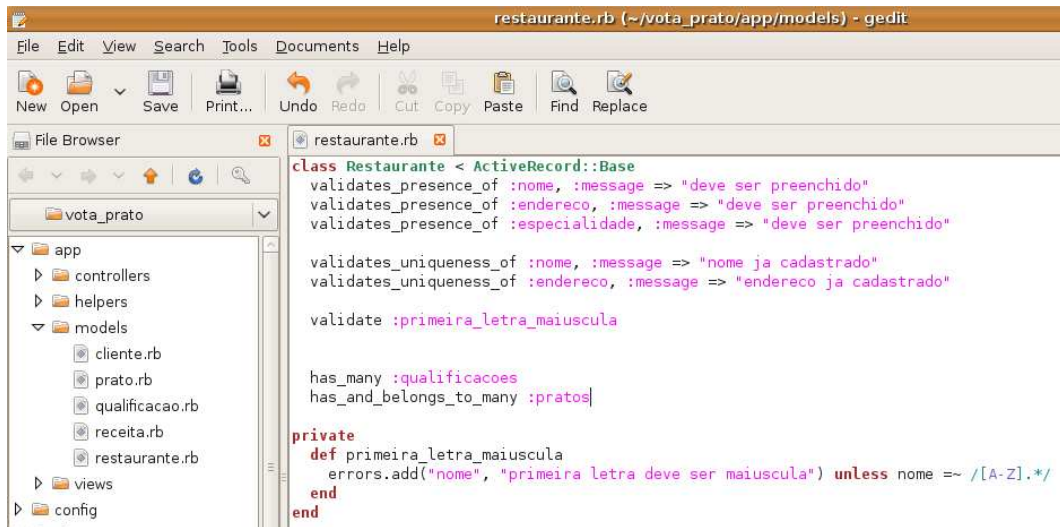
```
has_many :qualificacoes
```



c) Abra o arquivo "**app/models/restaurante.rb**"

d) Adicione a seguinte linha:

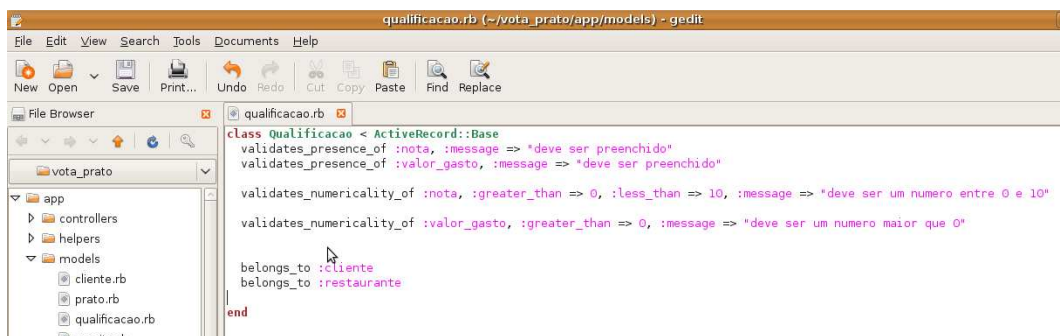
```
has_many :qualificacoes
has_and_belongs_to_many :pratos
```

e) Abra o arquivo **"app/models/qualificacao.rb"**

f) Adicione as seguintes linhas:

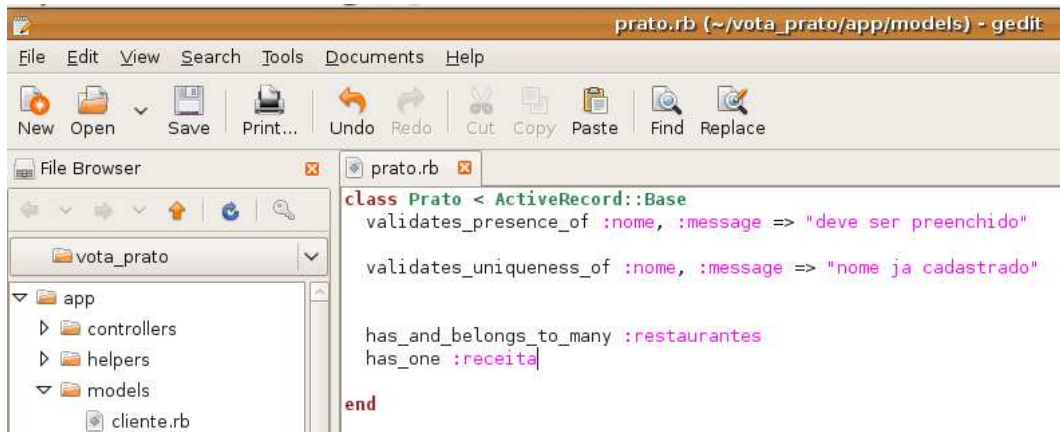
```
belongs_to :cliente
belongs_to :restaurante
```



g) Abra o arquivo **"app/models/prato.rb"**

h) Adicione as seguintes linhas:

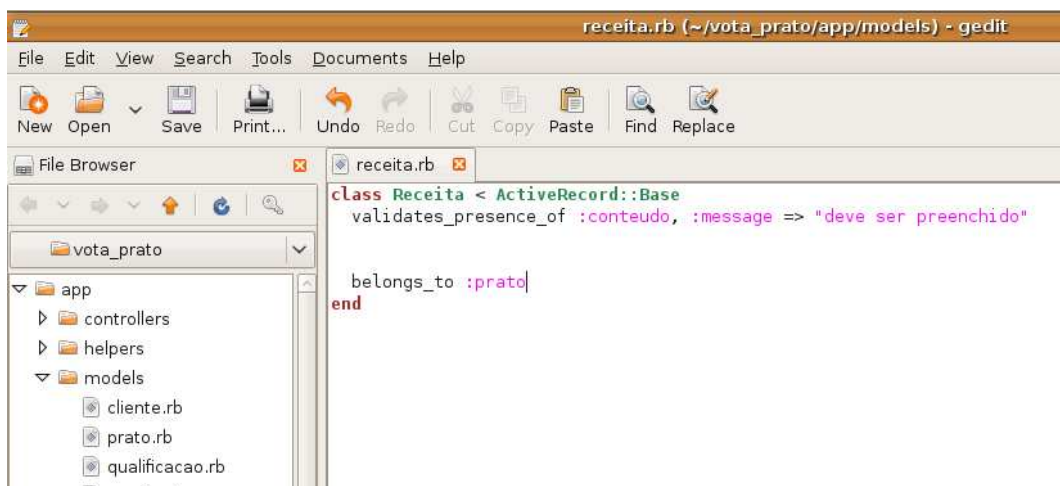
```
has_and_belongs_to_many :restaurantes
has_one :receita
```



i) Abra o arquivo "**app/models/receita**"

j) Adicione a seguinte linha:

`belongs_to :prato`



2) Vamos criar a tabela para nosso relacionamento **has_and_belongs_to_many**:

a) Vá para o Terminal

b) Digite: **rails generate migration createPratosRestaurantesJoinTable**

c) Abra o arquivo "**db/migrate/<timestamp>_create_pratos_restaurantes_join_table.rb**"

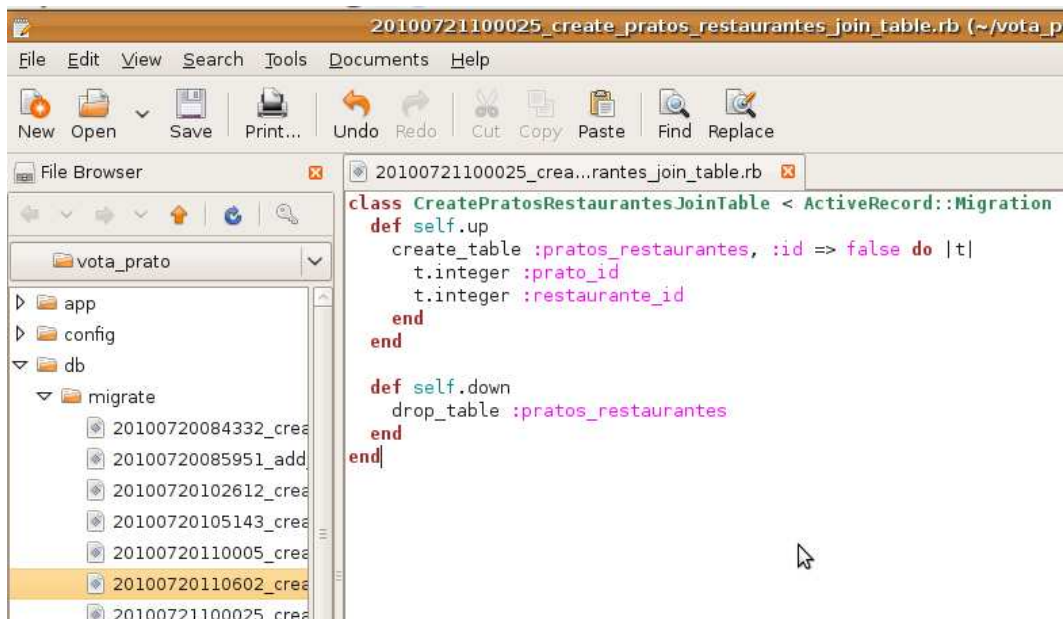
d) O rails não permite que uma tabela de ligação para um relacionamento **has_and_belongs_to_many** possua chave primária auto incremento. Por isso, vamos fazer com que o rails não gere essa chave. Basta passar o parâmetro `id: false` para o método `create_table`. Dessa forma, teremos que digitar o seguinte no `self.up`:

```
create_table :pratos_restaurantes, id: false do |t|  
  t.integer :prato_id
```

```
t.integer :restaurante_id  
end
```

e) E o seguinte no self.down

```
drop_table :pratos_restaurantes
```



f) Volte ao Terminal e execute as migrações:

```
$ rake db:migrate
```

g) Olhe no console o que foi feito

h) Olhe no banco de dados

3) Vamos incluir as validações que garantam que os relacionamentos foram feitos corretamente:

a) Abra o arquivo "**app/models/qualificacao.rb**"

b) Adicione as seguintes linhas:

```
validates_presence_of :cliente_id, :restaurante_id  
validates_associated :cliente, :restaurante
```

c) Abra o arquivo "**app/models/receita.rb**"

d) Adicione as seguintes linhas:

```
validates_presence_of :prato_id  
validates_associated :prato
```

e) Abra o arquivo "**app/models/prato.rb**"

f) Adicione as seguintes linhas:

```
validate :validate_presence_of_more_than_one_restaurante

private
def validate_presence_of_more_than_one_restaurante
  errors.add("restaurantes",
    "deve haver ao menos um restaurante") if restaurantes.empty?
end
```

4) Faça alguns testes no terminal para testar essas novas validações.

7.23 PARA SABER MAIS - EAGER LOADING

Podemos cair em um problema grave caso o número de qualificações para um restaurante seja muito grande. Imagine que um determinado restaurante do nosso sistema possua com 100 qualificações. Queremos mostrar todas as qualificações desse restaurante, então fazemos um for:

```
for qualificacao in Qualificacao.all
  puts "restaurante: " + qualificacao.restaurante.nome
  puts "cliente:      " + qualificacao.cliente.nome
  puts "qualificacao: " + qualificacao.nota
end
```

Para iterar sobre as 100 qualificações do banco de dados, seriam geradas 201 buscas! Uma busca para todas as qualificações, 100 buscas para cada restaurante mais 100 buscas para cada cliente! Podemos melhorar um pouco essa busca. Podemos pedir ao ActiveRecord que inclua o restaurante quando fizer a busca:

```
Qualificacao.find(:all, include: :restaurante)
```

Bem melhor! Agora a quantidade de buscas diminuiu para 102! Uma busca para as qualificações, outra para todos os restaurantes e mais 100 para os clientes. Podemos utilizar a mesma estratégia para otimizar a busca de clientes:

```
Qualificacao.includes(:restaurante, :cliente)
```

Com essa estratégia, teremos o número de buscas muito reduzido. A quantidade total agora será de 1 + o número de associações necessárias. Poderíamos ir mais além, e trazer uma associação de uma das associações existentes em qualificação:

```
Qualificacao.includes(:cliente, restaurante: {pratos: :receita})
```

7.24 PARA SABER MAIS - NAMED SCOPES

Para consultas muito comuns, podemos usar o recurso de **Named Scopes** oferecido pelo ActiveRecord, que permite deixarmos alguns tipos de consultas comuns “preparadas”.

Imagine que a consulta de restaurantes de especialidade “massa” seja muito comum no nosso sistema. Podemos facilitá-la criando um named scope na classe Restaurante:

```
class Restaurante < ActiveRecord::Base
  scope :massas, where(especialidade: 'massas')
end
```

As opções mais comuns do método find também estão disponíveis para *named scopes*, como :conditions, :order, :select e :include.

Com o *named scope* definido, a classe ActiveRecord ganha um método de mesmo nome, através do qual podemos recuperar os restaurantes de especialidade "massas" de forma simples:

```
Restaurante.massas
Restaurante.massas.first
Restaurante.massas.last
Restaurante.massas.where(["nome like ?", '%x%'])
```

O método associado ao *named scope* criado retorna um objeto da classe ActiveRecord::NamedScope, que age como um Array, mas aceita a chamada de alguns métodos das classes ActiveRecord, como o find para filtrar ainda mais a consulta.

Podemos ainda definir diversos *named scopes* e combiná-los de qualquer forma:

```
class Restaurante < ActiveRecord::Base
  scope :massas, where(especialidade: 'massas')
  scope :recentes, where(["created_at > ?", 3.months.ago])
  scope :pelo_nome, order('nome')
end

Restaurante.massas # todos de especialidade = 'massas'
Restaurante.recentes # todos de created_at > 3 meses atras

# especialidade = 'massas' e created_at > 3 meses atras
Restaurante.massas.recentes
Restaurante.recentes.massas

Restaurante.massas.pelo_nome.recentes
```

7.25 PARA SABER MAIS - MODULES

As associações procuram por relacionamentos entre classes que estejam no mesmo módulo. Caso precise de relacionamentos entre classes em módulos distintos, é necessário informar o nome completo da classe no relacionamento:

```
module Restaurante
  module RH
    class Pessoa < ActiveRecord::Base
      end
    end
  end

  module Financeiro
    class Pagamento < ActiveRecord::Base
      belongs_to :pessoa, class_name: "Restaurante::RH::Pessoa"
    end
  end
end
```

CAPÍTULO 8

Controllers e Views

“A Inteligência é quase inútil para quem não tem mais nada”

– Carrel, Alexis

Nesse capítulo, você vai aprender o que são controllers e como utilizá-los para o benefício do seu projeto, além de aprender a trabalhar com a camada visual de sua aplicação.

8.1 O “V” E O “C” DO MVC

O “V” de MVC representa a parte de **view** (visualização) da nossa aplicação, sendo ela quem tem contato com o usuário, recebe as entradas e mostra qualquer tipo de saída.

Há diversas maneiras de controlar as views, sendo a mais comum delas feita através dos arquivos HTML.ERB, ou eRuby (Embedded Ruby), páginas HTML que podem receber trechos de código em Ruby.

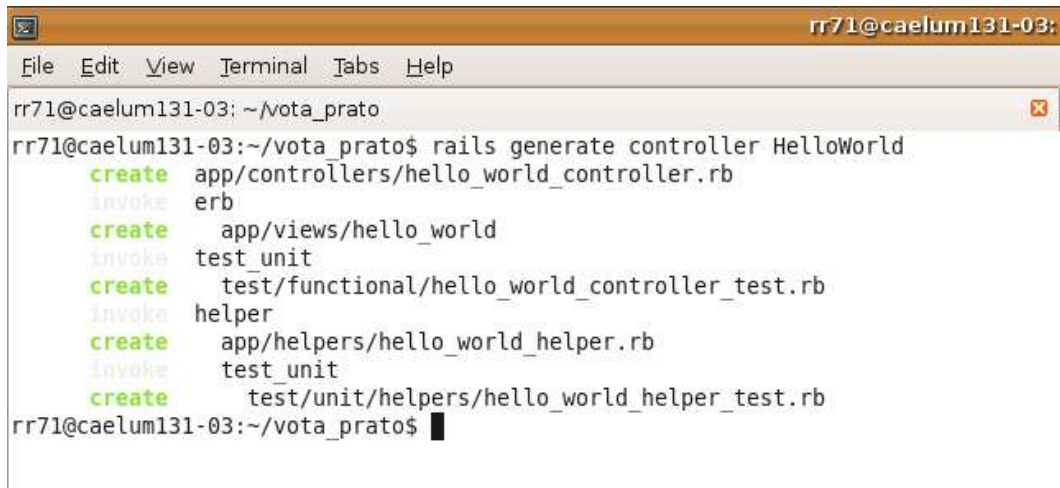
Controllers são classes que recebem uma ação de uma View e executam algum tipo de lógica ligada a um ou mais modelos. Em Rails esses controllers estendem a classe `ApplicationController`.

As urls do servidor são mapeadas da seguinte maneira: `/controller/action/id`. Onde “controller” representa uma classe controladora e “action” representa um método do mesmo. “id” é um parâmetro qualquer (opcional).

8.2 HELLO WORLD

Antes de tudo criaremos um controller que mostrará um *“Hello World”* para entender melhor como funciona essa ideia do mapeamento de urls.

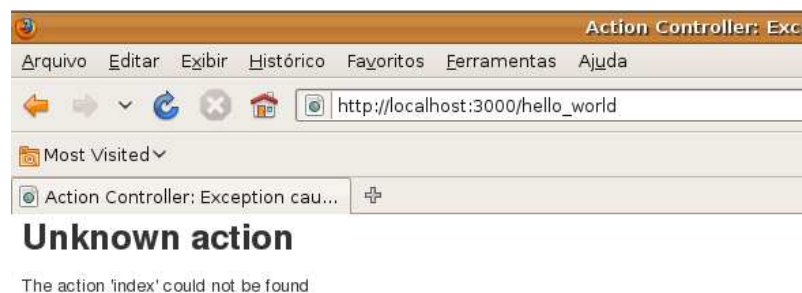
Vamos usar o generator do rails para criar um novo controller que se chamará *"HelloWorld"*. Veja que o Rails não gera apenas o Controller, mas também outros arquivos.



```
rr71@caelum131-03: ~/vota_prato
rr71@caelum131-03:~/vota_prato$ rails generate controller HelloWorld
create  app/controllers/hello_world_controller.rb
invoke  erb
create  app/views/hello_world
invoke  test_unit
create  test/functional/hello_world_controller_test.rb
invoke  helper
create  app/helpers/hello_world_helper.rb
invoke  test_unit
create  test/unit/helpers/hello_world_helper_test.rb
rr71@caelum131-03:~/vota_prato$
```

Apos habilitar o rota padrão do rails tente acessar a página http://localhost:3000/hello_world

Na URL acima passamos apenas o controller sem nenhuma action, por isso recebemos uma mensagem de erro.



Além de não dizer qual a action na URL, não escrevemos nenhuma action no controller.

Criaremos um método chamado `hello` no qual escreveremos na saída do cliente a frase *"Hello World!"*. **Cada método criado no controller é uma action**, que pode ser acessada através de um browser.

Para escrever na saída, o Rails oferece o comando `render`, que recebe uma opção chamada *"text"* (String). Tudo aquilo que for passado por esta chave será recebido no browser do cliente.

8.3 EXERCÍCIOS: CRIANDO O CONTROLADOR

- 1) Crie um controller que mostre na tela a mensagem *"Hello World"*

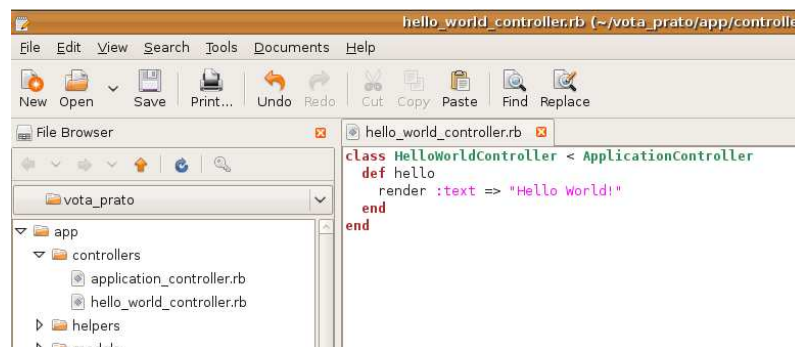
- a) Vá ao Terminal
- b) Execute **rails generate controller HelloWorld**



```
rr71@caelum131-03: ~/vota_prato
rr71@caelum131-03:~/vota_prato$ rails generate controller HelloWorld
create  app/controllers/hello_world_controller.rb
invoke  erb
create  app/views/hello_world
invoke  test_unit
create  test/functional/hello_world_controller_test.rb
invoke  helper
create  app/helpers/hello_world_helper.rb
invoke  test_unit
create  test/unit/helpers/hello_world_helper_test.rb
rr71@caelum131-03:~/vota_prato$
```

- c) Entre no seu novo controller (**app/controllers/hello_world_controller.rb**)
- d) Inclua o método “hello”:

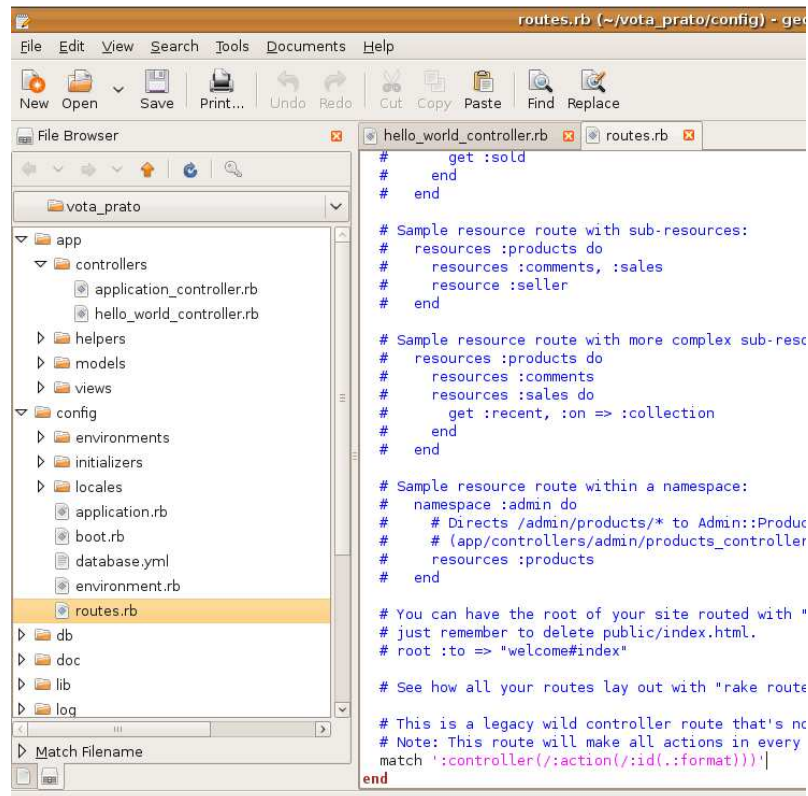
```
def hello
  render text: "Hello World!"
end
```



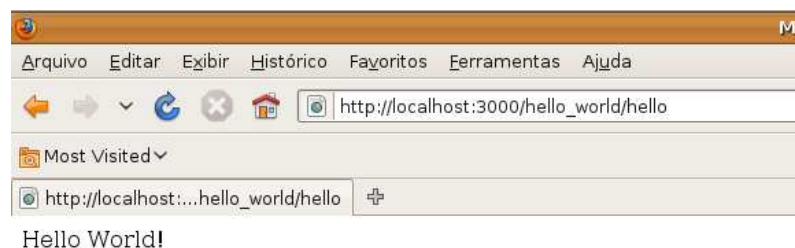
```
class HelloWorldController < ApplicationController
  def hello
    render :text => "Hello World!"
  end
end
```

- e) Descomente a última linha do arquivo **config/routes.rb** para habilitar a rota padrão do rails. No próximo capítulo falaremos mais sobre esse assunto.

```
match ':controller(/:action(/:id(.:format)))'
```



- f) O último passo antes de testar no browser é iniciar o server.
- g) Execute no Terminal: **rails server**
- h) Confira o link http://localhost:3000/hello_world/hello.



8.4 REDIRECIONAMENTO DE ACTION E ACTION PADRÃO

Mas não há necessidade de toda vez escolher qual action será executada. Temos a opção de escrever uma action padrão que será invocada sempre que nenhuma outra for definida na url sendo acessada.

Para definir uma action padrão temos que dar o nome do método de index:

```
def index
  render text: "Action Padrão!"
end
```

Dentro de uma action, podemos redirecionar a saída para uma outra action. No exemplo do Hello World, ao chamar o comando `redirect_to` e passar como argumento a action desejada (`action: "hello"`), o Rails fará o redirecionamento no cliente.

```
def index
  redirect_to(action: "hello")
end
```

Ao acessar `http://localhost:3000/hello_world`, o servidor nos redireciona para a página `http://localhost:3000/hello_world/hello`, mostrando um “Hello World” igual ao anterior.

Ao invés de fazer o redirecionamento, podemos chamar outra action diretamente. Assim não vamos ser redirecionados e a URL continuará a mesma.

```
def index
  hello
end
```

REDIRECIONAMENTO NO SERVIDOR E NO CLIENTE

O redirecionamento no servidor é conhecido como *forward* e a requisição é apenas repassada a um outro recurso (página, controlador) que fica responsável em tratar a requisição.

Há uma outra forma que é o **redirecionamento no cliente** (*redirect*). Nesta modalidade, o servidor responde a requisição original com um **pedido de redirecionamento**, fazendo com que o navegador dispare uma nova requisição para o novo endereço. Neste caso, a barra de endereços do navegador muda.

8.5 TRABALHANDO COM A VIEW: O ERB

ERB

ERB é uma implementação de eRuby que já acompanha a linguagem Ruby. Seu funcionamento é similar ao dos arquivos JSP/ASP: arquivos html com injeções de código. A ideia é que o HTML serve como um template, e outros elementos são dinamicamente inseridos em tempo de renderização.

Para uma página aceitar código Ruby, ela deve estar entre «%» e "%>". Há uma variação deste operador, o «%=», que não só executa códigos Ruby, mas também imprime o resultado na página HTML.

É importante notar que todos os atributos de instância (@variavel) de um controlador estão disponíveis em sua view. Além disso, ela deve ter o mesmo nome do controlador, o que significa que a view da nossa action index do controlador `restaurantes_controller.rb` deve estar em **app/views/restaurantes/index.html.erb**.

Precisamos de uma view que vá possibilitar inserir os valores de um novo restaurante:

```
<form action='/restaurantes/create'>
  Nome: <input type='text' name='nome' />
  <input type='submit' value='Create' />
</form>
```

E agora, para receber este valor nome no controlador, basta usar o hash `params`. (Repare que agora usamos outra action, `create`, para buscar os dados do formulário apresentado anteriormente):

```
class RestaurantesController < ApplicationController
  def create
    nome = params[:nome]
  end
end
```

O problema desta abordagem é que precisaríamos recuperar os valores enviados pelo formulário, um a um. Para um formulário com 15 campos, teríamos 15 linhas! Para simplificar esta tarefa, o Rails oferece a possibilidade de utilizar algo parecido com um hash para os valores dos atributos:

```
<form action='/restaurantes/create'>
  Nome: <input type='text' name='restaurante[nome]' />
  Endereço: <input type='text' name='restaurante[endereco]' />
  Especialidade: <input type='text' name='restaurante[especialidade]' />
  <input type='submit' value='Create' />
</form>
```

Desta forma, podemos receber todos os valores como um hash nos controladores:

```
class RestaurantesController < ApplicationController
  def create
    valores = params[:restaurante]
  end
end
```

O mais interessante é que as classes ActiveRecord já aceitam um hash com os valores iniciais do objeto, tanto no método `new`, no método `create` (que já salva), quanto no método `update_attributes`:

```
Restaurante.create params[:restaurante]
```

8.6 ENTENDENDO MELHOR O CRUD

Agora, queremos ser capazes de criar, exibir, editar e remover restaurantes. Como fazer?

Primeiro, temos de criar um controller para nosso restaurante. Pela view Generators, vamos criar um controller para restaurante. Rails, por padrão, utiliza-se de sete actions “CRUD”. São eles:

- `list`: exibe todos os itens
- `show`: exibe um item específico
- `new`: formulário para a criação de um novo item
- `create`: cria um novo item
- `edit`: formulário para edição de um item
- `update`: atualiza um item existente
- `destroy`: remove um item existente

Desejamos listar todos os restaurantes do nosso Banco de Dados, e portanto criaremos a action `list`. Como desejamos que o comportamento padrão do nosso controlador restaurante seja exibir a listagem de restaurantes, podemos renomeá-la para `index`.

Assim como no console buscamos todos os restaurantes do banco de dados com o comando `find`, também podemos fazê-lo em controllers (que poderão ser acessados pelas nossas views, como veremos mais adiante).

Basta agora passar o resultado da busca para uma variável:

```
def index
  @restaurantes = Restaurante.order("nome")
end
```

Para exibir um restaurante específico, precisamos saber qual restaurante buscar. Essa informação vem no “id” da url, e contem o id do restaurante que desejamos. Para acessá-la, podemos usar como parâmetro do método `find params[:id]`, que recupera as informações passadas no id da url.

Agora, podemos criar nossa action `show`:

```
def show
  @restaurante = Restaurante.find(params[:id])
end
```

Para incluir um novo restaurante, precisamos primeiro retornar ao browser um restaurante novo, sem informação alguma. Vamos criar nossa action `new`

```
def new
  @restaurante = Restaurante.new
end
```

Uma vez que o usuário do nosso sistema tenha preenchido as informações do novo restaurante e deseje salvá-las, enviará uma requisição à nossa action create, passando como parâmetro na requisição, o novo restaurante a ser criado. Vamos criar nossa action:

```
def create
  @restaurante = Restaurante.new(params[:restaurante])
  @restaurante.save
end
```

Para editar um restaurante, devemos retornar ao browser o restaurante que se quer editar, para só depois salvar as alterações feitas:

```
def edit
  @restaurante = Restaurante.find(params[:id])
end
```

Uma vez que o usuário tenha atualizado as informações do restaurante e deseje salvá-las, enviará uma requisição à nossa action update passando no id da url o id do restaurante a ser editado, bem como o restaurante que será “colado” em seu lugar:

```
def update
  @restaurante = Restaurante.find(params[:id])
  @restaurante.update_attributes(params[:restaurante])
end
```

Para remover um restaurante, o usuário enviará uma requisição à nossa action destroy passando no id da url o id do restaurante a ser excluído:

```
def destroy
  @restaurante = Restaurante.find(params[:id])
  @restaurante.destroy
end
```

8.7 EXERCÍCIOS: CONTROLADOR DO RESTAURANTE

- 1) Gere um controller para o modelo restaurante:
 - a) Vá ao Terminal;

- b) Execute **rails generate controller restaurantes**
- 2) Crie as actions CRUD para o controller criado:
- a) Abra o seu controller de Restaurantes (**app/controllers/restaurantes_controllers.rb**)
- b) Insira no controller a actions CRUD:

```
class RestaurantesController < ApplicationController
  def index
    @restaurantes = Restaurante.order("nome")
  end

  def show
    @restaurante = Restaurante.find(params[:id])
  end

  def new
    @restaurante = Restaurante.new
  end

  def create
    @restaurante = Restaurante.new(params[:restaurante])
    if @restaurante.save
      redirect_to(action: "show", id: @restaurante)
    else
      render action: "new"
    end
  end

  def edit
    @restaurante = Restaurante.find(params[:id])
  end

  def update
    @restaurante = Restaurante.find(params[:id])
    if @restaurante.update_attributes(params[:restaurante])
      redirect_to(action: "show", id: @restaurante)
    else
      render action: "edit"
    end
  end

  def destroy
    @restaurante = Restaurante.find(params[:id])
    @restaurante.destroy
  end
end
```

```
    redirect_to(action: "index")  
  end  
end
```

8.8 HELPER

Helpers são módulos que disponibilizam métodos para serem usados em nossas views. Eles provêm atalhos para os códigos mais usados e nos poupam de escrever muito código. O propósito de um helper é simplificar suas views.

Quando criamos um controller, o Rails automaticamente cria um helper para esse controller em **app/helpers/**. Todo método escrito num helper, estará automaticamente disponível em sua view. Existe um Helper especial, o **application_helper.rb**, cujos métodos ficam disponíveis para todas as views.

Quando trabalhamos com formulários, usamos os chamados **FormHelpers**, que são módulos especialmente projetados para nos ajudar nessa tarefa. Todo **FormHelper** está associado a um ActiveRecord. Existem também os **FormTagHelpers**, que contém um *_tag* em seu nome. **FormTagHelpers**, não estão necessariamente associados a ActiveRecord algum.

Abaixo, uma lista dos FormHelpers disponíveis:

- `check_box`
- `fields_for`
- `file_field`
- `form_for`
- `hidden_field`
- `label`
- `password_field`
- `radio_button`
- `text_area`
- `text_field`

E uma lista dos FormTagHelpers:

- `check_box_tag`
- `field_set_tag`
- `file_field_tag`
- `form_tag`

- `hidden_field_tag`
- `image_submit_tag`
- `password_field_tag`
- `radio_button_tag`
- `select_tag`
- `submit_tag`
- `text_area_tag`
- `text_field_tag`

Agora, podemos reescrever nossa view:

```
<%= form_tag :action => 'create' do %>
  Nome: <%= text_field :restaurante, :nome %>
  Endereço: <%= text_field :restaurante, :endereco %>
  Especialidade: <%= text_field :restaurante, :especialidade %>
  <%= submit_tag 'Criar' %>
<% end %>
```

Repare que como utilizamos o `form_tag`, que não está associado a nenhum ActiveRecord, nosso outro Helper **text_field** não sabe qual o ActiveRecord que estamos trabalhando, sendo necessário passar para cada um deles o parâmetro `:restaurante`, informando-o.

Podemos reescrever mais uma vez utilizando o FormHelper `form_for`, que está associado a um ActiveRecord:

```
<%= form_for :restaurante, :url => { :action => 'create' } do |f| %>
  Nome: <%= f.text_field :nome %>
  Endereço: <%= f.text_field :endereco %>
  Especialidade: <%= f.text_field :especialidade %>
  <%= submit_tag 'Criar' %>
<% end %>
```

Repare agora que não foi preciso declarar o nome do nosso modelo para cada `text_field`, uma vez que nosso Helper `form_for` já está associado a ele.

HELPER METHOD

Existe também o chamado `helper_method`, que permite que um método de seu controlador vire um Helper e esteja disponível na view para ser chamado. Exemplo:

```
class TesteController < ApplicationController
  helper_method :teste

  def teste
    "algum conteudo dinâmico"
  end
end
```

E em alguma das views deste controlador:

```
<%= teste %>
```

8.9 EXERCÍCIOS: UTILIZANDO HELPERS PARA CRIAR AS VIEWS

1) Vamos criar as views do restaurante:

a) Crie o arquivo **app/views/restaurantes/index.html.erb**.

b) Digite o conteúdo abaixo:

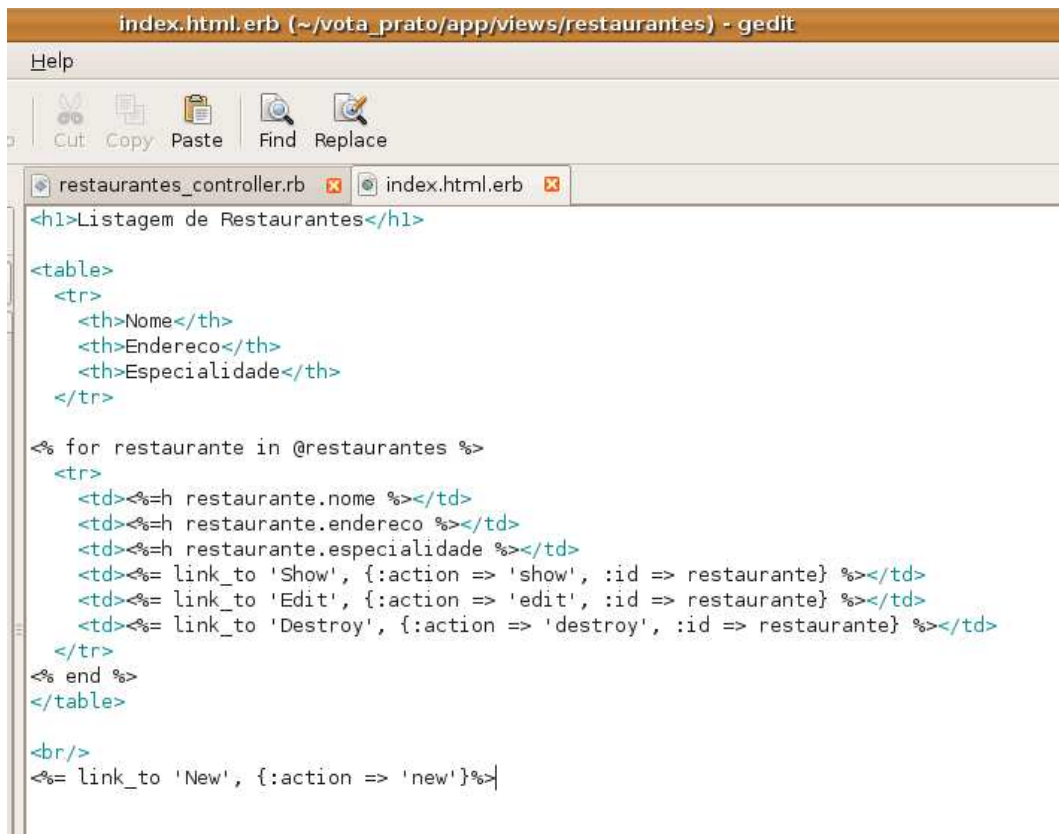
```
<h1>Listagem de Restaurantes</h1>

<table>
  <tr>
    <th>Nome</th>
    <th>Endereço</th>
    <th>Especialidade</th>
  </tr>

  <% @restaurantes.each do |restaurante| %>
    <tr>
      <td><%= restaurante.nome %></td>
      <td><%= restaurante.endereco %></td>
      <td><%= restaurante.especialidade %></td>
    <td>
```

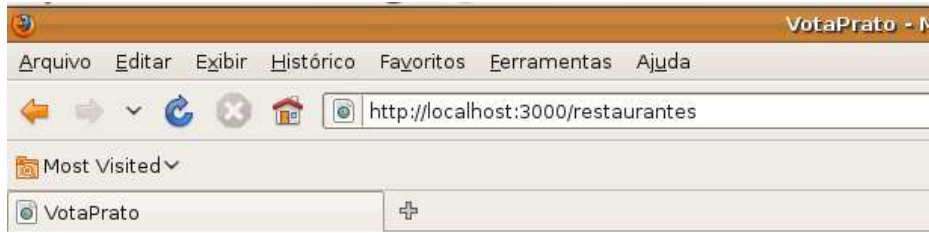
```
<%= link_to 'Mostrar', action:, id: restaurante %>
</td>
<td>
  <%= link_to 'Editar', action: 'edit', id: restaurante %>
</td>
<td>
  <%= link_to 'Deletar', action: 'destroy', id: restaurante %>
</td>
</tr>
<% end %>
</table>

<br/>
<%= link_to 'Novo', action: 'new' %>
```



```
index.html.erb (~/.vota_prato/app/views/restaurantes) - gedit
Help
Cut Copy Paste Find Replace
restaurantes_controller.rb index.html.erb
<h1>Listagem de Restaurantes</h1>
<table>
  <tr>
    <th>Nome</th>
    <th>Endereco</th>
    <th>Especialidade</th>
  </tr>
  <% for restaurante in @restaurantes %>
    <tr>
      <td><%=h restaurante.nome %></td>
      <td><%=h restaurante.endereco %></td>
      <td><%=h restaurante.especialidade %></td>
      <td><%= link_to 'Show', {:action => 'show', :id => restaurante} %></td>
      <td><%= link_to 'Edit', {:action => 'edit', :id => restaurante} %></td>
      <td><%= link_to 'Destroy', {:action => 'destroy', :id => restaurante} %></td>
    </tr>
  <% end %>
</table>
<br/>
<%= link_to 'New', {:action => 'new'}%>
```

c) Teste agora entrando em: **<http://localhost:3000/restaurantes>** (Não esqueça de iniciar o servidor)



Listagem de Restaurantes

Nome	Endereco	Especialidade	
Fasano	Av. dos restaurantes, 126	Italiana	Show Edit Destroy
Nakasa	Rua da Consolacao, 123	Japones	Show Edit Destroy
New			

d) Crie o arquivo **app/views/restaurantes/show.html.erb**

e) Digite o conteúdo abaixo:

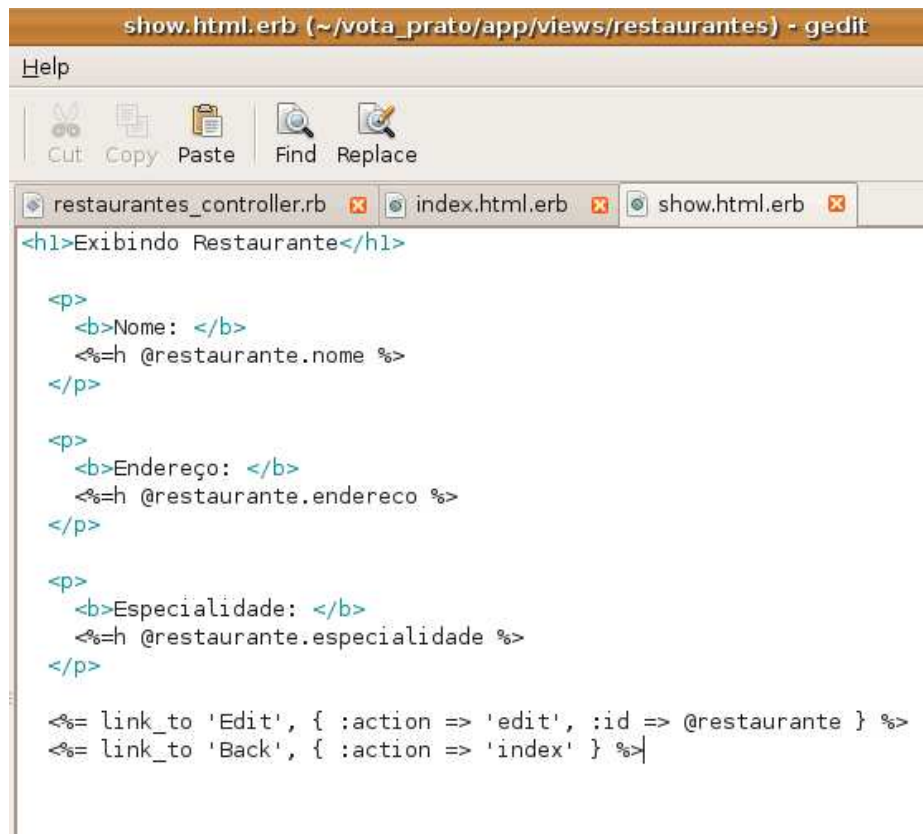
```
<h1>Exibindo Restaurante</h1>

<p>
  <b>Nome: </b>
  <%= @restaurante.nome %>
</p>

<p>
  <b>Endereço: </b>
  <%= @restaurante.endereco %>
</p>

<p>
  <b>Especialidade: </b>
  <%= @restaurante.especialidade %>
</p>

<%= link_to 'Editar', action: 'edit', id: @restaurante } %>
<%= link_to 'Voltar', action: 'index' } %>
```



```
<h1>Exibindo Restaurante</h1>

<p>
  <b>Nome: </b>
  <%=h @restaurante.nome %>
</p>

<p>
  <b>Endereço: </b>
  <%=h @restaurante.endereco %>
</p>

<p>
  <b>Especialidade: </b>
  <%=h @restaurante.especialidade %>
</p>

<%= link_to 'Edit', { :action => 'edit', :id => @restaurante } %>
<%= link_to 'Back', { :action => 'index' } %>
```

f) Crie o arquivo **app/views/restaurantes/new.html.erb**

g) Digite o conteúdo abaixo:

```
<h1>Adicionando Restaurante</h1>

<% if @restaurante.errors.any? %>
  <ul>
    <% @restaurante.errors.full_messages.each do |msg| %>
      <li><%= msg %></li>
    <% end %>
  </ul>
<% end %>

<%= form_for :restaurante, url: {action: 'create'} do |f| %>
  Nome: <%= f.text_field :nome %><br/>
  Endereço: <%= f.text_field :endereco %><br/>
  Especialidade: <%= f.text_field :especialidade %><br/>
  <%= submit_tag 'Criar' %>
<% end %>

<%= link_to 'Voltar', {action: 'index' } %>
```

h) Crie o arquivo **app/views/restaurantes/edit.html.erb**

i) Digite o conteúdo abaixo:

```
<h1>Editando Restaurante</h1>

<% if @restaurante.errors.any? %>
  <ul>
    <% @restaurante.errors.full_messages.each do |msg| %>
      <li><%= msg %></li>
    <% end %>
  </ul>
<% end %>

<%= form_for :restaurante, url: {action: 'update',
  id: @restaurante} do |f| %>
  Nome: <%= f.text_field :nome %><br/>
  Endereço: <%= f.text_field :endereco %><br/>
  Especialidade: <%= f.text_field :especialidade %><br/>
  <%= submit_tag 'Atualizar' %>
<% end %>

<%= link_to 'Voltar', action: 'index' %>
```

j) Teste suas views: **http://localhost:3000/restaurantes** Não esqueça que existe uma validação para **primeira letra maiúscula** no nome do restaurante.

8.10 PARTIAL

Agora, suponha que eu queira exibir em cada página do restaurante um texto, por exemplo: “Controle de Restaurantes”.

Poderíamos escrever esse texto manualmente, mas vamos aproveitar essa necessidade para conhecer um pouco sobre Partials.

Partials são fragmentos de *html.erb* que podem ser incluídas em uma view. Eles permitem que você reutilize sua lógica de visualização.

Para criar um Partial, basta incluir um arquivo no seu diretório de views (**app/views/restaurantes**) com o seguinte nome: `_meupartial`. Repare que Partials devem obrigatoriamente começar com `_`.

Para utilizar um Partial em uma view, basta acrescentar a seguinte linha no ponto que deseja fazer a inclusão:

```
render partial: "meupartial"
```

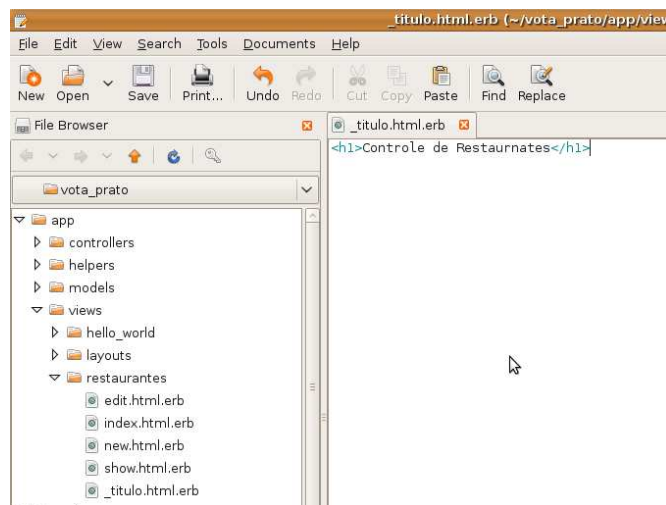
8.11 EXERCÍCIOS: CUSTOMIZANDO O CABEÇALHO

1) Vamos criar um partial:

a) Crie o arquivo: **app/views/restaurantes/_titulo.html.erb**

b) Coloque o seguinte conteúdo:

```
<h1>Controle de Restaurantes</h1><br/>
```

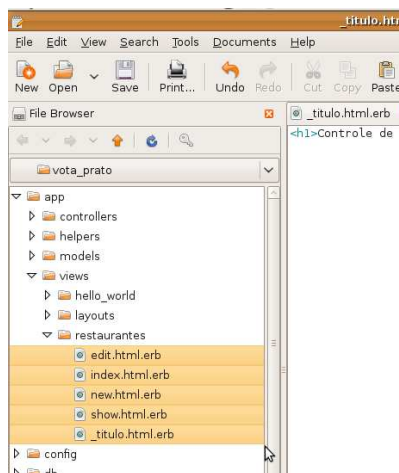


c) Abra todos os arquivos criados no exercício anterior (**app/views/restaurantes/***)

d) Insira a seguinte linha no início:

```
<%= render partial: "titulo" %>
```

Seu **app/views/restaurantes** deve estar com os seguintes arquivos:



e) Teste suas views: **http://localhost:3000/restaurantes**

8.12 LAYOUT

Como vimos, quando criamos um Partial, precisamos declará-lo em todas as páginas que desejamos utilizá-los. Existe uma alternativa melhor quando desejamos utilizar algum conteúdo estático que deve estar presente em todas as páginas: o **layout**.

Cada controller pode ter seu próprio layout, e uma alteração nesse arquivo se refletirá por todas as views desse controller. Os arquivos de layout devem ter o nome do controller, por exemplo **app/views/layouts/restaurantes.html.erb**.

Um arquivo de layout “padrão” tem o seguinte formato:

```
<html>
  <head>
    <title>Um título</title>
  </head>
  <body>
    <%= yield %>
  </body>
</html>
```

Tudo o que está nesse arquivo pode ser modificado, com exceção do `<%= yield %>`, que renderiza cada view do nosso controlador.

Podemos utilizar ainda o layout **application.html.erb**. Para isso, precisamos criar o arquivo **app/views/layouts/application.html.erb** e apagar os arquivos de layout dos controladores que queremos que utilizem o layout do controlador application. Com isso, se desejarmos ter um layout único para toda nossa aplicação, por exemplo, basta ter um único arquivo de layout.

8.13 EXERCÍCIOS: CRIANDO O HEADER

- 1) Nesse exercício, vamos utilizar o layout application para alterar o título de todas as janelas de nossa aplicação:
 - a) Abra o arquivo **app/views/layouts/application.html.erb**
 - b) Altere o title para **Programa de Qualificação de Restaurantes**

```
<!DOCTYPE html>
<html>
  <head>
    <title>Programa de Qualificação de Restaurantes</title>
    <title>VotaPrato</title>
    <%= stylesheet_link_tag "application", :media => "all" %>
    <%= javascript_include_tag "application" %>
```



```
<%= csrf_meta_tags %>
</head>
<body>
  <%= yield %>
</body>
</html>
```

c) Teste suas views: <http://localhost:3000/restaurantes>. Repare no título das janelas.

8.14 OUTRAS FORMAS DE GERAR A VIEW

O Rails já vem com suporte a outros *handlers* para geração de views. Além do ERB, podemos também usar o **Builder** e o **RJS**, que veremos mais adiante.

O **Builder** é adequado quando a view a ser gerada é um arquivo XML, já que permite a criação de um xml usando sintaxe Ruby. Veja um exemplo:

```
# app/views/authors/show.xml.builder
xml.author do
  xml.name('Alexander Pope')
end
```

O xml resultante é:

```
<author>
  <name>Alexander Pope</name>
</author>
```

Outra alternativa muito popular para a geração das views é o **HAML**:

<http://haml.hamptoncatlin.com>

```
#content
  .left.column
    %h2 Welcome to our site!
    %p= print_information
  .right.column= render partial: "sidebar"
```

E o equivalente com ERB:

```
<div id='content'>
  <div class='left column'>
    <h2>Welcome to our site!</h2>
```

```
<p>
  <%= print_information %>
</p>
</div>
<div class="right column">
  <%= render partial: "sidebar" %>
</div>
</div>
```

8.15 FILTROS

O módulo ActionController::Filters define formas de executar código antes e depois de todas as actions.

Para executar código antes das actions:

```
class ClientesController < ApplicationController
  before_filter :verifica_login

  private
  def verifica_login
    redirect_to controller: 'login' unless usuario_logado?
  end
end
```

De forma análoga, podemos executar código no fim do tratamento da requisição:

```
class ClientesController < ApplicationController
  after_filter :avisa_termino

  private
  def avisa_termino
    logger.info "Action #{params[:action]} terminada"
  end
end
```

Por fim, o mais poderoso de todos, que permite execução de código tanto antes, quanto depois da action a ser executada:

```
class ClientesController < ApplicationController
  around_filter :envolvendo_actions

  private
  def envolvendo_actions
```

```
    logger.info "Antes de #{params[:action]}: #{Time.now}"
    yield
    logger.info "Depois de #{params[:action]}: #{Time.now}"
  end
end
```

Os filtros podem também ser definidos diretamente na declaração, através de blocos:

```
class ClientesController < ApplicationController
  around_filter do |controller, action|
    logger.info "#{controller} antes: #{Time.now}"
    action.call
    logger.info "#{controller} depois: #{Time.now}"
  end
end
```

Caso não seja necessário aplicar os filtros a todas as actions, é possível usar as opções `:except` e `:only`:

```
class ClientesController < ApplicationController
  before_filter :verifica_login, :only => [:create, :update]

  # ...
end
```

LOGGER

As configurações do log podem ser feitas através do arquivo `config/environment.rb`, ou especificamente para cada environment nos arquivos da pasta `config/environments`. Entre as configurações que podem ser customizadas, estão qual nível de log deve ser exibido e para onde vai o log (stdout, arquivos, email, ...).

```
Rails::Initializer.run do |config|
  # ...
  config.log_level = :debug
  config.log_path = 'log/debug.log'
  # ...
end
```

Mais detalhes sobre a customização do log podem ser encontrados no wiki oficial do Rails:
<http://wiki.rubyonrails.org/rails/show/HowtoConfigureLogging>

Rotas e Rack

“Não é possível estar dentro da civilização e fora da arte”

– Rui Barbosa

O modo como urls são ligadas a controladores e actions pode ser customizado no Rails. O módulo responsável por esta parte é o que foi criado com o seu projeto `NomeDoProjeto::Application.routes` e as rotas podem ser customizadas no arquivo `config/routes.rb`.

9.1 RACK

O rack é uma abstração das requisições e respostas HTTP da maneira mais simples possível. Criando uma API unificada para servidores, frameworks, e softwares (os conhecidos middleware) em apenas uma chamada de método.

A grande motivação da criação do Rack é que, diferente do mundo java onde existe uma especificação que abstrai todo o HTTP, no mundo ruby cada framework havia criado a sua forma de tratar as requisições e respostas. Por isso, escrever um servidor ou mesmo permitir que o framework X pudesse rodar em um servidor que já existisse era um trabalho realmente complicado. Graças ao surgimento do rack e da sua padronização hoje é possível que qualquer servidor que conheça rack consiga executar qualquer aplicação que se comunique com o HTTP através do rack.

Mais do que isso, hoje também é possível fazer uma “aplicação” web em apenas uma linha. Exemplo:

```
run Proc.new {|env| [200, {"Content-Type" => "text/html"},  
  ["Hello World"]]}
```

Basta salvar esse arquivo, por exemplo como **hello.ru**, e subir nosso servidor pelo Terminal com o seguinte comando:

```
$ rackup hello.ru
```

Para criar uma “aplicação” em rack tudo o que precisamos é criar um método que retorne [statusCode, headers, body], como no exemplo acima.

O comando rackup é criado quando instalamos a gem ‘rack’ e serve para iniciar aplicações feitas em rack. Elas nada mais são que um arquivo ruby, mas devem ser salvos com a extensão .ru (RackUp) e devem chamar o método run.

9.2 EXERCÍCIOS - TESTANDO O RACK

1) Vamos fazer uma aplicação rack.

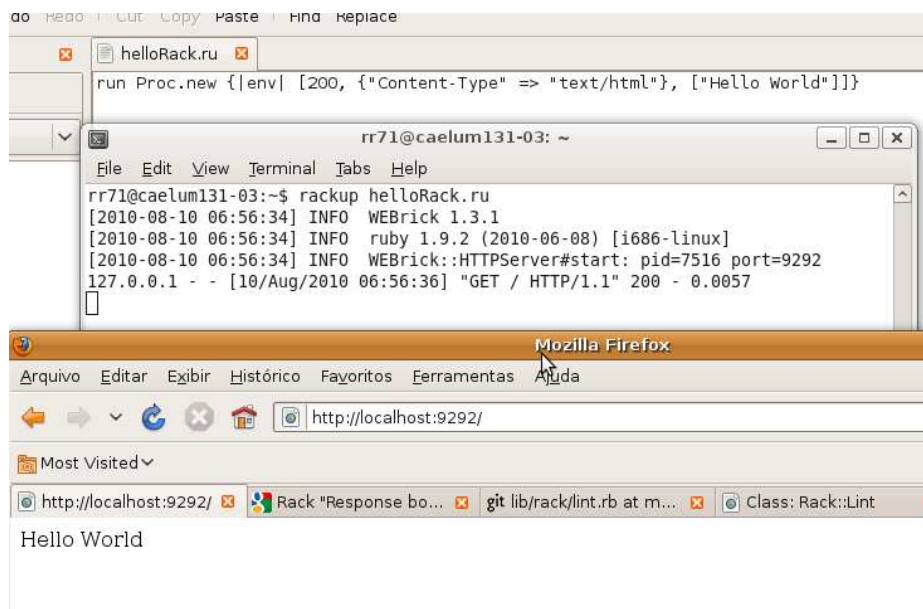
a) Crie um arquivo chamado **"helloRack.ru"**

b) Adicione as seguintes linhas:

```
$ run Proc.new {|env| [200, {"Content-Type" => "text/html"}, ["Hello World"]]}
```

c) Inicie a aplicação com o comando `rackup helloRack.ru`

d) Teste no browser pela url: `http://localhost:9292/`



9.3 ROUTES.RB

Veja abaixo como criar uma nova rota na nossa aplicação através do arquivo **config/routes.rb**:

```
VotaPrato::Application.routes.draw do
  match 'inicio', controller: 'restaurantes', action: 'index'
end
```

`match` cria uma nova rota e tem 2 formas de ser utilizado. Na nova maneira, ele recebe um único parâmetro que deve ser um hash, onde a chave do primeiro elemento é a rota que queremos mapear, e o valor para essa chave deve ser uma string com o nome do controlador e da action separado por um carácter '#', como no exemplo abaixo:

```
match 'inicio' => 'restaurantes#index'
```

Nesse exemplo o método `match` recebeu um hash onde a primeira chave é 'inicio' (rota a ser mapeada) e o valor para essa chave é 'restaurantes#index', ou seja, quando acessar `localhost:3000/inicio` o Rails vai executar a action `index` no controller `restaurantes`.

A outra abordagem é usar o método `match` recebendo dois parâmetros:

- url
- hash com o conjunto de parâmetros de requisição a serem preenchidos

No exemplo acima, para a url "localhost:3000/inicio" o método `index` do controlador de restaurantes (`RestaurantesController`) é chamado.

Qualquer parâmetro de requisição pode ser preenchido por uma rota. Tais parâmetros podem ser recuperados posteriormente através do hash de parâmetros da requisição, `params['nome']`.

Os parâmetros `:controller` e `:action` são especiais, representam o controlador e a action a serem executados.

Uma das características mais interessantes do rails é que as urls das rotas podem ser usadas para capturar alguns dos parâmetros:

```
match 'categorias/:nome', controller: 'categorias', action: 'show'
```

Neste caso, o parâmetro de requisição `params['nome']` é extraído da própria url!

ROTAS PADRÃO

Antigamente o Rails criava uma rota padrão. Hoje em dia ela continua no arquivo `config/routes.rb`, mas vem comentada por padrão:

```
match ':controller(/:action(/:id(.:format)))'
```

Esta rota padrão que nos permitiu usar o formato de url que temos usado até agora. O nome do controlador e a action a ser executada são retirados da própria url chamada.

Você pode definir quantas rotas forem necessárias. A ordem define a prioridade: rotas definidas no início tem mais prioridade que as do fim.

9.4 PRETTY URLS

A funcionalidade de roteamento embutida no Rails é bastante poderosa, podendo até substituir `mod_rewrite` em muitos casos. As rotas permitem uma grande flexibilidade para criação de urls que se beneficiem de técnicas de *Search Engine Optimization* (SEO).

Um exemplo interessante seria para um sistema de blog, que permitisse a exibição de posts para determinado ano:

```
match 'blog/:ano' => 'posts#list'
```

Ou ainda para um mês específico:

```
match 'blog/:ano/:mes' => 'posts#list'
```

Os parâmetros capturados pela url podem ter ainda valores *default*:

```
match 'blog(/:ano)' => 'posts#list', :ano => 2011
```

Para o último exemplo, a url `'http://localhost:3000/blog'` faria com que a action `list` do controlador `PostsController` fosse chamada, com o `params['ano']` sendo 2011.

9.5 NAMED ROUTES

Cada uma das rotas pode ter um nome único:

```
match 'blog/:ano' => 'posts#list', :as => 'posts'
```

O funcionamento é o mesmo de antes, com a diferença que usando o `'as'` demos um nome à rota.

Para cada uma das *Named Routes* são criados automaticamente dois helpers, disponíveis tanto nos controladores quanto nas views:

- `posts_path => '/blog/:ano'`
- `posts_url => 'http://localhost:3000/blog/:ano'`

A convenção para o nome dos helpers é sempre `nome_da_rota_path` e `nome_da_rota_url`.

Você pode ainda ver o roteamento para cada uma das urls disponíveis em uma aplicação rails com a ajuda de uma task do rake:

```
$ rake routes
```

9.6 REST - RESOURCES

REST é um modelo arquitetural para sistemas distribuídos. A ideia básica é que existe um conjunto fixo de operações permitidas (*verbs*) e as diversas aplicações se comunicam aplicando este conjunto fixo de operações em recursos (*nouns*) existentes, podendo ainda pedir diversas representações destes recursos.

A sigla REST vem de *Representational State Transfer* e surgiu da tese de doutorado de Roy Fielding, descrevendo as ideias que levaram a criação do protocolo HTTP. A web é o maior exemplo de uso de uma arquitetura REST, onde os verbos são as operações disponíveis no protocolo (GET, POST, DELETE, PUT, HEADER, ...), os recursos são identificados pelas URLs e as representações podem ser definidas através de *Mime Types*.

Ao desenhar aplicações REST, pensamos nos recursos a serem disponibilizados pela aplicação e em seus formatos, ao invés de pensar nas operações.

Desde o Rails 1.2, o estilo de desenvolvimento REST para aplicações web é encorajado pelo framework, que possui diversas facilidades para a adoção deste estilo arquitetural.

As operações disponíveis para cada um dos recursos são:

- **GET**: retorna uma representação do recurso
- **POST**: criação de um novo recurso
- **PUT**: altera o recurso
- **DELETE**: remove o recurso

Os quatro verbos do protocolo HTTP são comumente associados às operações de CRUD em sistemas *Restful*. Há uma grande discussão dos motivos pelos quais usamos *POST* para criação (*INSERT*) e *PUT* para alteração (*UPDATE*). A razão principal é que o protocolo HTTP especifica que a operação PUT deve ser *idempotente*, já POST não.

IDEMPOTÊNCIA

Operações idempotentes são operações que podem ser chamadas uma ou mais vezes, sem diferenças no resultado final. Idempotência é uma propriedade das operações.

A principal forma de suporte no Rails a estes padrões é através de rotas que seguem as convenções da arquitetura REST. Ao mapear um recurso no `routes.rb`, o Rails cria automaticamente as rotas adequadas no controlador para tratar as operações disponíveis no recurso (GET, POST, PUT e DELETE).

```
# routes.rb
resources :restaurantes
```

Ao mapear o recurso `:restaurantes`, o rails automaticamente cria as seguintes rotas:

- GET /restaurantes:controller => 'restaurantes', :action => 'index'
- POST /restaurantes:controller => 'restaurantes', :action => 'create'
- GET /restaurantes/new:controller => 'restaurantes', :action => 'new'
- GET /restaurantes/:id:controller => 'restaurantes', :action => 'show'
- PUT /restaurantes/:id:controller => 'restaurantes', :action => 'update'
- DELETE /restaurantes/:id:controller => 'restaurantes', :action => 'destroy'
- GET /restaurantes/:id/edit:controller => 'restaurantes', :action => 'edit'

Como é possível perceber através das rotas, todo recurso mapeado implica em sete métodos no controlador associado. São as famosas sete actions REST dos controladores rails.

Além disso, para cada rota criada, são criados os helpers associados, já que as rotas são na verdade *Named Routes*.

```
restaurantes_path      # => "/restaurantes"
new_restaurante_path    # => "/restaurantes/new"
edit_restaurante_path(3) # => "/restaurantes/3/edit"
```

Rails vem com um generator pronto para a criação de novos recursos. O controlador (com as sete actions), o modelo, a migration, os esqueleto dos testes (unitário, funcional e fixtures) e a rota podem ser automaticamente criados.

```
$ rails generate resource comentario
```

O gerador de scaffolds do Rails 2.0 em diante, também usa o modelo REST:

```
$ rails generate scaffold comentario conteudo:text author:string
```

Na geração do scaffold são produzidos os mesmos artefatos de antes, com a adição das views e de um layout padrão.

Não deixe de verificar as rotas criadas e seus nomes (*Named Routes*):

```
$ rake routes
```

9.7 ACTIONS EXTRAS EM RESOURCES

As sete actions disponíveis para cada resource costumam ser suficientes na maioria dos casos. Antes de colocar alguma action extra nos seus resources, exercite a possibilidade de criar um novo resource para tal.

Quando necessário, você pode incluir algumas actions extras para os resources:

```
resources :comentarios do
  member do
    post :desabilita
  end
  # url: /comentarios/:id/desabilita
  # named_route: desabilita_comentario_path
end
```

:member define actions que atuam sobre um recurso específico: */comentarios/1/desabilita*. Dentro do bloco member usar dessa forma `method :action`, onde method pode ser get, post, put, delete ou any.

Outro bloco que pode ser usado dentro de um resource é o `collection` que serve para definir actions extras que atuem sobre o conjunto inteiro de resources. Definirá rotas do tipo */comentarios/action*.

```
resources :comentarios do
  collection do
    get :feed
  end
  # url: /comentarios/feed
  # named_route: feed_comentarios_path
end
```

Para todas as actions extras, são criadas *Named Routes* adequadas. Use o rake `routes` como referência para conferir os nomes dados às rotas criadas.

9.8 DIVERSAS REPRESENTAÇÕES

Um controlador pode ter diversos resultados. Em outras palavras, controladores podem responder de diversas maneiras, através do método `respond_to`:

```
class MeuController < ApplicationController
  def list
    respond_to do |format|
      format.html
      format.js do
        render :update do |page|
          page.insert_html :top, 'div3', "Novo conteudo"
        end
      end
      format.xml
    end
  end
end
```

O convenção para o nome da view de resultado é sempre:

```
app/views/:controller/:action.:format.:handler
```

Os *handlers* disponíveis por padrão no Rails são: **erb**, **builder**, **rhtml**, **rxml** e **rjs**. Os formatos instalados por padrão ficam na constante `Mime::SET` e você pode instalar outros pelo arquivo `config/initializers/mime_types.rb`.

9.9 PARA SABER MAIS - NESTED RESOURCES

Quando há relacionamentos entre resources, podemos aninhar a definição das rotas, que o rails cria automaticamente as urls adequadas.

No nosso exemplo, `:restaurante` has_many `:qualificacoes`, portanto:

```
# routes.rb
resources :restaurantes do
  resources :qualificacoes
end
```

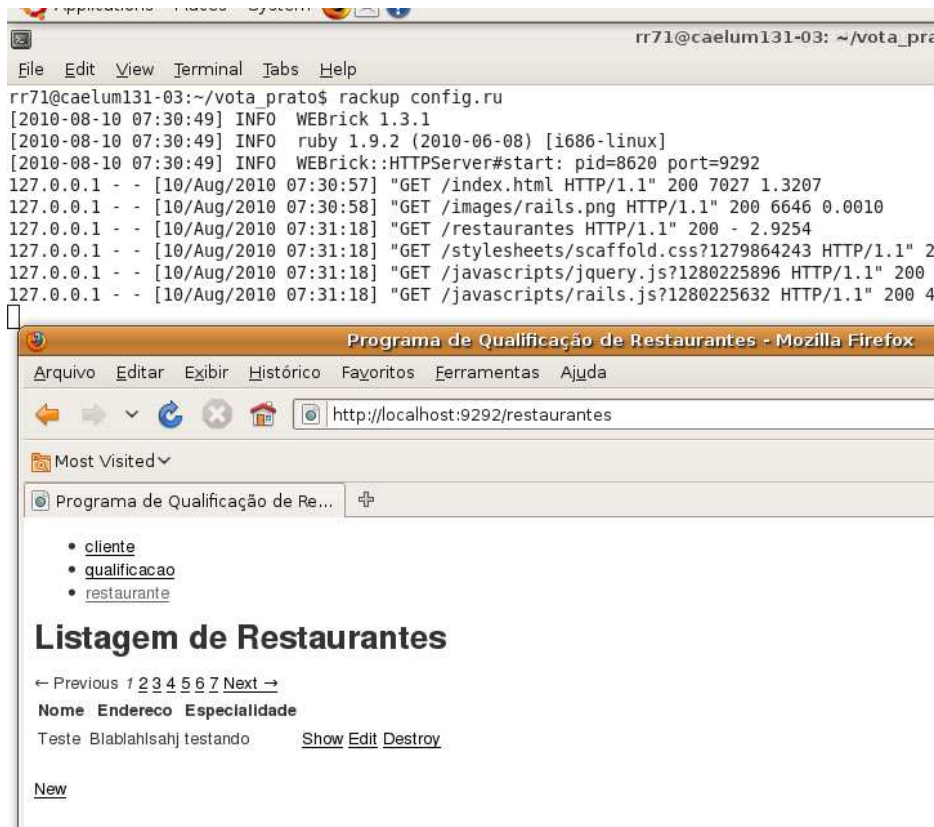
A rota acima automaticamente cria as rotas para qualificações específicas de um restaurante:

- GET /restaurante/:restaurante_id/qualificacoes:controller => 'qualificacoes', :action => 'index'
- GET /restaurante/:restaurante_id/qualificacoes/:id:controller => 'qualificacoes', :action => 'show'
- GET /restaurante/:restaurante_id/qualificacoes/new:controller => 'qualificacoes', :action => 'new'
- ...

As sete rotas comuns são criadas para o recurso `:qualificacao`, mas agora as rotas de `:qualificacoes` são sempre específicas a um `:restaurante` (todos os métodos recebem o `params['restaurante_id']`).

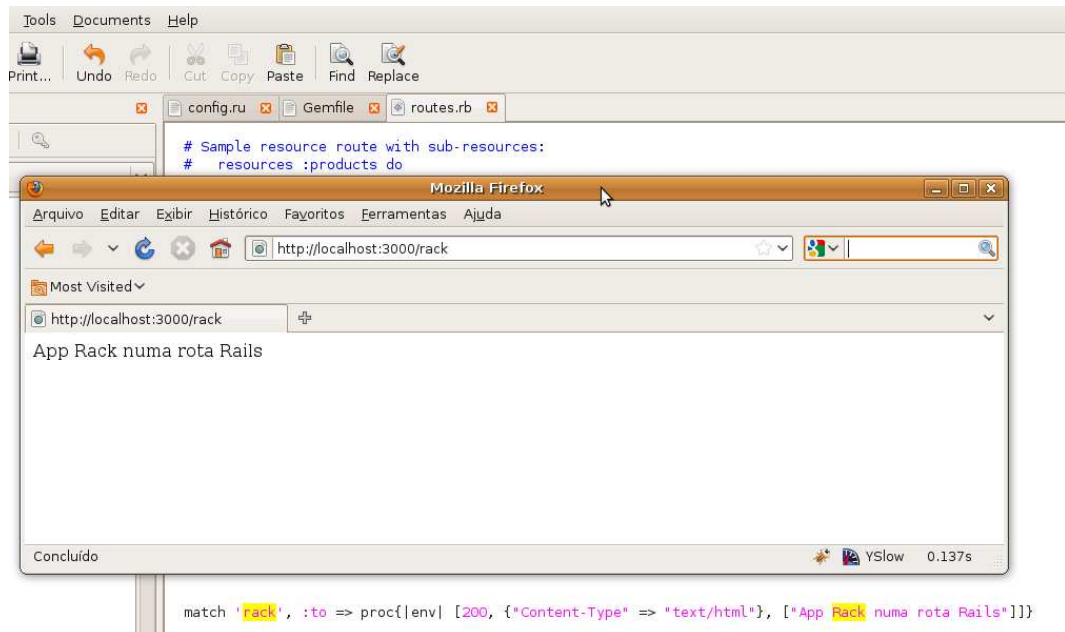
9.10 RAILS E O RACK

A partir do Rails 3, quando criamos uma nova aplicação, um dos arquivos que ele cria é `config.ru` na raiz do projeto e, mais que isso, podemos afirmar que toda aplicação Rails 3 é uma aplicação rack. Prova disso é que conseguimos iniciar a aplicação através do comando `rackup config.ru`



Outro ponto interessante sobre o rack e o Rails, é que agora é possível mapear uma aplicação rack diretamente em uma rota de uma aplicação rails.

```
# routes.rb
match 'rack',
  :to => proc{|env| [200, {"Content-Type" => "text/html"},
    ["App Rack numa rota Rails"]]}
```



9.11 EXERCÍCIOS - CRIANDO UM ROTA PARA UMA APLICAÇÃO RACK

1) Vamos fazer uma aplicação rack.

a) Abra o arquivo **"routes.rb"**

b) Adicione a seguinte linha:

```
match 'rack',  
  :to => proc{|env| [200, {"Content-Type" => "text/html"},  
    ["App Rack numa rota Rails"]]}
```

c) Inicie a aplicação com o comando `rails server`

d) Teste no browser pela url: `http://localhost:3000/rack`

Completando o Sistema

“O êxito parece doce a quem não o alcança”

– Dickinson, Emily

10.1 UM POUCO MAIS SOBRE O SCAFFOLD

Vimos que quando usamos o gerador `scaffold` o Rails cria os arquivos necessários em todas as camadas. **Controller, Model, Views** e até mesmo arquivos de testes e a **Migration**. Para concluir nossos projeto precisamos criar apenas **Controller + Views** pois tanto o modelo quanto as migrations já estão prontos. Para isso vamos usar o mesmo gerador `scaffold` mas vamos passar os parâmetros: `--migration=false` para ele ignorar a criação da migration e o parâmetro `-s` que é a abreviação para `--skip` que faz com que o rails “pule” os arquivos já existentes.

Para concluir os modelos que já começamos vamos executar o gerador da seguinte maneira: **`rails generate scaffold cliente nome:string idade:integer --migration=false -s`**

OUTROS PARÂMETROS NOS GERADORES

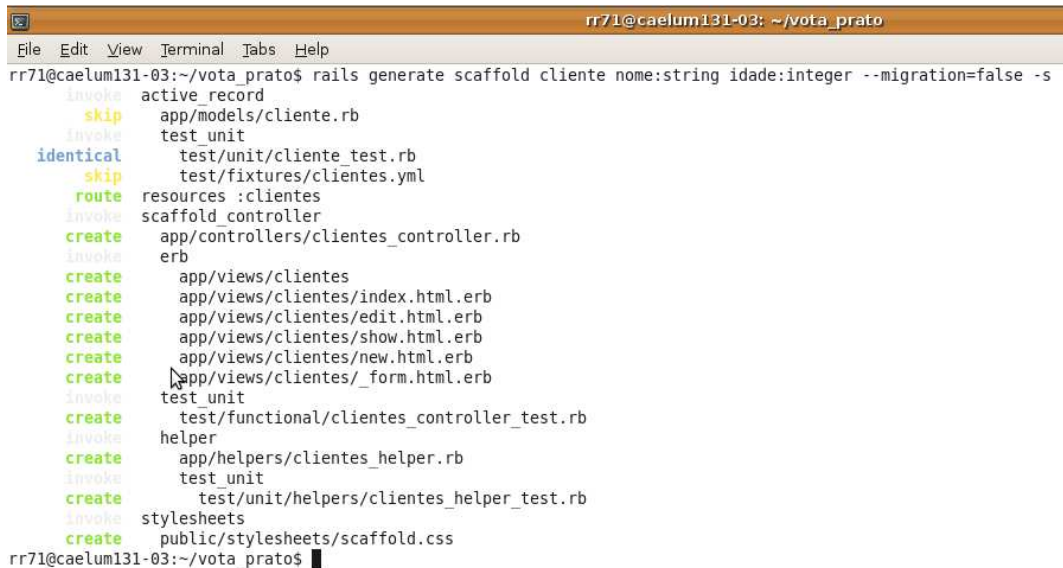
Para conhecer todas as opções de parâmetros que um gerador pode receber tente executá-lo passando o parâmetro `-h` Ex. **`rails generate scaffold -h`**

10.2 EXERCÍCIOS

Vamos gerar os outros controllers e views usando o `scaffold`:

1) Primeiro vamos gerar o `scaffold` para **cliente**, no terminal execute:

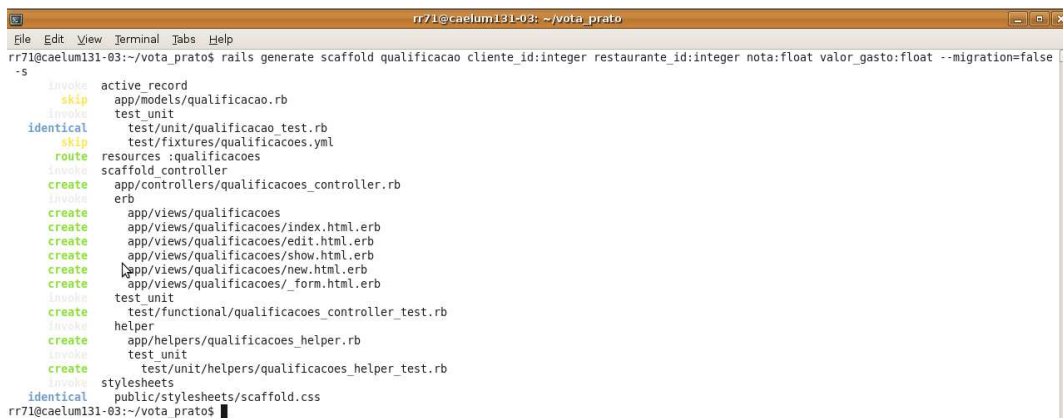
```
$ rails generate scaffold cliente nome:integer --migration=false -s
```



```
rr71@caelum131-03: ~/vota_prato
File Edit View Terminal Tabs Help
rr71@caelum131-03:~/vota_prato$ rails generate scaffold cliente nome:string idade:integer --migration=false -s
  invoke  active_record
  skip    app/models/cliente.rb
  invoke  test_unit
  identical test/unit/cliente_test.rb
  skip    test/fixtures/clientes.yml
  route   resources :clientes
  invoke  scaffold_controller
  create  app/controllers/clientes_controller.rb
  invoke  erb
  create  app/views/clientes
  create  app/views/clientes/index.html.erb
  create  app/views/clientes/edit.html.erb
  create  app/views/clientes/show.html.erb
  create  app/views/clientes/new.html.erb
  create  app/views/clientes/_form.html.erb
  invoke  test_unit
  create  test/functional/clientes_controller_test.rb
  invoke  helper
  create  app/helpers/clientes_helper.rb
  invoke  test_unit
  create  test/unit/helpers/clientes_helper_test.rb
  invoke  stylesheets
  create  public/stylesheets/scaffold.css
rr71@caelum131-03:~/vota_prato$
```

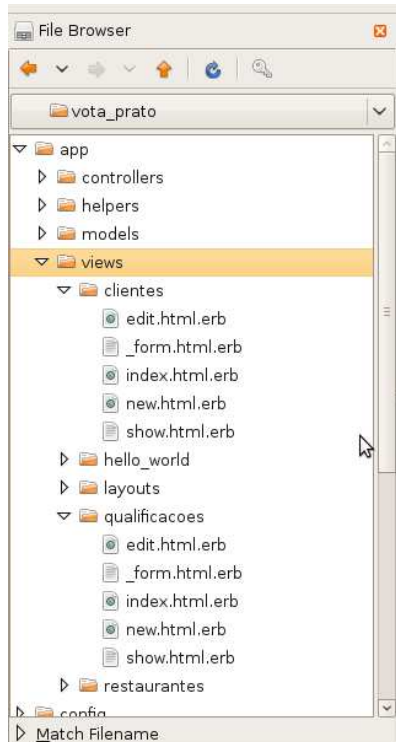
2) Agora vamos gerar o scaffold de **qualificacao**, execute:

```
$ rails generate scaffold qualificacao cliente_id:integer
  restaurante_id:integer nota:float valor_gasto:float
  --migration=false -s**
```

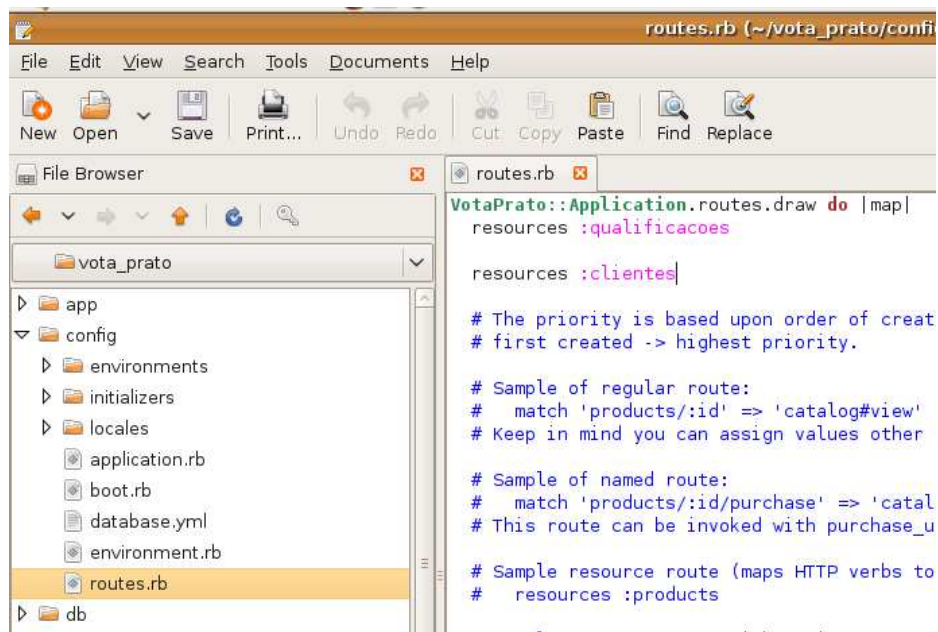


```
rr71@caelum131-03: ~/vota_prato
File Edit View Terminal Tabs Help
rr71@caelum131-03:~/vota_prato$ rails generate scaffold qualificacao cliente_id:integer restaurante_id:integer nota:float valor_gasto:float --migration=false -s
  invoke  active_record
  skip    app/models/qualificacao.rb
  invoke  test_unit
  identical test/unit/qualificacao_test.rb
  skip    test/fixtures/qualificacoes.yml
  route   resources :qualificacoes
  invoke  scaffold_controller
  create  app/controllers/qualificacoes_controller.rb
  invoke  erb
  create  app/views/qualificacoes
  create  app/views/qualificacoes/index.html.erb
  create  app/views/qualificacoes/edit.html.erb
  create  app/views/qualificacoes/show.html.erb
  create  app/views/qualificacoes/new.html.erb
  create  app/views/qualificacoes/_form.html.erb
  invoke  test_unit
  create  test/functional/qualificacoes_controller_test.rb
  invoke  helper
  create  app/helpers/qualificacoes_helper.rb
  invoke  test_unit
  create  test/unit/helpers/qualificacoes_helper_test.rb
  invoke  stylesheets
  identical public/stylesheets/scaffold.css
rr71@caelum131-03:~/vota_prato$
```

3) a) Olhe as views criadas (**app/views/clientes** e **app/views/qualificacoes**)



b) Olhe as rotas criadas (**config/routes.rb**)



- c) Abra os arquivos **app/views/clientes/index.html.erb** e **app/views/restaurantes/index.html.erb** e apague as linhas que chamam a action destroy Lembre-se de que não queremos inconsistências na nossa tabela de qualificações
- d) Reinicie o servidor

e) Teste: <http://localhost:3000/clientes> e <http://localhost:3000/qualificacoes>

Note que precisamos utilizar a opção “--migration=false” no comando `scaffold`, além de informar manualmente os atributos utilizados em nossas migrations. Isso foi necessário, pois já tínhamos um migration pronto, e queríamos que o Rails gerasse os formulários das views para nós, e para isso ele precisaria conhecer os atributos que queríamos utilizar.

CSS SCAFFOLD

O comando `scaffold`, quando executado, gera um css mais bonito para nossa aplicação. Se quiser utilizá-lo, edite nosso layout (**`app/views/layouts/application.html.erb`**) e adicione a seguinte linha logo abaixo da tag `<title>`:

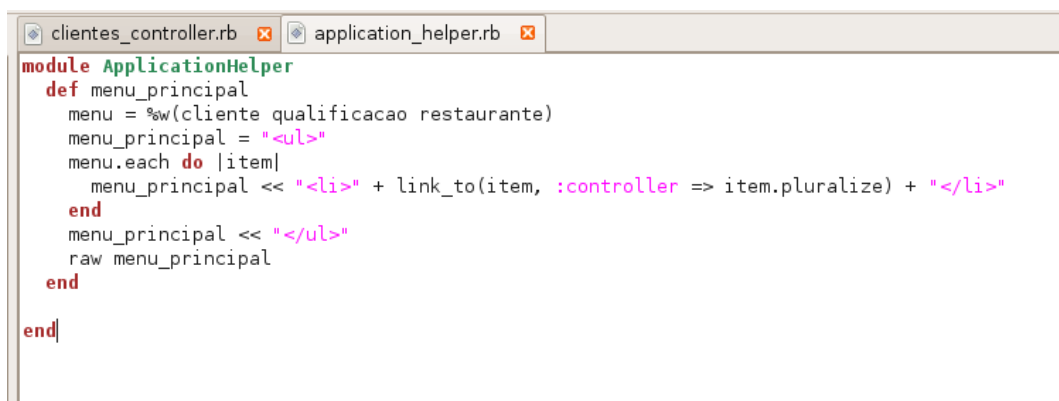
```
<%= stylesheet_link_tag 'scaffold' %>
```

4) Vamos agora incluir **Helpers** no nosso **Layout**, para poder navegar entre Restaurante, Cliente e Qualificação sem precisarmos digitar a url:

a) Abra o Helper “applications”: “**`app/helpers/application_helper.rb`**”

b) Digite o seguinte método:

```
def menu_principal
  menu = %w(cliente qualificacao restaurante)
  menu_principal = "<ul>"
  menu.each do |item|
    menu_principal << "<li>"
    menu_principal << link_to(item, :controller => item.pluralize)
    menu_principal << "</li>"
  end
  menu_principal << "</ul>"
  raw menu_principal
end
```



```
module ApplicationHelper
  def menu_principal
    menu = %w(cliente qualificacao restaurante)
    menu_principal = "<ul>"
    menu.each do |item|
      menu_principal << "<li>" + link_to(item, :controller => item.pluralize) + "</li>"
    end
    menu_principal << "</ul>"
    raw menu_principal
  end
end
```

- c) Adicione a chamada ao método `<%=menu_principal %>` no nosso layout (`app/views/layouts/application.html.erb`), ficando desse jeito:

```
...  
<body>  
  <%= menu_principal %>  
  
  <%= yield %>  
</body>  
...
```

- d) Teste: <http://localhost:3000/restaurantes>

10.3 SELECIONANDO CLIENTES E RESTAURANTE NO FORM DE QUALIFICAÇÕES

Você já deve ter reparado que nossa view de adição e edição de qualificações está um tanto quanto estranha: precisamos digitar os IDs do cliente e do restaurante manualmente.

Para corrigir isso, podemos utilizar o **FormHelper** `select`, inserindo o seguinte código nas nossas views de adição e edição de qualificações:

```
<%= select('qualificacao', 'cliente_id',  
          Cliente.order(:nome)  
          {|p| [ p.nome, p.id]}) %>
```

em substituição ao:

```
<%= f.text_field :cliente_id %>
```

Mas existe um outro **FormHelper** mais elegante, que produz o mesmo efeito, o `collection_select`:

```
<%= collection_select(:qualificacao, :cliente_id,  
  Cliente.order(:nome),  
  :id, :nome, {:prompt => true}) %>
```

Como estamos dentro de um `form_for`, podemos usar do fato de que o formulário sabe qual o nosso `ActiveRecord`, e com isso fazer apenas:

```
<%= f.collection_select(:cliente_id,  
  Cliente.order(:nome),  
  :id, :nome, {:prompt => true}) %>
```

10.4 EXERCÍCIOS

- 1) Vamos utilizar o **FormHelper** `collection_select` para exibirmos o nome dos clientes e restaurantes nas nossas views da qualificação:

a) Abra o arquivo **app/views/qualificacoes/_form.html.erb**

b) Troque a linha:

```
<%= f.text_field :cliente_id %>
```

por:

```
<%= f.collection_select(:cliente_id, Cliente.order('nome'),  
                        :id, :nome, {:prompt => true}) %>
```

c) Troque a linha:

```
<%= f.text_field :restaurante_id %>
```

por:

```
<%= f.collection_select(:restaurante_id, Restaurante.order('nome'),  
                        :id, :nome, {:prompt => true}) %>
```

d) Teste: **http://localhost:3000/qualificacoes**

- 2) Agora vamos exibir o nome dos restaurantes e clientes nas views **index** e **show** de qualificações:

a) Abra o arquivo **app/views/qualificacoes/show.html.erb**

b) Troque a linha:

```
<%= @qualificacao.cliente_id %>
```

por:

```
<%= @qualificacao.cliente.nome %>
```

c) Troque a linha:

```
<%= @qualificacao.restaurante_id %>
```

por:

```
<%= @qualificacao.restaurante.nome %>
```

d) Abra o arquivo **app/views/qualificacoes/index.html.erb**

e) Troque as linhas:

```
<td><%= qualificacao.cliente_id %></td>
```

```
<td><%= qualificacao.restaurante_id %></td>
```

por:

```
<td><%= qualificacao.cliente.nome %></td>
<td><%= qualificacao.restaurante.nome %></td>
```

f) Teste: **http://localhost:3000/qualificacoes**

3) Por fim, vamos utilizar o FormHelper `hidden_field` para permitir a qualificação de um restaurante a partir da view **show** de um cliente ou de um restaurante. No entanto, ao fazer isso, queremos que não seja necessário a escolha de cliente ou restaurante. Para isso:

a) Abra o arquivo **app/views/qualificacoes/_form.html.erb**

b) Troque as linhas

```
<p>
  <%= f.label :cliente_id %><br />
  <%= f.collection_select(:cliente_id,
    Cliente.order('nome'),
    :id, :nome, {:prompt => true}) %>
</p>
```

por:

```
<% if @qualificacao.cliente %>
  <%= f.hidden_field 'cliente_id' %>
<% else %>
  <p><%= f.label :cliente_id %><br />
  <%= f.collection_select(:cliente_id, Cliente.order('nome'),
    :id, :nome, {:prompt => true}) %></p>
<% end %>
```

c) Troque as linhas

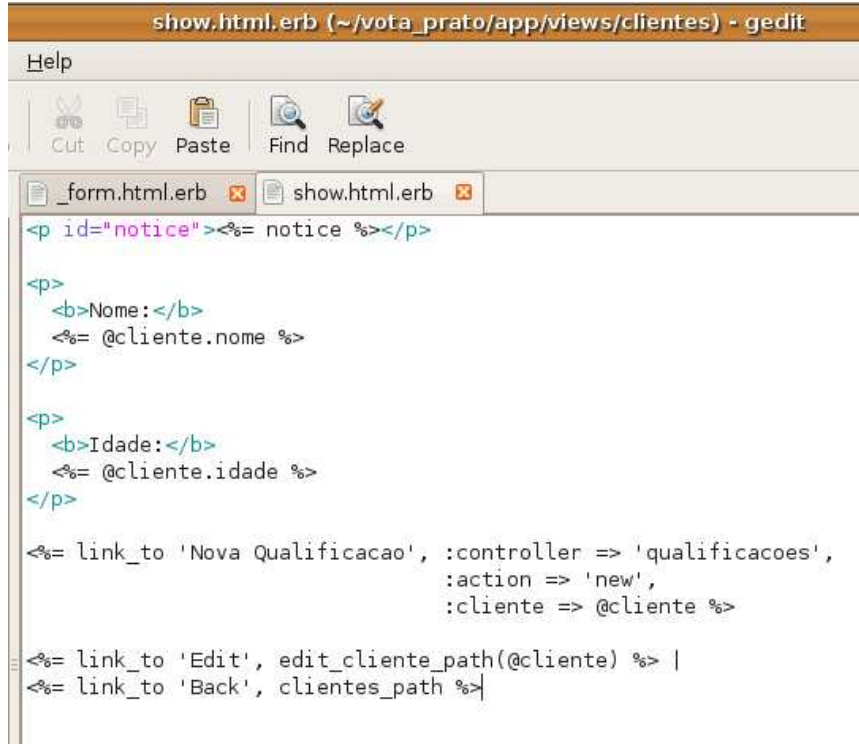
```
<p>
  <%= f.label :restaurante_id %><br />
  <%= f.collection_select(:restaurante_id, Restaurante.order('nome'),
    :id, :nome, {:prompt => true}) %>
</p>
```

por:

```
<% if @qualificacao.restaurante %>
  <%= f.hidden_field 'restaurante_id' %>
<% else %>
  <p><%= f.label :restaurante_id %><br />
  <%= f.collection_select(:restaurante_id, Restaurante.order('nome'),
    :id, :nome, {:prompt => true}) %></p>
<% end %>
```

d) Adicione a seguinte linha na view **show** do cliente (**app/views/clientes/show.html.erb**):

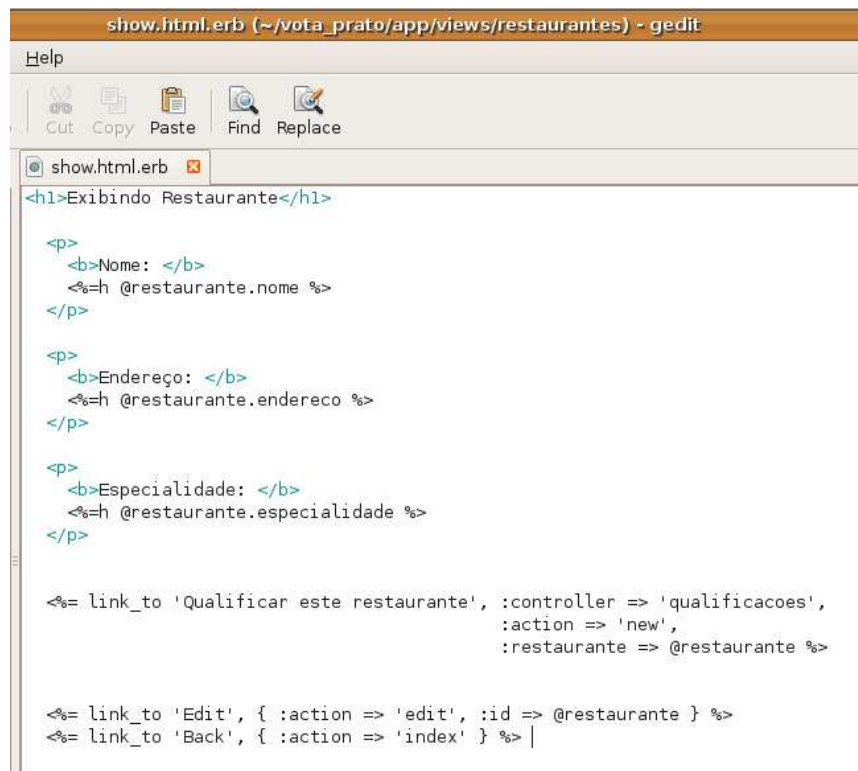
```
<%= link_to "Nova qualificação", :controller => "qualificacoes",  
      :action => "new",  
      :cliente => @cliente %>
```



```
show.html.erb (~/.vota_prato/app/views/clientes) - gedit  
Help  
Cut Copy Paste Find Replace  
_form.html.erb show.html.erb  
<p id="notice"><%= notice %></p>  
  
<p>  
  <b>Nome:</b>  
  <%= @cliente.nome %>  
</p>  
  
<p>  
  <b>Idade:</b>  
  <%= @cliente.idade %>  
</p>  
  
<%= link_to 'Nova Qualificacao', :controller => 'qualificacoes',  
      :action => 'new',  
      :cliente => @cliente %>  
  
<%= link_to 'Edit', edit_cliente_path(@cliente) %> |  
<%= link_to 'Back', clientes_path %>
```

e) Adicione a seguinte linha na view **show** do restaurante (**app/views/restaurantes/show.html.erb**):

```
<%= link_to "Qualificar este restaurante", :controller => "qualificacoes",  
      :action => "new",  
      :restaurante => @restaurante %>
```



```
show.html.erb (~/.vota_prato/app/views/restaurantes) - gedit
Help
Cut Copy Paste Find Replace
show.html.erb
<h1>Exibindo Restaurante</h1>

<p>
  <b>Nome: </b>
  <%=h @restaurante.nome %>
</p>

<p>
  <b>Endereço: </b>
  <%=h @restaurante.endereco %>
</p>

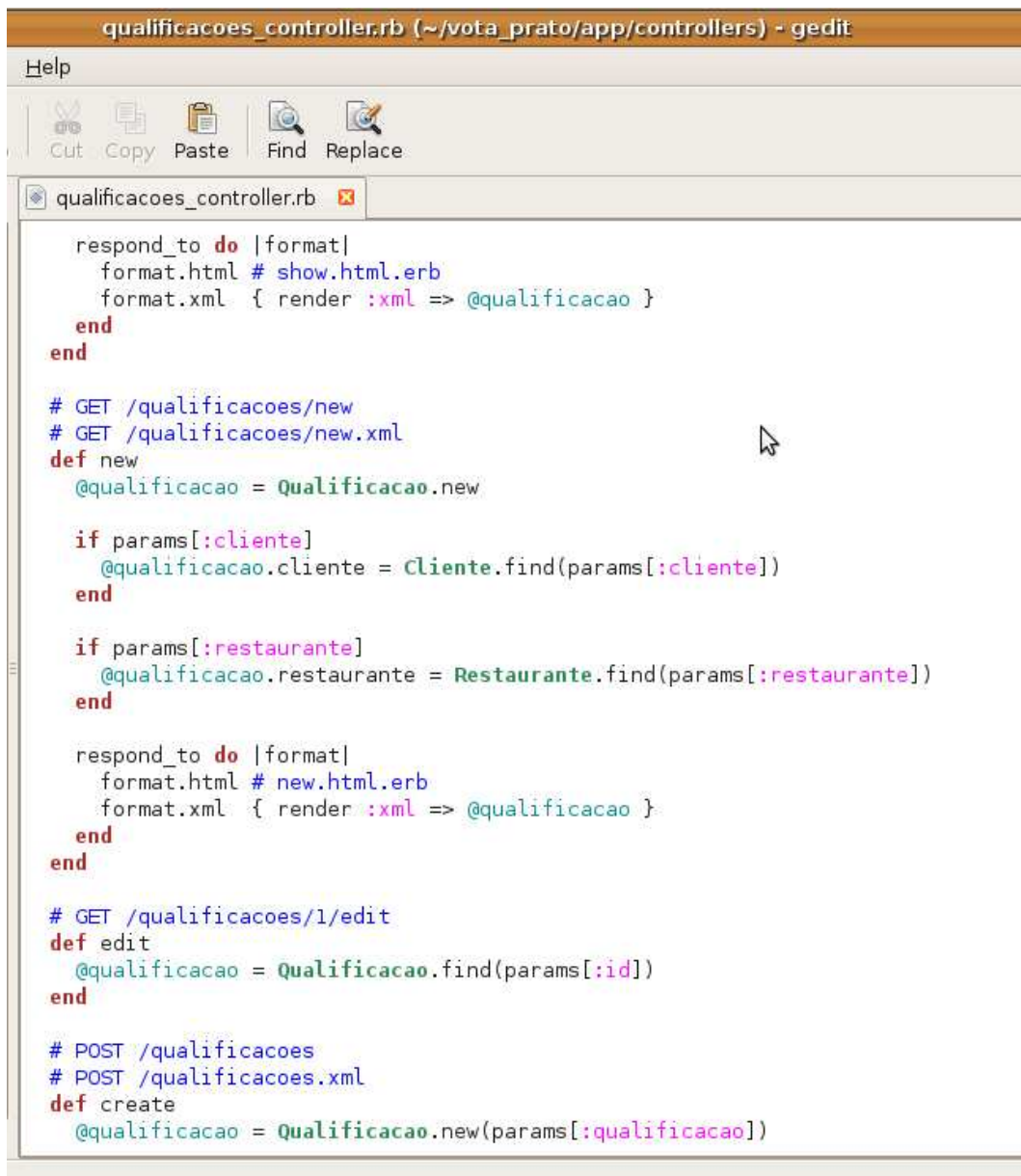
<p>
  <b>Especialidade: </b>
  <%=h @restaurante.especialidade %>
</p>

<%= link_to 'Qualificar este restaurante', :controller => 'qualificacoes',
          :action => 'new',
          :restaurante => @restaurante %>

<%= link_to 'Edit', { :action => 'edit', :id => @restaurante } %>
<%= link_to 'Back', { :action => 'index' } %> |
```

- f) Por fim, precisamos fazer com que o controlador da action new das qualificações receba os parâmetros para preenchimento automático. Abra o controller **app/controllers/qualificacoes_controller.rb**
- g) Adicione as seguintes linhas à nossa action new:

```
if params[:cliente]
  @qualificacao.cliente = Cliente.find(params[:cliente])
end
if params[:restaurante]
  @qualificacao.restaurante = Restaurante.find(params[:restaurante])
end
```



```
qualificacoes_controller.rb (~/vota_prato/app/controllers) - gedit
Help
Cut Copy Paste Find Replace

qualificacoes_controller.rb

  respond_to do |format|
    format.html # show.html.erb
    format.xml { render :xml => @qualificacao }
  end
end

# GET /qualificacoes/new
# GET /qualificacoes/new.xml
def new
  @qualificacao = Qualificacao.new

  if params[:cliente]
    @qualificacao.cliente = Cliente.find(params[:cliente])
  end

  if params[:restaurante]
    @qualificacao.restaurante = Restaurante.find(params[:restaurante])
  end

  respond_to do |format|
    format.html # new.html.erb
    format.xml { render :xml => @qualificacao }
  end
end

# GET /qualificacoes/1/edit
def edit
  @qualificacao = Qualificacao.find(params[:id])
end

# POST /qualificacoes
# POST /qualificacoes.xml
def create
  @qualificacao = Qualificacao.new(params[:qualificacao])
```

- h) Teste: **http://localhost:3000/clientes**, entre na página Show de um cliente e faça uma nova qualificação.

10.5 EXERCÍCIOS OPCIONAIS

- 1) Crie um método no nosso Application Helper para converter um número para valor monetário:
 - a) Abra o arquivo **app/helpers/application_helper.rb**
 - b) Adicione o seguinte método:

```
def valor_formatado(number)
  number_to_currency(number, :unit => "R$",
```

```
        :separator => ",", :delimiter => ".")  
    end
```

- c) Em `app/views/qualificacoes/index.html.erb` e `app/views/qualificacoes/show.html.erb`, troque o seguinte código:

```
@qualificacao.valor_gasto  
  
por:  
  
valor_formatado(@qualificacao.valor_gasto)
```

10.6 MAIS SOBRE OS CONTROLLERS

Podemos notar que nossas actions, por exemplo a `index`, fica muito parecida com a action `index` de outros controladores, mudando apenas o nome do modelo em questão.

```
#qualificacoes_controller  
def index  
  @qualificacoes = Qualificacao.all  
  
  respond_to do |format|  
    format.html # index.html.erb  
    format.xml { render :xml => @qualificacoes }  
  end  
end  
  
#clientes_controller  
def index  
  @clientes = Cliente.all  
  
  respond_to do |format|  
    format.html # index.html.erb  
    format.xml { render :xml => @clientes }  
  end  
end
```

Normalmente precisamos fazer exatamente a mesma ação para formatos iguais e por isso acabamos repetindo o mesmo bloco de `respond_to` nas actions. Para solucionar esse problema, no rails 3 acrescentaram o método `respond_to` **nos controllers** e o método `respond_with` **nas actions**. Veja o exemplo:

```
class ClientesController < ApplicationController  
  
  respond_to :html, :xml
```



```
# GET /clientes
# GET /clientes.xml
def index
  @clientes = Cliente.all

  respond_with @clientes
end

# GET /clientes/1
# GET /clientes/1.xml
def show
  @cliente = Cliente.find(params[:id])

  respond_with @cliente
end
...
end
```

Dessa forma estamos dizendo para o rails que esse controller irá responder para os formatos html e xml, dentro da action basta eu dizer qual objeto é pra ser usado. No caso da action **index**, se a requisição pedir o formato html, o rails simplesmente vai enviar a chamada para o arquivo `views/clientes/index.html.erb` e a variável `@clientes` estará disponível lá. Se a requisição pedir o formato xml, o rails fará exatamente o mesmo que estava no bloco de `respond_to` que o scaffold criou, vai renderizar a variável `@clientes` em xml. O bloco de código acima é equivalente ao gerado pelo scaffold:

```
class ClientesController < ApplicationController
  # GET /clientes
  # GET /clientes.xml
  def index
    @clientes = Cliente.all

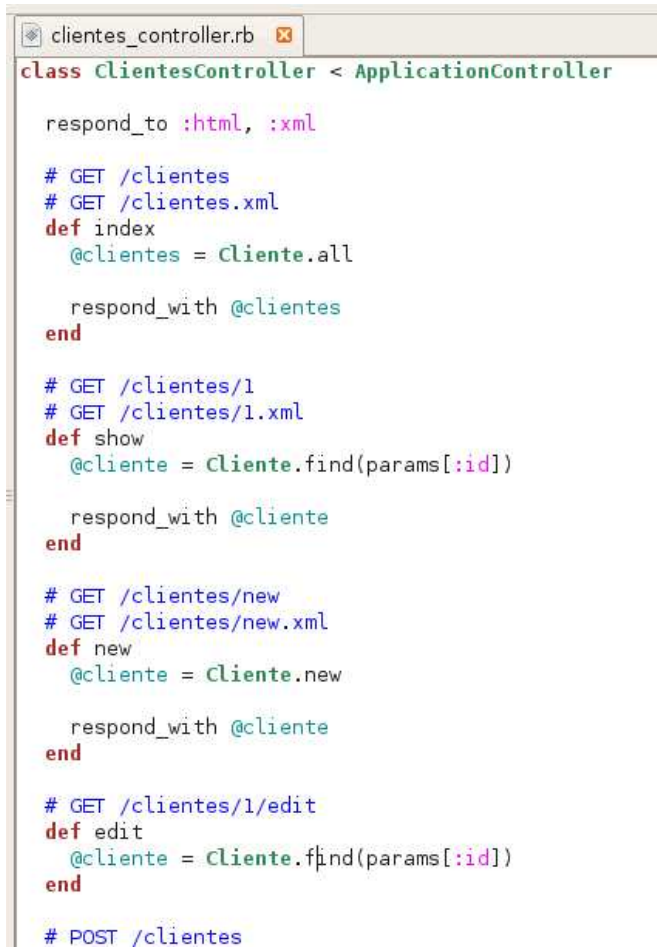
    respond_to do |format|
      format.html # index.html.erb
      format.xml { render :xml => @clientes }
    end
  end

  # GET /clientes/1
  # GET /clientes/1.xml
  def show
    @cliente = Cliente.find(params[:id])

    respond_to do |format|
      format.html # show.html.erb
      format.xml { render :xml => @cliente }
    end
  end
end
```

```
end
end
...
end
```

Veja na imagem como ficaria o **ClientesController** usando essa outra maneira de configurar os controllers.



```
clientes_controller.rb
class ClientesController < ApplicationController
  respond_to :html, :xml

  # GET /clientes
  # GET /clientes.xml
  def index
    @clientes = Cliente.all

    respond_with @clientes
  end

  # GET /clientes/1
  # GET /clientes/1.xml
  def show
    @cliente = Cliente.find(params[:id])

    respond_with @cliente
  end

  # GET /clientes/new
  # GET /clientes/new.xml
  def new
    @cliente = Cliente.new

    respond_with @cliente
  end

  # GET /clientes/1/edit
  def edit
    @cliente = Cliente.find(params[:id])
  end

  # POST /clientes
```

Calculations

“Ao examinarmos os erros de um homem conhecemos o seu caráter”

– Chamfort, Sébastien Roch

Nesse capítulo, você aprenderá a utilizar campos para calcular fórmulas como, por exemplo, a média de um campo.

11.1 MÉTODOS

Uma vez que existem os campos valor gasto e nota, seria interessante disponibilizar para os visitantes do site a média de cada um desses campos para determinado restaurante.

Em Rails esse recurso é chamado **calculations**, métodos dos nossos modelos que fazem operações mais comuns com campos numéricos como, por exemplo:

- `average(column_name, options = {})` - média
- `maximum(column_name, options = {})` - maior valor
- `minimum(column_name, options = {})` - menor valor
- `sum(column_name, options = {})` - soma
- `count(*args)` - número de entradas

11.2 MÉDIA

Supondo que o cliente pediu para adicionar a nota média de um restaurante na tela com as informações do mesmo (**show**). Basta adicionar uma chamada ao método `average` das qualificações do nosso restaurante:

```
<b>Nota média: </b><%= @restaurante.qualificacoes.average(:nota) %><br/>
```

Podemos mostrar também o número total de qualificações que determinado restaurante possui:

```
<b>Qualificações: </b><%= @restaurante.qualificacoes.count %><br/>
```

E, por último, fica fácil adicionar o valor médio gasto pelos clientes que visitam tal restaurante:

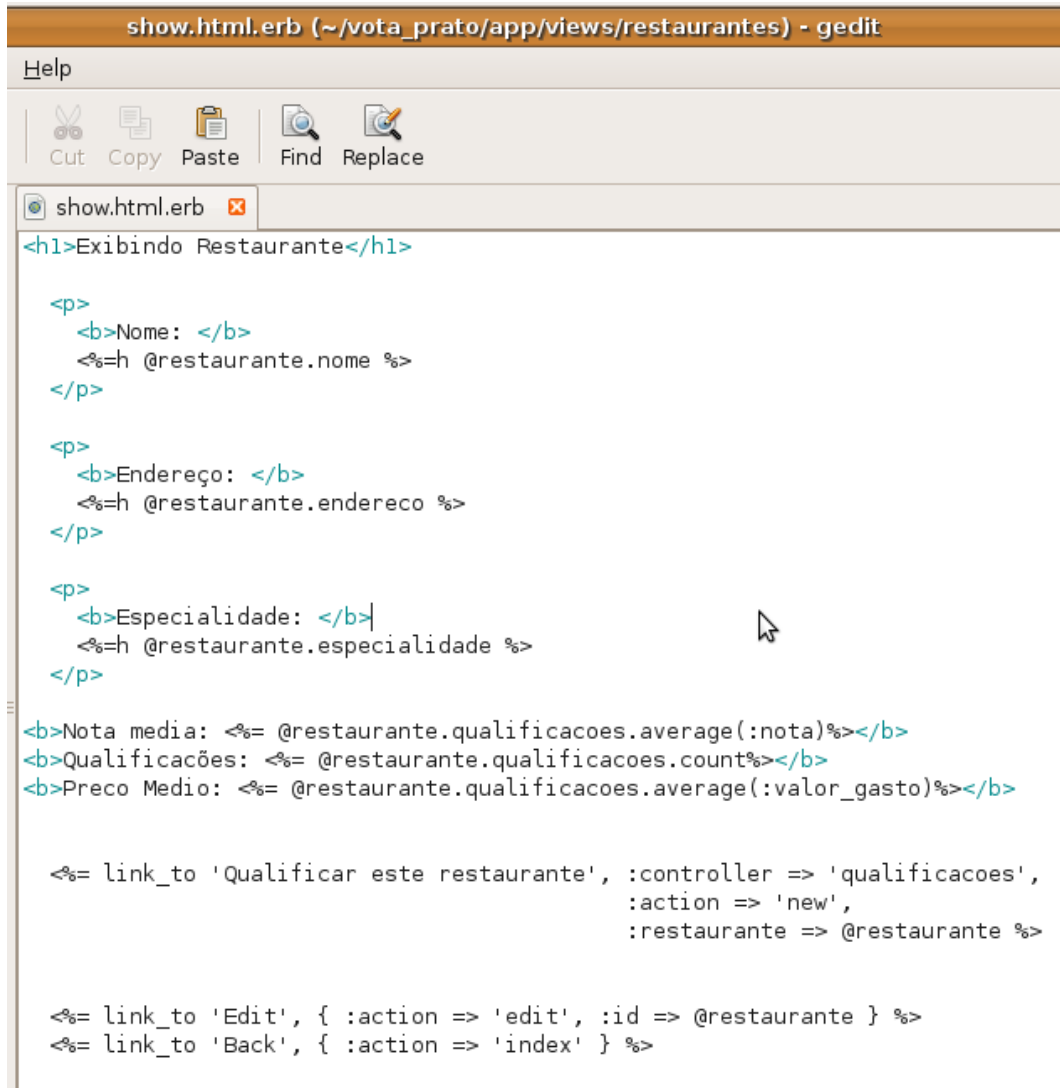
```
<b>Preço médio: </b><%= @restaurante.qualificacoes.average(:valor_gasto) %><br/>
```

11.3 EXERCÍCIOS

1) Altere a view **show** de restaurante para mostrar sua nota média, quantas qualificações possui e preço médio:

a) Insira as seguintes linhas em **app/views/restaurantes/show.html.erb**:

```
<b>Nota média: </b>
    <%= @restaurante.qualificacoes.average(:nota) %><br/>
<b>Qualificações: </b>
    <%= @restaurante.qualificacoes.count %><br/>
<b>Preço médio: </b>
    <%= @restaurante.qualificacoes.average(:valor_gasto) %><br/><br/>
```



```
show.html.erb (~/.vota_prato/app/views/restaurantes) - gedit
Help
Cut Copy Paste Find Replace
show.html.erb
<h1>Exibindo Restaurante</h1>

<p>
  <b>Nome: </b>
  <%=h @restaurante.nome %>
</p>

<p>
  <b>Endereço: </b>
  <%=h @restaurante.endereco %>
</p>

<p>
  <b>Especialidade: </b>|
  <%=h @restaurante.especialidade %>
</p>

<b>Nota media: <%= @restaurante.qualificacoes.average(:nota)%></b>
<b>Qualificações: <%= @restaurante.qualificacoes.count%></b>
<b>Preco Medio: <%= @restaurante.qualificacoes.average(:valor_gasto)%></b>

<%= link_to 'Qualificar este restaurante', :controller => 'qualificacoes',
          :action => 'new',
          :restaurante => @restaurante %>

<%= link_to 'Edit', { :action => 'edit', :id => @restaurante } %>
<%= link_to 'Back', { :action => 'index' } %>
```

b) Entre no link <http://localhost:3000/restaurantes> e escolha um restaurante para ver suas estatísticas.

Associações Polimórficas

“Os negócios são o dinheiro dos outros”

– Alexandre Dumas

Nesse capítulo você verá como criar uma relação *muitos-para-muitos* para mais de um tipo de modelo.

12.1 NOSSO PROBLEMA

O cliente pede para a equipe de desenvolvedores criar uma funcionalidade que permita aos visitantes deixar comentários sobre suas visitas aos restaurantes.

Para complicar a vida do programador, o cliente pede para permitir comentários também em qualificações, permitindo aos usuários do site justificar a nota que deram.

Esse problema poderia ser resolvido de diversas maneiras sendo que trabalharemos em cima de um modelo para representar um comentário, relacionado com restaurantes e qualificações, aproveitando para mostrar como realizar tal tipo de relacionamento.

Seria simples se pudéssemos criar mais uma tabela com o comentário em si e o id da entidade relacionada. O problema surge no momento de diferenciar um comentário sobre qualificação de um sobre restaurante.

Para diferenciar os comentários de restaurantes e qualificações, podemos usar um atributo de nome “tipo”.

Em Ruby podemos criar apelidos para um ou mais modelos, algo similar a diversas classes implementarem determinada interface (sem métodos) em java. Podemos chamar nossos modelos Restaurante e Qualificacao como comentáveis, por exemplo.

Um exemplo dessa estrutura em Java é o caso de `Serializable` - interface que não obriga a implementação de nenhum método mas serve para marcar classes como serializáveis, sendo que diversas classes da api padrão do Java implementam a primeira.

No caso do Ruby, começamos criando um modelo chamado `Comentario`.

12.2 ALTERANDO O BANCO DE DADOS

O conteúdo do script de migração criará as colunas “comentário”, “id de quem tem o comentário”, e o “tipo”.

Nos campos `id` e `tipo`, colocamos o nome da coluna com o apelido seguido de `_id` e `_type`, respectivamente, notificando o Ruby que ele deve buscar tais dados daquilo que é “comentável”.

Note que no português a palavra “comentável” soa estranho e parece esquisito trabalhar com ela, mas para seguir o padrão definido no inglês em diversas linguagens, tal apelido indica o que os modelos são capazes de fazer e, no caso, eles são “comentáveis”.

O script deve então criar três colunas, sem nada de novo comparado com o que vimos até agora:

```
rails generate scaffold comentario conteudo:text \
  comentavel_id:integer comentavel_type
```

Caso seja necessário, podemos ainda adicionar índices físicos nas colunas do relacionamento, deixando a migration criada como a seguir:

```
class CreateComentarios < ActiveRecord::Migration
  def change
    create_table :comentarios do |t|
      t.text :conteudo
      t.integer :comentavel_id
      t.string :comentavel_type

      t.timestamps
    end

    add_index :comentarios, :comentavel_type
    add_index :comentarios, :comentavel_id
  end
end
```

Para trabalhar nos modelos, precisamos antes gerar a nova tabela necessária:

```
$ rake db:migrate
```

O modelo `Comentario` (`app/models/comentario.rb`) deve poder ser associado a qualquer objeto do grupo de modelos comentáveis. Qualquer objeto poderá fazer o papel de `comentavel`, por isso dizemos que a associação é polimórfica:

```
class Comentario < ActiveRecord::Base
  attr_accessible :conteudo, :comentavel_id, :comentavel_type
  belongs_to :comentavel, polymorphic: true
end
```

A instrução `:polymorphic` indica a não existência de um modelo com o nome `:comentavel`.

Falta agora comentar que uma qualificação e um restaurante terão diversos comentários, fazendo o papel de algo comentavel. Para isso usaremos o relacionamento `has_many`:

```
class Qualificacao < ActiveRecord::Base
  # ...

  belongs_to :cliente
  belongs_to :restaurante

  has_many :comentarios, as: :comentavel

  # ...
end
```

E o Restaurante:

```
class Restaurante < ActiveRecord::Base
  # ...

  has_many :qualificacoes
  has_many :comentarios, as: :comentavel

  # ...
end
```

A tradução do texto pode ser quase literal: o modelo **TEM MUITOS** comentários **COMO** comentável.

12.3 EXERCÍCIOS

1) Vamos criar o modelo do nosso comentário e fazer a migração para o banco de dados:

a) Vá ao Terminal

b) Digite:

```
$ rails generate scaffold comentario conteudo:text
comentavel_id:integer comentavel_type
```

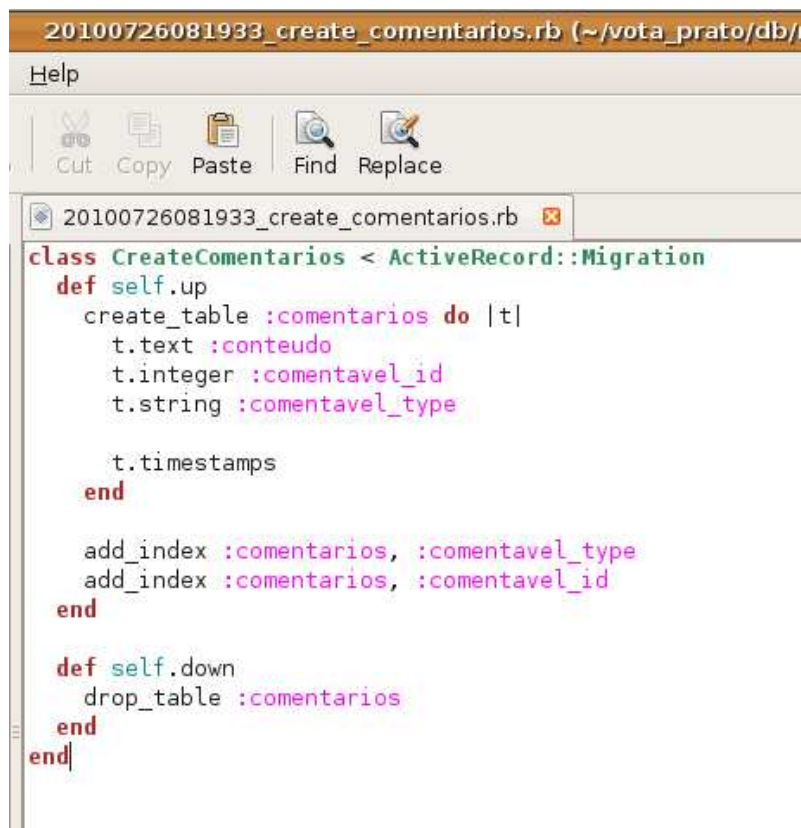


```
rr71@caelum131-03: ~/vota_prato
rr71@caelum131-03:~/vota_prato$ rails generate scaffold comentario conteudo:text comentavel_id:integer comentavel_type:string
  invoke  active_record
  create  db/migrate/20100726081933_create_comentarios.rb
  create  app/models/comentario.rb
  invoke  test_unit
  create  test/unit/comentario_test.rb
  create  test/fixtures/comentarios.yml
  route   resources :comentarios
  invoke  scaffold_controller
  create  app/controllers/comentarios_controller.rb
  invoke  erb
  create  app/views/comentarios
  create  app/views/comentarios/index.html.erb
  create  app/views/comentarios/edit.html.erb
  create  app/views/comentarios/show.html.erb
  create  app/views/comentarios/new.html.erb
  create  app/views/comentarios/_form.html.erb
  invoke  test_unit
  create  test/functional/comentarios_controller_test.rb
  invoke  helper
  create  app/helpers/comentarios_helper.rb
  invoke  test_unit
  create  test/unit/helpers/comentarios_helper_test.rb
  invoke  stylesheets
  identical public/stylesheets/scaffold.css
rr71@caelum131-03:~/vota_prato$
```

c) Vamos inserir alguns índices físicos. Abra o arquivo **db/migrate/<timestamp>_create_comentarios.rb**

d) Insira as seguintes linhas:

```
add_index :comentarios, :comentavel_type
add_index :comentarios, :comentavel_id
```



```
class CreateComentarios < ActiveRecord::Migration
  def self.up
    create_table :comentarios do |t|
      t.text :conteudo
      t.integer :comentavel_id
      t.string :comentavel_type

      t.timestamps
    end

    add_index :comentarios, :comentavel_type
    add_index :comentarios, :comentavel_id
  end

  def self.down
    drop_table :comentarios
  end
end
```

e) Volte ao Terminal e rode as migrações com o comando:

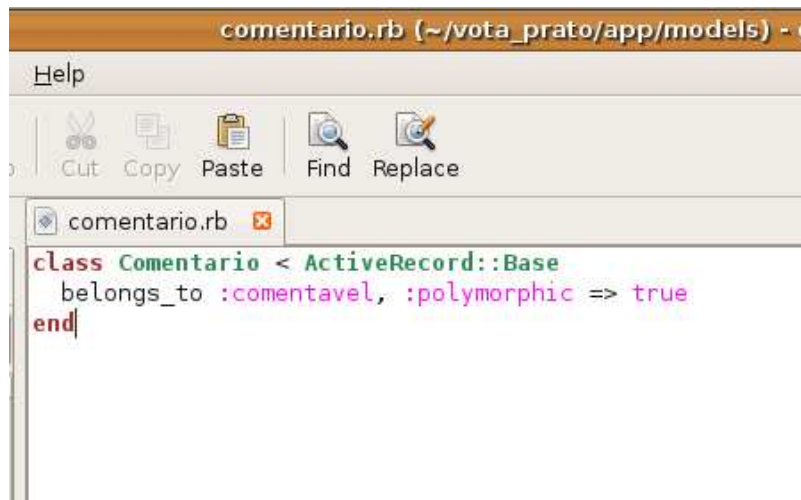
```
$ rake db:migrate
```

2) Vamos modificar nossos modelos:

a) Abra o arquivo **app/models/comentario.rb**

b) Adicione a seguinte linha:

```
belongs_to :comentavel, polymorphic: true
```



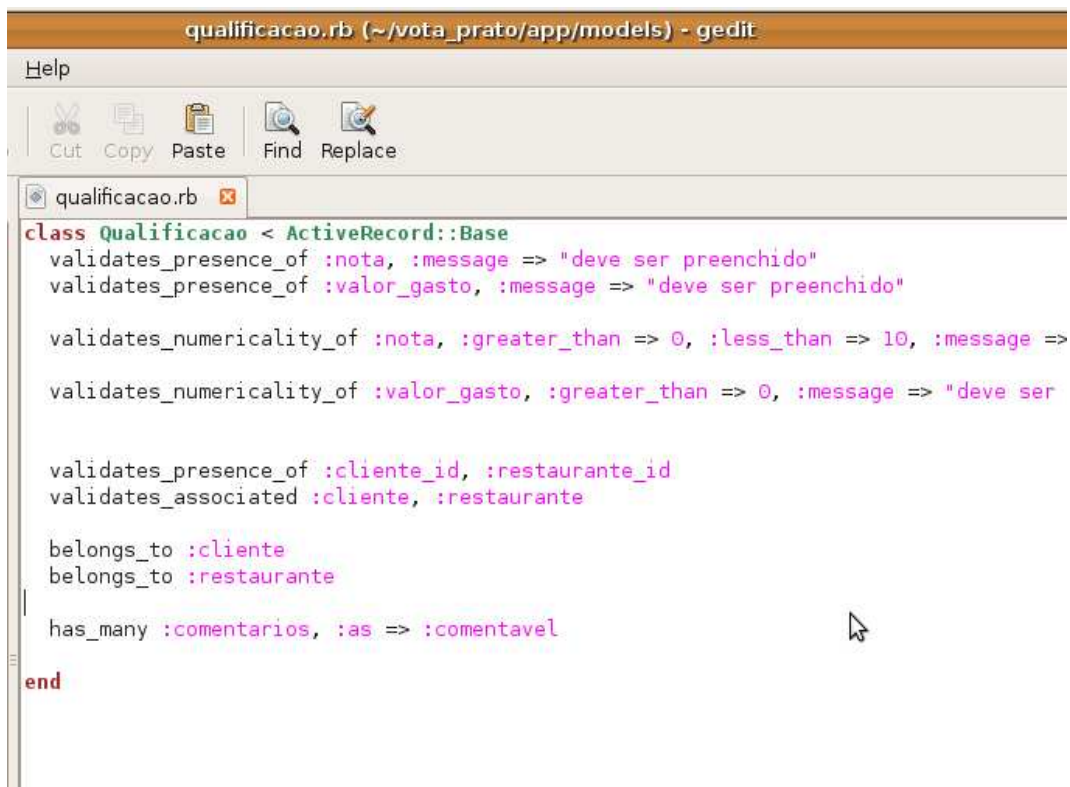
The screenshot shows a text editor window titled "comentario.rb (~/.vota_prato/app/models) - gedit". The editor contains the following Ruby code:

```
class Comentario < ActiveRecord::Base
  belongs_to :comentavel, :polymorphic => true
end
```

c) Abra o arquivo **app/models/qualificacao.rb**

d) Adicione a seguinte linha:

```
has_many :comentarios, as: :comentavel
```



The screenshot shows a text editor window titled "qualificacao.rb (~/.vota_prato/app/models) - gedit". The editor contains the following Ruby code:

```
class Qualificacao < ActiveRecord::Base
  validates_presence_of :nota, :message => "deve ser preenchido"
  validates_presence_of :valor_gasto, :message => "deve ser preenchido"

  validates_numericality_of :nota, :greater_than => 0, :less_than => 10, :message =>
  validates_numericality_of :valor_gasto, :greater_than => 0, :message => "deve ser

  validates_presence_of :cliente_id, :restaurante_id
  validates_associated :cliente, :restaurante

  belongs_to :cliente
  belongs_to :restaurante

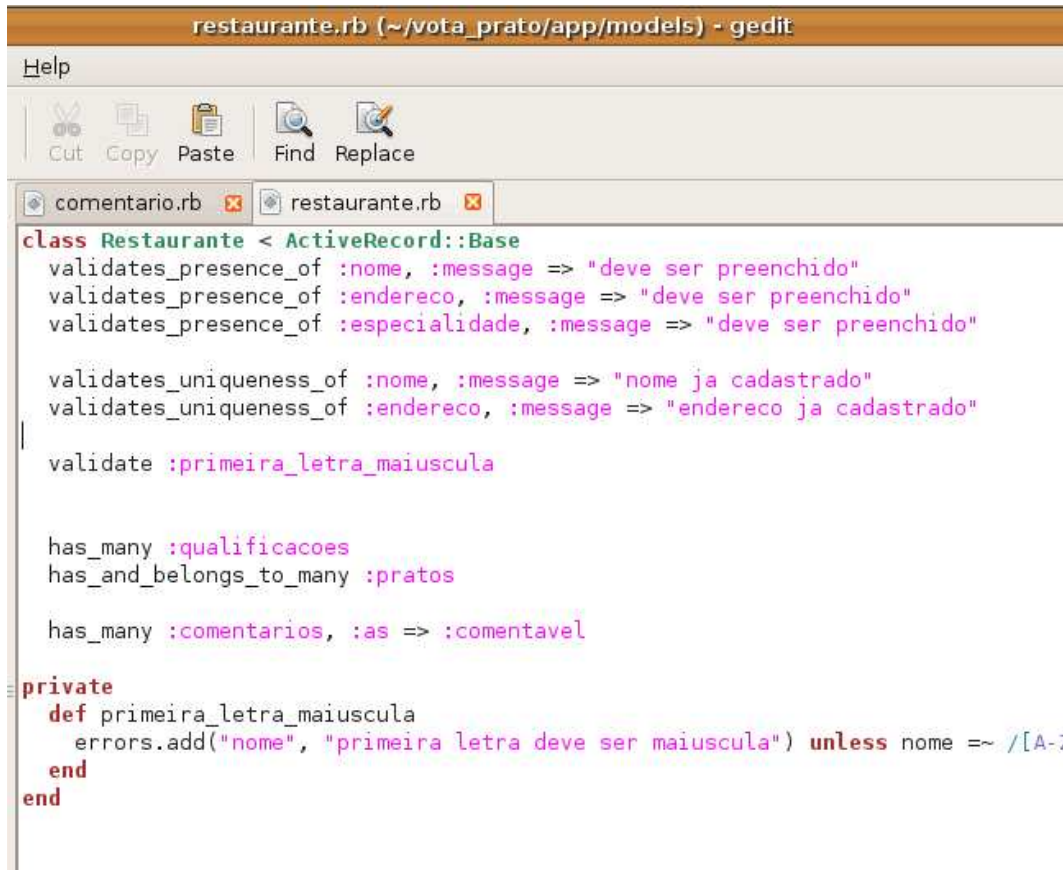
  has_many :comentarios, :as => :comentavel

end
```

e) Abra o arquivo **app/models/restaurante.rb**

f) Adicione a seguinte linha:

```
has_many :comentarios, as: :comentavel
```



```
restaurante.rb (~/vota_prato/app/models) - gedit

Help

Cut Copy Paste Find Replace

comentario.rb x restaurante.rb x

class Restaurante < ActiveRecord::Base
  validates_presence_of :nome, :message => "deve ser preenchido"
  validates_presence_of :endereco, :message => "deve ser preenchido"
  validates_presence_of :especialidade, :message => "deve ser preenchido"

  validates_uniqueness_of :nome, :message => "nome ja cadastrado"
  validates_uniqueness_of :endereco, :message => "endereco ja cadastrado"

  |
  validate :primeira_letra_maiuscula

  has_many :qualificacoes
  has_and_belongs_to_many :pratos

  has_many :comentarios, :as => :comentavel

private
  def primeira_letra_maiuscula
    errors.add("nome", "primeira letra deve ser maiuscula") unless nome =~ /[A-Z]
  end
end
```

3) Para o próximo capítulo, vamos precisar que o nosso sistema já inclua alguns comentários. Para criá-los, você pode usar o rails console ou ir em <http://localhost:3000/comentarios> e adicionar um comentário qualquer para o “Comentavel” 1, por exemplo, e o tipo “Restaurante”. Isso criará um comentário para o restaurante de ID 1.

Ajax com Rails

“O Cliente tem sempre razão”

– Selfridge, H.

Nesse capítulo, você verá como trabalhar com AJAX de maneira não obstrusiva no Rails.

13.1 ADICIONANDO COMENTÁRIOS NAS VIEWS

Até agora trabalhamos com arquivos `html.erb`, que são páginas html com scripts em ruby, mas nada de javascript.

Como o Ruby on Rails é um framework voltado para o desenvolvimento web, é natural que a questão do javascript seja levantada. O Rails, antes da versão 3, tentava facilitar o desenvolvimento de código javascript com recursos como o RJS Templates, que produzia código javascript a partir de código Ruby. Contudo, o código gerado era acoplado ao Prototype, o que dificultava o uso de outras bibliotecas populares como o jQuery. No Rails 3 optou-se por uma nova forma, não obstrutiva, de se trabalhar com javascript, permitindo que os desenvolvedores tenham controle absoluto do código criado, e podendo, inclusive, escolher a biblioteca que será usada (Prototype, jQuery, Moo-tools ou qual quer outra, desde que um driver exista para essa biblioteca).

Outra mudança interessante que mostra que o *Rails* é um framework em constante evolução foi a adoção do framework *jQuery* como padrão a partir da versão 3.1.

13.2 CRIANDO AS CHAMADAS AJAX

Falta ainda escrever a funcionalidade de adicionar comentários aos nossos restaurantes e qualificações. Podemos, aqui, utilizar AJAX para uma experiência mais marcante no uso do site pelos usuários.

Nosso primeiro passo será possibilitar a inclusão da lista de comentários nas páginas de qualquer modelo que seja comentável. Para não repetir este código em todas as páginas que aceitem comentários, podemos isolá-lo em um helper:

```
1 def comentarios(comentavel)
2   comentarios = "<div id='comentarios'>"
3   comentarios << "<h3>Comentarios</h3>"
4   comentavel.comentarios.each do |comentario|
5     comentarios << render partial: "comentarios/comentario",
6                        locals: {comentario: comentario}
7   end
8   comentarios << "</div>"
9   raw comentarios
10 end
```

Podemos simplificar o código acima, utilizando a opção `:collection`. Dessa maneira, o partial é renderizado uma vez para cada elemento que eu tenha no meu array:

```
1 def comentarios(comentavel)
2   comentarios = "<div id='comentarios'>"
3   comentarios << "<h3>Comentarios</h3>"
4   comentarios << render (partial: "comentarios/comentario",
5                        collection: comentavel.comentarios)
6   comentarios << "</div>"
7   raw comentarios
8 end
```

Perceba que nosso objetivo aqui é *renderizar* uma coleção. Estamos usando algumas convenções do *Rails* que irão permitir simplificar ainda mais o código acima. Além de criar uma variável com o o mesmo nome do partial que vamos *renderizar*, no nosso exemplo essa variável será chamada *comentario*, o *Rails* também será capaz de decidir **qual** é o arquivo partial que deve ser utilizado para exibir os itens de uma coleção.

Como temos uma coleção de **comentários** para exibir, basta invocar o método *render* passando a coleção como parâmetro, e o *Rails* cuidará do resto. Assim será possível omitir tanto o nome do *partial* como o *hash* que passamos como parâmetro anteriormente. Vamos fazer essa simplificação em nosso código e além disso garantir que a coleção seja *renderizada* apenas se não estiver vazia, veja:

```
1 def comentarios(comentavel)
2   comentarios = "<div id='comentarios'>"
3   comentarios << "<h3>Comentarios</h3>"
4   comentarios << render(comentavel.comentarios) unless comentavel.comentarios.empty?
5   comentarios << "</div>"
6   raw comentarios
7 end
```

Agora, vamos criar o partial responsável pela renderização de cada um dos comentários. Repare que usamos o método `link_to` passando o parâmetro `:remote => true`, que faz uma chamada a uma url utilizando para isso uma requisição assíncrona (AJAX):

```
1 <!-- /app/views/comentarios/_comentario.html.erb -->
2 <p id="comentario_<%= comentario.id %>">
3   <%= comentario.conteudo %> -
4   <%= link_to '(remover)', comentario, :method => :delete,
5                                     :remote => true,
6                                     :id => "remove_comentario_#{comentario.id}" %>
7 </p>
```

Além disso, precisamos invocar nosso helper em **app/views/restaurantes/show.html.erb** e **app/views/qualificacoes/show.html.erb**:

```
<%= comentarios @qualificacao %>
```

Tente clicar no link e verá que nada acontece, porém ao recarregar a página, o comentário foi removido!

A ação `ComentariosController#destroy` está sendo chamada de forma assíncrona (*Ajax*), porém a página não foi atualizada. Por isso precisamos tratar a resposta do servidor para remover o item da página.

Para isso, primeiramente vamos alterar o *app/views/layouts/application.html.erb* para nos possibilitar escrevermos javascript no mesmo arquivo que exibe os comentários.

```
1 <!-- /app/views/layouts/application.html.erb -->
2 <!DOCTYPE html>
3 <html>
4 <head>
5   <title>VotaPrato</title>
6   <%= stylesheet_link_tag "application", :media => "all" %>
7   <%= javascript_include_tag "application" %>
8   <%= csrf_meta_tag %>
9 </head>
10 <body>
11   <%= yield %>
12
13   <script type="text/javascript">
14     <%= yield :js%>
15   </script>
16 </body>
17 </html>
```

USANDO O YIELD

Usando o *yield* passando um símbolo como parâmetro podemos usar o método *content_for* nas nossas views e fazer um tipo de include inteligente nos nossos arquivos.

Agora vamos alterar novamente o arquivo *app/views/comentarios/_comentario.html.erb* para colocar o javascript que será renderizado no lugar do `'yield :js'`.

```
<p id="comentario_<%= comentario.id %>">
  <%= comentario.conteudo %>
  <%= link_to 'remover', comentario, :method => :delete,
    :remote => true,
    :id => "remove_comentario_#{comentario.id}"%>
</p>

<%= content_for :js do%>
  $('#remove_comentario_<%= comentario.id %>').bind('ajax:success',
    function(xhr, result){
      $('#comentario_<%= comentario.id %>').remove();
    }
  );
<%end%>
```

Dessa forma, teremos um html gerado parecido com esse:

```
<div id='comentarios'><h3><Comentarios></h3>
<p id="comentario_1">
  comentario 1
  <a href="/comentarios/1" data-method="delete" data-remote="true"
    id="remove_comentario_1" rel="nofollow">remover</a>
</p>

<p id="comentario_2">
  Comentario 2
  <a href="/comentarios/2" data-method="delete" data-remote="true"
    id="remove_comentario_2" rel="nofollow">remover</a>
</p>

<!-- Continuação do HTML -->

<script type="text/javascript">
$('#remove_comentario_1').bind('ajax:success',
  function(xhr, result){
    $('#comentario_1').remove();
```



```
    }  
  );  
  
  $('#remove_comentario_2').bind('ajax:success',  
    function(xhr, result){  
      $('#comentario_2').remove();  
    }  
  );  
</script>
```

Basta fazermos com que nossa action destroy não renderize nada , adicionando mais um formato ao bloco respond_to:

```
1 def destroy  
2   @comentario = Comentario.find(params[:id])  
3   @comentario.destroy  
4  
5   respond_to do |format|  
6     format.xml { head :ok }  
7     format.js { head :ok }  
8   end  
9 end
```

13.3 EXERCÍCIOS

1) Vamos adicionar os comentários nas views.

a) Abra o arquivo **app/helpers/application_helper.rb**

b) Insira as seguintes linhas:

```
1 def comentarios(comentavel)  
2   comentarios = "<div id='comentarios'>"  
3   comentarios << "<h3>Comentarios</h3>"  
4   comentarios << render(comentavel.comentarios) unless comentario.comentarios.empty?  
5   comentarios << "</div>"  
6   raw comentarios  
7 end
```

c) Crie o arquivo **app/views/comentarios/_comentario.html.erb** com o seguinte conteúdo:

```
1 <p id="comentario_<%= comentario.id %>">  
2   <%= comentario.conteudo %> -  
3   <%= link_to '(remover)', comentario, :method => :delete,  
4                                     :remote => true,
```



```
5                                     :id => "remove_comentario_#{comentario.id}" %>
6 </p>
7
8 <%= content_for :js do %>
9   $('#remove_comentario_<%=comentario.id%>').bind('ajax:success',
10     function(xhr, result){
11       $('#comentario_<%=comentario.id%>').remove();
12     }
13   );
14 <% end %>
```

d) Altere as seguintes linhas da action **destroy** do controller **comentarios_controller.rb**

```
1 def destroy
2   @comentario = Comentario.find(params[:id])
3   @comentario.destroy
4
5   respond_to do |format|
6     format.xml { head :ok }
7     format.js { head :ok }
8   end
9 end
```

e) Adicione a seguinte linha no final do arquivo **app/views/restaurantes/show.html.erb**

```
<%= comentarios @restaurante %>
```

f) Adicione a seguinte linha no final do arquivo **app/views/qualificacoes/show.html.erb**

```
<%= comentarios @qualificacao %>
```

g) Teste em **http://localhost:3000/restaurantes**, escolhendo um restaurante no qual você já tenha inserido comentários.

13.4 ADICIONANDO COMENTÁRIOS

Para possibilitar a adição de comentários podemos adicionar um novo link logo após a lista de comentários, criando um novo helper method no **ApplicationController**:

```
def novo_comentario(comentavel)
  form_novo_comentario = render partial: "comentarios/novo_comentario",
                                     locals: {comentavel: comentavel}

  raw form_novo_comentario
end
```

É necessário agora criar o novo partial. Lembre que para criar um comentário, é preciso saber o que está sendo comentado (comentável).

```
<!-- /app/views/comentarios/_novo_comentario.html.erb -->
<div id='novo_comentario'>
  <a href="javascript:$('#novo_comentario').children().toggle();">
    Novo comentario
  </a>
  <div id="form_comentario" style="display: none;">
    <%= form_for Comentario.new, :remote => true do |f| %>
      <%= hidden_field :comentario, :comentavel_id,
        :value => comentavel.id %>
      <%= hidden_field :comentario, :comentavel_type,
        :value => comentavel.class %>

      <div class="field">
        <%= f.label 'conteudo' %><br />
        <%= f.text_area 'conteudo' %>
      </div>

      <div class="actions">
        <%= f.submit %>
        <a href="javascript:$('#novo_comentario').children().toggle();">
          Cancelar
        </a>
      </div>
    <% end %>
  </div>
</div>

<%content_for :js do%>
  $('#form_comentario').bind('ajax:complete', function(xhr, result){
    $('#comentarios').html(result.responseText);
    $('#form_comentario textarea').val("")
    $('#form_comentario').hide();
    $('#novo_comentario > a').show();
  });
<%end%>
```

13.5 EXERCÍCIOS

- 1) Vamos permitir a adição de novos comentários:
 - a) Abra o arquivo **app/helpers/application_helper.rb**
 - b) Vamos criar um novo método nesse *Helper*. A função desse método será renderizar um *partial* contendo o link para criação de novos comentários. Para isso, adicione o método `novo_comentario` na sua classe `ApplicationHelper`:

```
def novo_comentario(comentavel)
  form_novo_comentario = render partial: "comentarios/novo_comentario",
                                   locals: {comentavel: comentavel}

  raw form_novo_comentario
end
```

c) Crie o novo partial **app/views/comentarios/_novo_comentario.html.erb** com o seguinte conteúdo:

```
<div id='novo_comentario'>
  <a href="#" onclick="$('#novo_comentario').children().toggle();">
    Novo comentario
  </a>
  <div id="form_comentario" style="display: none;">
    <%= form_for Comentario.new, :remote => true do |f| %>
      <%= hidden_field :comentario, :comentavel_id,
                      :value => comentavel.id %>
      <%= hidden_field :comentario, :comentavel_type,
                      value => comentavel.class %>

      <div class="field">
        <%= f.label 'conteudo' %><br />
        <%= f.text_area 'conteudo' %>
      </div>

      <div class="actions">
        <%= f.submit %>
        <a href="#"
          onclick="$('#novo_comentario').children().toggle();"

          Cancelar

        </a>
      </div>
    <% end %>
  </div>
</div>

<%content_for :js do%>
  $('#form_comentario').bind('ajax:complete', function(xhr, result){
    $('#comentarios').html(result.responseText);
    $('#form_comentario textarea').val("")
    $('#form_comentario').hide();
    $('#novo_comentario > a').show();
  });
<%end%>
```

- d) O link deverá ser colocado após a verificação da existência de comentários para mostrar a lista dos comentários, para isso, abra o arquivo `app/views/restaurantes/show.html.erb` e abaixo do `unless` que faz a verificação adicione a chamada para o seu novo *Helper*:

```
1 <% unless @restaurante.comentarios.empty?%>
2   <%= comentarios @restaurante %>
3 <% end %>
4
5 <%= novo_comentario(@restaurante) %>
```

- e) Faça o mesmo para o arquivo `app/views/qualificacoes/show.html.erb`.

- f) Para finalizar o processo temos que alterar o nosso controller para exibir a lista de comentários do nosso comentável. Altere o método `create` no **`app/controllers/comentarios_controller.rb`**

```
if @comentario.save
  format.xml { render :xml => @comentario, :status => :created,
                                     :location => @comentario }

  format.js
else
```

- g) E agora crie o arquivo **`views/comentarios/create.js.erb`**

```
1 <%= render(:partial => "comentarios/comentario",
2           :collection => @comentario.comentavel.comentarios) %>
```

- h) Para que nosso partial exiba corretamente os javascripts precisamos criar um layout para colocar o bloco de `yield` dos javascripts.

- i) Crie o arquivo **`app/views/layouts/application.js.erb`** com o seguinte conteúdo

```
1 <%= yield %>
2
3 <script type="text/javascript">
4   <%=yield :js%>
5 </script>
```

- j) Agora já é possível cadastrar um comentário direto do show dos comentáveis

Algumas Gems Importantes

“A minoria pode ter razão, a maioria está sempre errada”
– Mikhail Aleksandrovitch Bakunin

14.1 ENGINES

A partir do *Rails 3.1* foi criado um conceito para facilitar a extensão de uma aplicação chamado *Rails Engine*. Uma *engine* é uma espécie de mini aplicação *Rails* que permite isolar funcionalidades que podem ser reaproveitadas em qualquer aplicação *Rails* “comum” de uma forma estruturada e **plugável**.

Um bom exemplo de *Rails engine* é a *gem kaminari* que facilita muito a criação de uma lista paginada para exibição de registros. Basta adicionar a dependência do *Gemfile*, rodar o comando `bundle install` e pronto! Toda a funcionalidade de paginação será fornecida sem a necessidade de nenhum tipo de configuração.

Primeiro vamos editar o arquivo *Gemfile* para adicionar a dependência:

```
gem "kaminari"
```

Agora para que os dados sejam paginados a partir de uma busca no banco de dados, basta adicionar a invocação do método `page` do *kaminari*. Podemos usar essa funcionalidade na listagem de restaurantes como no código a seguir:

```
@restaurantes = Restaurante.all.page params['page']
```

O método `page` funciona como um *finder* normal. Suporta todas as opções previamente vistas, como `:conditions`, `:order` e `:include`. Note também que passamos um parâmetro para o método `page` que é a **página** a partir da qual queremos listar os registros. Esse parâmetro será gerado pelo próprio *kaminari* através de um *helper method* que invocaremos na *view*, mais adiante você verá como isso funciona.

O número de itens por página é padronizado em 25, mas você pode customizar de duas formas. Através do método `per_page` nas classes `ActiveRecord::Base`:

```
class Restaurante < ActiveRecord::Base
  paginates_per 10
  # ...
end
```

Ou invocando o método `per` e passando como parâmetro o número máximo de registros por página. O método `per` está disponível no objeto criado pelo `kaminari` portanto só pode ser invocado após uma chamada ao método `page`. O código seguinte exemplifica o seu uso:

```
@restaurantes = Restaurante.all.page(params['page']).per(10)
```

E como dito, veja como usar o *helper method* do `kaminari` para gerar os links de paginação já com o parâmetro `page` que usamos durante os exemplos de paginação:

```
1 # index.html.erb
2 <% @restaurantes.each do |restaurante| %>
3   <li><%= restaurante.nome %></li>
4 <% end %>
5 <%= paginate @restaurantes %>
```

Uma coisa bem interessante sobre o `kaminari` é sua facilidade de customização. Você pode alterar o texto padrão usado por ele para geração de links, assim como a aparência desses mesmos links. Veja o repositório oficial para maiores detalhes: <https://github.com/amatsuda/kaminari>.

14.2 EXERCÍCIOS - TÍTULO

- 1) Adicione o `kaminari` ao seu projeto, no arquivo `Gemfile`

```
gem 'kaminari'
```

Execute o seguinte comando no terminal para instalar a gem:

```
$ bundle install
```

- 2) Abra o arquivo `app/controllers/restaurantes_controller.rb`. Na action `index`, troque a linha:

```
@restaurantes = Restaurante.order("nome")
```

por

```
@restaurantes = Restaurante.order("nome").page(params['page']).per(3)
```

- 3) Abra o arquivo **app/views/restaurantes/index.html.erb**. Adicione a linha abaixo após o fechamento da *tag table* (`</table>`):

```
<%= paginate @restaurantes %>
```

- 4) Abra a listagem de restaurantes e verifique a paginação.

14.3 FILE UPLOADS: PAPERCLIP

Podemos fazer upload de arquivos sem a necessidade de plugins adicionais, utilizando algo como o código abaixo:

```
File.open("public/"+path, "nome") do |f|  
  f.write(params[:upload]['picture_path'].read)  
end
```

O código acima recebe o binário do arquivo e faz o upload para a pasta public. Porém, ao fazer um upload seria interessante fazer coisas como redimensionar a imagem, gerar thumbs, associar com models ActiveRecord, etc.

Um dos primeiros plugins rails voltados para isso foi o *attachment_fu*. Hoje em dia o plugin mais indicado é o **Paperclip**. O Paperclip tem como finalidade ser um plugin de fácil uso com o modelos ActiveRecord. As configurações são simples e é possível validar tamanho do arquivo ou tornar sua presença obrigatória. O paperclip tem como pré-requisito o *ImageMagick*,

14.4 EXERCÍCIOS

- 1) Para instalar o paperclip, abra o arquivo Gemfile e adicione a gem:

```
gem 'paperclip'
```

E no terminal, rode :

```
bundle install
```

- 2) Adicione o **has_attached_file** do paperclip na classe Restaurante. Vamos configurar mais uma opção que daremos o nome de *styles*. Toda vez que a view chamar a foto do restaurante com essa opção, o Rail buscará pelo thumb.

```
class Restaurante < ActiveRecord::Base  
  has_attached_file :foto, :styles =>  
    { :medium => "300x300>", :thumb => "100x100>" }  
end
```

ECONOMIZANDO ESPAÇO EM DISCO

É boa prática ao configurar os tamanhos das imagens que armazenaremos, sobrescrever o tamanho do estilo “original”, dessa maneira evitamos que o Paperclip salve imagens muito maiores do que utilizaremos em nossa aplicação.

```
has_attached_file :foto, :styles => { :medium => "300x300>",  
                                       :thumb => "100x100>",  
                                       :original => "800x600>" }
```

No exemplo acima, a imagem “original” será salva com tamanho máximo de 800x600 pixels, isso evita que armazenemos desnecessariamente em nossa aplicação, por exemplo, imagens de 4000x3000 pixels, tamanho resultante de uma foto tirada em uma máquina fotográfica digital de 12MP.

- 3) Precisamos de uma migration que defina novas colunas para a foto do restaurante na tabela de restaurantes. O paperclip define 4 colunas básicas para nome, conteúdo, tamanho do arquivo e data de update.

Crie a migration **AddFotoColumnsToRestaurante** abaixo na pasta db/migrate:

```
class AddAttachmentFotoToRestaurante < ActiveRecord::Migration  
  def self.up  
    add_column :restaurantes, :foto_file_name, :string  
    add_column :restaurantes, :foto_content_type, :string  
    add_column :restaurantes, :foto_file_size, :integer  
    add_column :restaurantes, :foto_updated_at, :datetime  
  end  
  
  def self.down  
    remove_column :restaurantes, :foto_file_name  
    remove_column :restaurantes, :foto_content_type  
    remove_column :restaurantes, :foto_file_size  
    remove_column :restaurantes, :foto_updated_at  
  end  
end
```

GERADORES DO PAPERCLIP

Você também pode usar o generator do próprio Paperclip: **rails generate paperclip Restaurante foto**

Rode a migration no terminal com:


```
rake db:migrate
```

- 4) Abra a view **app/views/restaurantes/_form.html.erb** e altere o formulário. Seu form deve ficar como o abaixo:

```
<%= form_for :restaurantes, :url => {:action=>'create'},  
      :html => {:multipart=>true} do |f| %>  
  
  <!--outros campos-->  
  <%= f.file_field :foto %>  
  
<% end %>
```

- 5) Abra a view **app/views/restaurantes/show.html.erb** e adicione:

```
<p>  
  <b>Foto:</b>  
  <%= image_tag @restaurante.foto.url(:thumb) %>  
</p>
```

Repare que aqui chamamos o thumb, que foi configurado como um dos styles do model. Suba o server e insira um novo restaurante com foto.

14.5 HPRICOT

HPricot é uma biblioteca poderosa para manipulação de xhtml. Bastante útil para capturar conteúdo da internet que não tenha sido criado pensando em integração e não oferece formatos mais adequados para serem consumidos por outros sistemas, como json ou xml.

```
gem install hpricot
```

open-uri é uma biblioteca que usaremos para fazer requisições http:

```
doc = Hpricot(open('http://twitter.com/guilhermecaelum'))
```

Analisando o html gerado pelo twitter, vemos que as mensagens estão sempre dentro de elementos com a classe "hentry". Além disso, dentro de cada hentry, a única parte que nos interessa é o conteúdo dos subitens de classe "entry-content".

Podemos procurar estes itens com o Hpricot, usando seletores CSS. Expressões *XPath* também poderiam ser usadas:

```
doc / ".hentry .entry-content"
```

Para imprimir cada um dos itens de uma maneira mais interessante:

```
items = doc / ".hentry .entry-content"
items.each do |item|
  puts item.inner_text
end
```

14.6 EXERCÍCIOS - TESTANDO O HPRICOT

1) Vamos fazer um leitor de twitties de um determinado usuário

a) Crie um arquivo chamado "twitter_reader.rb"

b) Adicione às seguintes linhas:

```
require 'hpricot'
require 'open-uri'

doc = Hpricot(open('http://twitter.com/caelum'))
items = doc / ".stream-item"
items.each do |item|
  tweet = item / ".tweet .js-tweet-text"
  puts tweet.inner_text
  puts "_____\n\n"
end
```

c) Teste usando o comando `ruby twitter_reader.rb`

Testes

“Ninguém testa a profundidade de um rio com os dois pés.”

– Provérbio Africano

15.1 O PORQUÊ DOS TESTES?

Testes de Unidade são classes que o programador desenvolve para se certificar que partes do seu sistema estão funcionando corretamente.

Eles podem testar validações, processamento, domínios etc, mas lembre-se que um teste unitário deve testar *somente um pedaço de código* (de onde veio o nome *unitário*).

Criar esse tipo de testes é uma das partes mais importantes do desenvolvimento de uma aplicação pois possibilita a verificação real de todas as partes do programa automaticamente.

EXTREME PROGRAMMING (XP)

Extreme Programming é um conjunto de práticas de programação que visam a simplicidade, praticidade, qualidade e flexibilidade de seu sistema. Os testes de unidade fazem parte dessa metodologia de programação.

O Ruby já possui classes que nos auxiliam no desenvolvimento destes testes.

15.2 `Test::Unit`

`Test::Unit` é a biblioteca usada para escrever suas classes de teste.

Ao escrever testes em Ruby utilizando esse framework, você deve herdar a classe `TestCase` que provê a funcionalidade necessária para fazer os testes.

```
require 'test/unit'

class PessoaTest < Test::Unit::TestCase
  # ...
end
```

Ao herdar `Test::Unit::TestCase`, você ganha alguns métodos que irão auxiliar os seus testes:

- `assert(boolean, msg=nil)`
- `assert_equal(esperado, atual, msg=nil)`
- `assert_not_equal(esperado, atual, msg=nil)`
- `assert_in_delta(esperado, atual, delta, msg=nil)`
- `assert_instance_of(classe, objeto, msg=nil)`
- `assert_kind_of(classe, objeto, msg=nil)`
- `assert_match(regex, texto, msg=nil)`
- `assert_no_match(regex, texto, msg=nil)`
- `assert_nil(objeto, msg=nil)`
- `assert_not_nil(objeto, msg=nil)`
- `assert_respond_to(objeto, metodo, msg=nil)`
- `assert_same(esperado, atual, msg=nil)`
- `assert_not_same(esperado, atual, msg=nil)`

O método `assert` simples recebe como parâmetro qualquer expressão que devolva um valor booleano e todos os métodos `assert` recebem opcionalmente como último argumento uma mensagem que será exibida caso a asserção falhe.

Mais detalhes e outros métodos `assert` podem ser encontrados na documentação do módulo `Test::Unit::Assertions`, na documentação da biblioteca core da linguagem Ruby (<http://ruby-doc.org/core/>).

Os testes podem ser executados em linha de comando, bastando chamar `ruby o_que_eu_quero_testar.rb`. O resultado é um ”.

para os testes que passarem, "E" para erros em tempo de execução e "F" para testes que falharem.

Também é possível executar todos os testes com algumas tasks do rake:

```
# roda todos os testes de unidade, de integração e funcionais
rake test
```

```
# roda todos os testes da pasta test/unit
rake test:units
```

```
# roda todos os testes da pasta test/functional
rake test:functionals
```

```
# roda todos os testes da pasta test/integration
rake test:integration
```

```
# roda todos os testes de plugins, na pasta vendor/plugins
rake test:plugins
```

Existem ainda outras tarefas disponíveis para o rake. Sempre podemos consultá-las com `rake -T`, no diretório do projeto.

Podemos criar uma classe de teste que só possua um único “assert true”, no diretório `test/unit/`.

```
class MeuTeste < Test::Unit::TestCase
  def test_truth
    assert true
  end
end
```

Ao escrever testes de unidade em projetos Ruby On Rails, ao invés de herdar diretamente de `Test::Unit::TestCase`, temos a opção de herdar da classe fornecida pelo ActiveSupport do Rails:

```
require 'test_helper'

class RestauranteTest < ActiveSupport::TestCase
  def test_anything
    assert true
  end
end
```

Além disso, todos os testes em projetos Rails devem carregar o arquivo **test_helper.rb**, disponível em qualquer projeto gerado pelo Rails. As coisas comuns a todos os testes, como métodos utilitários e configurações, ficam neste arquivo.

A vantagem de herdar de `ActiveSupport::TestCase` ao invés da original é que o Rails provê diversas funcionalidades extras aos testes, como fixtures e métodos assert extras. Alguns dos asserts extras:

- `assert_difference`

- `assert_no_difference`
- `assert_valid(record)` - disponível em testes de unidade
- `assert_redirected_to(path)` - para testes de controladores
- `assert_template(esperado)` - também para controladores
- entre outros

15.3 EXERCÍCIOS - TESTE DO MODELO

- 1) Vamos testar nosso modelo restaurante. Para isso precisamos utilizar o banco de dados específico para testes.

```
rake db:create:all
rake db:migrate RAILS_ENV=test
```

- 2) O Rails já tem suporte inicial para testes automatizados. Nossa aplicação já possui arquivos importantes para nossos testes. Abra o arquivo `test/fixtures/restaurantes.yml`. Esse arquivo simula os dados de um restaurante. Crie os dois restaurantes abaixo. **Importante:** Cuidado com a identificação!

```
fasano:
  nome: Fasano
  endereco: Rua Vergueiro

fogo_de_chao:
  nome: Fogo de Chao
  endereco: Avenida dos Bandeirantes
```

- 3) O arquivo de teste do modelo está em `test/unit/restaurante_test.rb`.

```
require 'test_helper'

class RestauranteTest < ActiveSupport::TestCase
  fixtures :restaurantes

  def test_restaurante

    restaurante = Restaurante.new(
      :nome => restaurantes(:fasano).nome,
      :endereco => restaurantes(:fasano).endereco,
      :especialidade => restaurantes(:fasano).especialidade)

    msg = "restaurante não foi salvo. "
      + "errors: #{restaurante.errors.inspect}"
    assert restaurante.save, msg
```

```
    restaurante_fasano_copia = Restaurante.find(restaurante.id)

    assert_equal restaurante.nome, restaurante_fasano_copia.nome
  end
end
```

- 4) Para rodar o teste, vá na raiz do projeto pelo terminal e digite:

```
rake test
```

Verifique se tudo está certo:

```
Loaded suite test/unit/restaurante_test
Started
.
Finished in 0.044991 seconds.
```

```
1 tests, 4 assertions, 0 failures, 0 errors
```

15.4 EXERCÍCIOS - TESTE DO CONTROLLER

- 1) Para testar o controller de restaurantes vamos criar uma nova action chamada **busca**. Essa action direciona para o restaurante buscado caso encontre ou devolve uma mensagem de erro caso contrário. Abra o arquivo **app/controllers/restaurantes_controller.rb** e adicione a action **busca**:

```
def busca
  @restaurante = Restaurante.find_by_nome(params[:nome])
  if @restaurante
    redirect_to :action => 'show', :id => @restaurante.id
  else
    flash[:notice] = 'Restaurante não encontrado.'
    redirect_to :action => 'index'
  end
end
```

- 2) Abra o arquivo **test/functional/restaurantes_controller_test.rb**.

```
require 'test/test_helper'

class RestaurantesControllerTest < ActionController::TestCase
  fixtures :restaurantes

  def test_procura_restaurante
    get :busca, :nome => 'Fasano'
    assert_not_nil assigns(:restaurante)
    assert_equal restaurantes(:fasano).nome, assigns(:restaurante).nome
  end
end
```

```
    assert_redirected_to :action => 'show'
  end

  def test_procura_restaurante_nao_encontra
    get :busca, :nome => 'Botequin'
    assert_redirected_to :action => 'index'
    assert_equal 'Restaurante não encontrado.', flash[:notice]
  end
end
```

Verifique se tudo está certo;

```
Loaded suite test/functional/restaurantes_controller_test
Started
..
Finished in 0.206066 seconds.
```

2 tests, 4 assertions, 0 failures, 0 errors

3) Rode o teste no terminal com **rake test**.

15.5 RSpec

Muito mais do que uma nova forma de criar testes de unidade, RSpec fornece uma forma de criar especificações executáveis do seu código.

No TDD, descrevemos a funcionalidade esperada para nosso código através de testes de unidade. BDD (*Behavior Driven Development*) leva isso ao extremo e diz que nossos testes de unidade devem se tornar especificações executáveis do código. Ao escrever as especificações estaremos pensando no **comportamento esperado** para nosso código.

INTRODUÇÃO AO BDD

Uma ótima descrição sobre o termo pode ser encontrada no site do seu próprio criador: Dan North.

<http://dannorth.net/introducing-bdd/>

RSpec fornece uma DSL (*Domain Specific Language*) para criação de especificações executáveis de código. As especificações do RSpec funcionam como exemplos de uso do código, que validam se o código está mesmo fazendo o que deveria e funcionam como documentação.

<http://rspec.info>

Para instalar o rspec e usar em qualquer programa Ruby, basta instalar o gem:


```
gem install rspec
```

Para usar em aplicações Rails, precisamos instalar mais um gem que dá suporte ao rspec ao Rails. Além disso, precisamos usar o gerador que vem junto desta gem, para adicionar os arquivos necessários nos projetos que forem usar rspec:

```
cd projeto-rails
rails generate rspec:install
```

O último comando também adiciona algumas tasks do rake para executar as specs do projeto, além de criar a estrutura de pastas e adicionar os arquivos necessários.

```
rake spec      # executa todas as specs do projeto
rake -T spec   # para ver as tasks relacionadas ao rspec
```

O rspec-rails também pode ser instalado como plugin, porém hoje é altamente recomendado seu uso como gem. Mais detalhes podem ser encontrados na documentação oficial

<http://wiki.github.com/rspec/rspec-rails/>

RSpec é compatível com testes feitos para rodar com `Test::Unit`. Desta forma, é possível migrar de forma gradativa. Apesar disso, a sintaxe oferecida pelo RSpec se mostra bem mais interessante, já que segue as ideias do Behavior Driven Development e faz com que os testes se tornem especificações executáveis do código:

```
describe Restaurante, " com nome" do
  it "should have name"
    Restaurante.all.should_not be_empty
    Restaurante.first.should_not be_nil
    Restaurante.first.name.should == "Fasano"
  end
end
```

A classe de teste vira um **Example Group** (`describe`). Cada método de teste vira um **Example** (`it "should ..."`).

Além disso, os métodos `assert` tradicionais do `Test::Unit` viram uma chamada de `should`. O RSpec adiciona a **todos** os objetos os métodos `should` e `should_not`, que servem para validarmos alguma condição sobre o estado dos nossos objetos de uma forma mais legível e expressiva que com `assert`s.

Como argumento para o método `should`, devemos passar uma instância de `Matcher` que verifica uma condição particular. O RSpec é extremamente poderoso, pois nos permite escrever nossos próprios `Matchers`. Apesar disso, já vem com muitos prontos, que costumam ser mais do que suficientes:

- `be_<nome>` para métodos na forma `<nome>?`.

```
# testa: objeto.empty?  
objeto.should be_empty
```

```
# testa: not objeto.nil?  
objeto.should_not be_nil
```

```
# testa: objeto.kind_of(Restaurante)  
objeto.should be_kind_of(Restaurante)
```

Além de `be_<nome>`, também podemos usar `be_a_<nome>` ou `be_an_<nome>`, aumentando a legibilidade.

- `be_true`, `be_false`, `eql`, `equal`, `exist`, `include`:

```
objeto.should be_true  
objeto.should_not be_false
```

```
# testa: objeto.eql?(outro)  
objeto.should eql(outro)
```

```
# testa: objeto.equal?(outro)  
objeto.should equal(outro)
```

```
objeto.should exist # testa: objeto.exist?  
[4,5,3].should include(3) # testa: [4,5,3].include?(3)
```

- `have_<nome>` para métodos na forma `has_<nome>?`.

```
itens = { :um => 1, :dois => '2' }
```

```
# testa: itens.has_key?(:dois)  
itens.should have_key(:dois)
```

```
# testa: not itens.has_value?(/3/)  
itens.should_not have_value(/3/)
```

- `be_close`, inclui tolerância.

```
conta = 10.0 / 3.0  
conta.should be_close(3.3, 0.1) # == 3.3 ~0.1
```

- `have(num).<colecão>`, para testar a quantidade de itens em uma associação.

```
# testa categoria.produtos.size == 15  
categoria.should have(15).produtos
```

Um uso especial deste *matcher* é para objetos que já são coleções. Neste caso, podemos usar o nome que quisermos:

```
array = [1,2,3]

# testa array.size == 3
array.should have(3).items

# mesma coisa
array.should have(3).numbers
```

- `have_at_least(num).<colecao>`: mesma coisa que o anterior, porém usa `>=`.
- `have_at_most(num).<colecao>`: mesma coisa que o anterior, porém usa `<=`.
- `match`, para expressões regulares.

```
# verifica se começa com F
texto.should match(/^F/)
```

Este são os principais, mas ainda existem outros. Você pode encontrar a lista de Matchers completa na documentação do módulo `Spec::Matchers`:

EXEMPLOS PENDENTES

Um exemplo pode estar vazio. Desta forma, o RSpec o indicará como pendente:

```
describe Restaurante do
  it "should have endereço"
end
```

Isto facilita muito o ciclo do BDD, onde escrevemos o teste primeiro, antes do código de verdade. Podemos ir pensando nas funcionalidades que o sistema deve ter e deixá-las pendentes, antes mesmo de escrever o código. Em outras palavras, começamos especificando o que será escrito.

BEFORE E AFTER

Podemos definir algum comportamento comum para ser executado antes ou depois de cada um dos exemplos, como o `setup` e o `teardown` do `Test::Unit`:

```
describe Restaurante do
  before do
    @a_ser_testado = Restaurante.new
  end

  it "should ..."

  after do
    fecha_e_apaga_tudo
  end
end
```

```
end  
end
```

Estes métodos podem ainda receber um argumento dizendo se devem ser executados novamente para cada exemplo (:each) ou uma vez só para o grupo todo (:all):

```
describe Restaurante do  
  before(:all) do  
    @a_ser_testado = Restaurante.new  
  end  
  
  it "should ..."  
  
  after(:each) do  
    fecha_e_apaga_tudo  
  end  
end
```

15.6 CUCUMBER, O NOVO STORY RUNNER

RSpec funciona muito bem para especificações em níveis próximos ao código, como as especificações unitárias.

User Stories é uma ferramenta indicada para especificações em níveis mais altos, como funcionalidades de negócio, ou requisitos. Seu uso está sendo bastante difundido pela comunidade Rails. User Stories costumam ter o seguinte formato:

```
In order to <benefício>  
As a <interessado>  
I want to <funcionalidade>.
```

Cucumber é uma excelente biblioteca escrita em Ruby, que serve para tornar especificações como esta, na forma de *User Stories*, escritas em texto puro, executáveis. Cucumber permite a associação de código Ruby arbitrário, usualmente código de teste com RSpec, a cada um dos passos desta descrição da funcionalidade.

Para instalar tudo o que é necessário:

```
gem install cucumber capybara database_cleaner  
  
gem 'database_cleaner'  
gem 'cucumber-rails'  
gem 'cucumber'  
gem 'rspec-rails'
```

```
gem 'spork'  
gem 'launchy'
```

Para projetos rails, é possível usar o *generator* fornecido pelo Cucumber para adicionar os arquivos necessários ao projeto:

```
cd projetorails  
rails generate cucumber:install
```

As User Stories são chamadas de **Features** pelo Cucumber. São arquivos de texto puro com a extensão *.feature*. Arquivos com definição de features sempre contém uma descrição da funcionalidade (**Story**) e alguns exemplos (**Scenários**), na seguinte estrutura:

```
Feature: <nome da story>  
  In order to <beneficio>  
  As a <interessado>  
  I want to <funcionalidade>  
  
Scenario: <nome do exemplo>  
  Given <pré condições>  
  And   <mais pré condições>  
  When  <ação>  
  And   <mais ação>  
  Then  <resultado>  
  And   <mais resultado>  
  
Scenario: <outro exemplo>  
  Given ...  
  When ...  
  Then ...
```

Antigamente, o RSpec incluía sua própria implementação de Story Runner, que hoje está sendo substituída pelo Cucumber. O RSpec Story Runner original utilizava um outro formato para features, mais tradicional, que não dá prioridade ao *Return Of Investment*. O benefício da funcionalidade fica em segundo plano, no final da descrição:

```
Story: transfer from savings to checking account  
  As a savings account holder  
  I want to transfer money from my savings account to my checking account  
  So that I can get cash easily from an ATM  
  
Scenario: ...
```

O importante para o Cucumber são os exemplos (**Scenários**) que explicam a funcionalidade. Cada um dos Scenários contém um conjunto de passos, que podem ser do tipo **Given** (pré-requisitos), **When** (ações), ou **Then** (resultado).

A implementação de cada um dos passos (*steps*) dos *scenarios* devem ficar dentro do diretório **step_definitions/**, na mesma pasta onde se encontram os arquivos *.feature*, texto puro.

O nome destes arquivos que contém a definição de cada um dos passos deve terminar com `_steps.rb`. Cada passo é representado na chamada dos métodos `Given`, `Then` ou `When`, que recebem como argumento uma **String** ou **expressão regular** batendo com o que estiver escrito no arquivo de texto puro (*.feature*).

Tipicamente, os projetos contém um diretório **features/**, com a seguinte estrutura:

```
projeto/  
|-- features/  
|   |-- minha.feature  
|   |-- step_definitions/  
|       |-- alguns_steps.rb  
|       |-- outros_steps.rb  
|   |-- support/  
|       |-- env.rb
```

O arquivo **support/env.rb** é especial do Cucumber e sempre é carregado antes da execução dos testes. Geralmente contém a configuração necessária para os testes serem executados e código de suporte aos testes, como preparação do Selenium ou Webrat.

Os arquivos com definições dos passos são arquivos Ruby:

```
Given "alguma condição descrita no arquivo texto puro" do  
  # código a ser executado para este passo  
end
```

```
Given /e outra condicao com valor: (.*)/ do |valor|  
  # código de teste para esse passo  
end
```

```
When /alguma acao/  
  # ...  
end
```

```
Then /verifica resultado/  
  # ...  
end
```

O código de teste para cada passo pode ser qualquer código Ruby. É comum o uso do RSpec para verificar condições (métodos `should`) e **Webrat** ou **Selenium** para controlar testes de aceitação. Mais detalhes sobre estes frameworks para testes de aceitação podem ser vistos no capítulo *"Outros testes e specs"*.

Não é necessário haver um arquivo com definição de passos para cada arquivo de feature texto puro. Isto é até considerado má prática por muitos, já que inibe o reuso para definições de *steps*.

Apêndice A - Integrando Java e Ruby

“Não há poder. Há um abuso do poder nada mais”

– Montherlant, Henri

Nesse capítulo você aprenderá a acessar código escrito anteriormente em Java através de scripts escritos em Ruby: o projeto JRuby.

16.1 O PROJETO

JRuby (<http://www.jruby.org/>) é uma implementação de um interpretador Ruby escrito totalmente em java, e mais ainda, com total integração com a Virtual Machine.

Além de ser open-source, ele disponibiliza a integração entre as bibliotecas das duas linguagens.

Atualmente há algumas limitações como, por exemplo, não é possível herdar de uma classe abstrata. O suporte a Ruby on Rails também não está completo.

Os líderes desse projeto open source já trabalharam na Sun, o que permitiu termos uma implementação muito rápida e de boa qualidade.

16.2 TESTANDO O JRUBY

Vamos criar um script que imprima um simples “Testando o JRuby” na tela do seu console:

```
print "Testando o JRuby!\n"
```


Pode parecer muito simples, mas a grande diferença é que agora quem estará realmente rodando é uma Virtual Machine Java! Não há mais a necessidade de instalar o ruby na máquina, apenas a JVM e algumas bibliotecas do JRuby! Isto pode ajudar muito na adoção da linguagem.

Agora se executarmos tanto com os comandos “ruby” ou “jruby” o resultado será o mesmo:

16.3 EXERCÍCIOS

1) Crie um arquivo chamado `testando.rb` que imprime na tela “Testando o JRuby!”:

a) Edite o arquivo: `testando.rb`.

b) Adicione o seguinte conteúdo:

```
print "Testando o JRuby!\n"
```

c) Rode o arquivo com o JRuby:

```
jruby testando.rb
```

16.4 COMPILANDO RUBY PARA .CLASS COM JRUBY

Existe a possibilidade de compilarmos o arquivo `.rb` para um `.class` através do JRuby. Para isso devemos utilizar o `jrubyc` (JRuby Compiler) de modo muito semelhante ao `javac`:

```
jrubyc <path do arquivo .rb>
```

Vamos criar um arquivo `ola_mundo_jruby.rb`:

```
# ola_mundo_jruby.rb  
puts 'Ola Mundo com JRuby!'
```

Agora vamos compilar esse arquivo:

```
jrubyc ola_mundo_jruby.rb
```

Após isso, o arquivo `ola_mundo_jruby.class` já foi criado na mesma pasta do arquivo `ola_mundo_jruby.rb` e nós podemos utilizá-lo a partir de outro arquivo `.rb` através do `require`, porém esse `.class` é diferente do que o `javac` cria a partir do `.java`, sendo assim é impossível rodá-lo direto na JVM como rodamos outra classe qualquer do java.

16.5 RODANDO O .CLASS DO RUBY NA JVM

Como foi dito anteriormente, não é possível executar diretamente na JVM um arquivo compilado pelo jruby, isso acontece pelas características dinâmicas do ruby que tornam necessário a utilização de um jar. Tal jar pode ser baixada no site do Jruby(<http://jruby.org/download>).

Com o .jar em mãos, é fácil executar um bytecode do jruby na JVM, simplesmente devemos utilizar a opção “-jar” da seguinte maneira:

```
java -jar <path do arquivo .jar> <path do arquivo .class>
```

Lembrando que é necessário que a extensão do arquivo(.class) esteja explícita.

Vamos copiar o arquivo .jar do jruby para a pasta onde o ola_mundo_jruby.class está e rodar o nosso olá mundo:

```
java -jar jruby.jar ola_mundo_jruby.class
```

Após isso veremos nosso “Ola Mundo com JRuby!”.

16.6 IMPORTANDO UM BYTECODE(.CLASS) CRIADO PELO JRUBY

Para importar um bytecode que foi criado a partir de um arquivo .rb utilizamos o conhecido require.

```
require '<path do bytecode>'
```

Obs.: Lembre-se de retirar a extensão do arquivo (.class), o certo seria fazer algo como:

```
require 'app/funcionario'
```

e não:

```
# desta maneira o arquivo não será encontrado  
require 'app/funcionario.class'
```

16.7 IMPORTANDO CLASSES DO JAVA PARA SUA APLICAÇÃO JRUBY

Para importar classes Java utilizamos o método java_import, porém devemos ter o cuidado de antes requerer a biblioteca que tem esse método.

Vamos criar uma classe Pessoa em Java, e importar ela para dentro do JRuby. Primeiramente criaremos a classe Pessoa no java:

```
// Pessoa.java
public class Pessoa {
    private String nome;

    public Pessoa(String meuNome) {
        this.nome = meuNome;
    }

    public void setNome(String novoNome) {
        this.nome = novoNome;
    }

    public String getNome(){
        return this.nome;
    }

    public void seMostra(){
        System.out.println(this.getNome());
    }
}
```

Agora vamos compilar o código fonte com o javac utilizando:

```
javac Pessoa.java
```

Teremos então o arquivo Pessoa.class. Vamos criar um arquivo testando_jruby.rb onde vamos testar essa classe:

```
# testando_jruby.rb
require 'java' # o java_import faz parte desta biblioteca
java_import 'Pessoa'

pessoa = Pessoa.new 'João'
pessoa.se_mostra
# Observe que o nome do método no código Java é
# seMostra, porém o JRuby faz um alias para
# todos os métodos passando-os de Camelcase para
# Underscore case.
# Obs.: o método seMostra ainda existe.

pessoa.nome = 'Carlos'
# Observe que ao criarmos um setter
# para o nome(setNome), o JRuby criou
# o método nome= automaticamente.
# Obs.: Os métodos setNome e set_nome
```

```
# continuam existindo.

puts pessoa.nome
# Observe que ao criarmos um getter
# para o nome(getNome), o JRuby criou
# o método nome automaticamente
# Obs.: Os métodos getNome e get_nome
# continuam existindo.
```

Ao executarmos o exemplo acima, teremos como saída:

```
João
Carlos
```

Lembrando que para executar este exemplo basta utilizar

```
jruby testando_jruby.rb
```

16.8 TESTANDO O JRUBY COM SWING

Agora vamos integrar nosso “Testando o JRuby” com um pouco de Java, criando uma janela. Instanciamos um objeto Java em JRuby usando a notação:

```
require 'java'

module Swing
  include_package 'java.awt'
  include_package 'javax.swing'
end

module AwtEvent
  include_package 'java.awt.event'
end

# Reparem que não é necessário herdar nem
# implementar nada, apenas definir o metodo
# com o nome que o java exige (duck typing)
class ListenerDoBotao
  def action_performed(evento)
    Swing::JOptionPane.showMessageDialog(
      nil, "ActionListener feito em ruby")
  end
end
```

```
frame = Swing::JFrame.new
painel = Swing::JPanel.new
frame.add painel

label = Swing::JLabel.new
# label.setText("Testando o JRuby!")
label.text = "Testando o JRuby!"
painel.add label

botao = Swing::JButton.new 'clique aqui'
botao.add_action_listener ListenerDoBotao.new
painel.add botao

frame.pack
frame.set_size(400, 400)
frame.visible = true
```

O `include_package` é parecido com um `import`, e depois estamos criando uma instância de `JFrame`. Dessa mesma maneira você pode acessar qualquer outra classe da biblioteca do Java. Assim você tem toda a expressividade e poder do Ruby, somado a quantidade enorme de bibliotecas do Java.

PARA SABER MAIS: SUPORTE A CLOSURE COM JRUBY

O JRuby permite a passagem de blocos de código como argumento para métodos do Java que recebem como parâmetro uma interface que define apenas um método, assim como o futuro suporte a closures prometido para o Java 8. No exemplo acima poderíamos ter passado o `ActionListener` para o botão sem necessidade de escrever uma classe só para isso, e nem mesmo seria preciso instanciar um objeto, fazendo desta forma:

```
botao.add_action_listener do |evento|
  Swing::JOptionPane.showMessageDialog(nil, "ActionListener em closure")
end
```

Apêndice B - Deployment

“Há noites que eu não posso dormir de remorso por tudo o que eu deixei de cometer.”

– Mario Quintana

Como construir ambientes de produção e deployment para aplicações Rails sempre foram alguns dos maiores desafios desta plataforma. Existem diversos detalhes a serem considerados e diversas opções disponíveis.

17.1 WEBRICK

A forma mais simples de executar aplicações rails é usar o servidor que vem embutido em todas estas aplicações: **Webrick**.

É um servidor web muito simples, escrito em Ruby, que pode ser iniciado através do arquivo **script/server**, dentro do projeto:

```
cd projetorails
rails server
```

Por padrão, o Webrick inicia na porta 3000, porém isto pode ser mudado com a opção `-p`:

```
rails server -p 3002
```

Por ser muito simples, **não é recomendado** o uso do webrick em produção.

17.2 CGI

Uma das primeiras alternativas de deployment para aplicações Rails foi o uso de servidores web famosos, como o Apache Httpd. Porém, como o httpd só serve conteúdo estático, precisar delegar as requisições dinâmicas para processos Ruby que rodam o Rails, através do protocolo CGI.

Durante muito tempo, esta foi inclusive uma das formas mais comuns de servir conteúdo dinâmico na internet, com linguagens como Perl, PHP, C, entre outras.

O grande problema no uso do CGI, é que o servidor Web inicia um novo processo Ruby a cada requisição que chega. Processos são recursos caros para o sistema operacional e iniciar um novo processo a cada requisição acaba limitando bastante o tempo de resposta das requisições.

17.3 FCGI - FASTCGI

Para resolver o principal problema do CGI, surgiu o **FastCGI**. A grande diferença é que os processos que tratam requisições dinâmicas (*workers*) são iniciados junto ao processo principal do servidor Web.

Desta forma, não é mais necessário iniciar um novo processo a cada requisição, pois já foram iniciados. Os processos ficam disponíveis para todas as requisições, e cada nova requisição que chega usa um dos processos existentes.

POOL DE PROCESSOS

O conjunto de processos disponíveis para tratar requisições dinâmicas também é popularmente conhecido como **pool** dos processos.

A implementação de FCGI para aplicações Rails, com o apache Httpd nunca foi satisfatória. Diversos bugs traziam muita instabilidade para as aplicações que optavam esta alternativa.

Infelizmente, FCGI nunca chegou a ser uma opção viável para aplicações Rails.

17.4 LIGHTTPD E LITESPEED

Implementações parecidas com Fast CGI para outros servidores Web pareceram ser a solução para o problema de colocar aplicações Rails em produção. Duas alternativas ficaram famosas.

Uma delas é a implementação de Fast CGI e/ou SCGI do servidor web **Lighttpd**. É um servidor web escrito em C, bastante performático e muito leve. Muitos reportaram problemas de instabilidade ao usar o Lighttpd em aplicações com grandes cargas de requisições.

Litespeed é uma outra boa alternativa, usado por aplicações Rails em produção até hoje. Usa o protocolo proprietário conhecido como LSAPI. Por ser um produto pago, não foi amplamente difundido dentro da

comunidade de desenvolvedores Rails.

<http://www.litespeedtech.com/ruby-lsapi-module.html>

17.5 MONGREL

Paralelamente às alternativas que usam FCGI (e variações) através de servidores Web existentes, surgiu uma alternativa feita em Ruby para rodar aplicações Rails.

Mongrel é um servidor web escrito por Zed Shaw, em Ruby. É bastante performático e foi feito especificamente para servir aplicações Rails. Por esses motivos, ele rapidamente se tornou a principal alternativa para deployment destas aplicações. Hoje suporta outros tipos de aplicações web em Ruby.

17.6 PROXIES REVERSOS

O problema com o Mongrel é que uma instância do Rails não pode servir mais de uma requisição ao mesmo tempo. Em outras palavras, o Rails não é thread-safe. Possui um lock que não permite a execução de seu código apenas por uma thread de cada vez.

Por causa disso, para cada requisição simultânea que precisamos tratar, é necessário um novo processo Mongrel. O problema é que cada Mongrel roda em uma porta diferente. Não podemos fazer os usuários terem de se preocupar em qual porta deverá ser feita a requisição.

Por isto, é comum adicionar um **balanceador de carga** na frente de todos os Mongrels. É o balanceador que recebe as requisições, geralmente na porta 80, e despacha para as instâncias de Mongrel.

Como todas as requisições passam pelo balanceador, ele pode manipular o conteúdo delas, por exemplo adicionando informações de cache nos cabeçalhos HTTP. Neste caso, quando faz mais do que apenas distribuir as requisições, o balanceador passa a ser conhecido como **Proxy Reverso**.

REVERSO?

Proxy é o nó de rede por onde passam todas as conexões que saem. O nome Proxy Reverso vem da ideia de que todas as conexões **que entram** passam por ele.

O principal ponto negativo no uso de vários Mongrels é o processo de deployment. A cada nova versão, precisaríamos instalar a aplicação em cada um dos Mongrels e reiniciar todos eles.

Para facilitar o controle (*start*, *stop*, *restart*) de vários Mongrels simultaneamente, existe o projeto **mongrel_cluster**.

17.7 PHUSION PASSENGER (MOD_RAILS)

Ninh Bui, Hongli Lai e Tinco Andringa da empresa Phusion decidiram tentar novamente criar um módulo para rodar aplicações Rails usando o Apache Httpd.

Phusion **Passenger**, também conhecido como **mod_rails**, é um módulo para o Apache Httpd que adiciona suporte a aplicações Web escritas em Ruby. Uma de suas grandes vantagens é usar o protocolo **Rack** para enviar as requisições a processos Ruby.

Como o **Rack** foi criado especificamente para projetos Web em Ruby, praticamente todos os frameworks web Ruby suportam este protocolo, incluindo o Ruby on Rails, o Merb e o Sinatra. Só por serem baseados no protocolo Rack, são suportados pelo **Passenger**.

A outra grande vantagem do **mod_rails** é a facilidade de deployment. Uma vez que o módulo esteja instalado no Apache Httpd, bastam três linhas de configuração no arquivo **httpd.conf**:

```
<VirtualHost *:80>
  ServerName www.aplicacao.com.br
  DocumentRoot /webapps/aplicacoes/projetorails
</VirtualHost>
```

A partir daí, fazer deployment da aplicação Rails consiste apenas em copiar o código para a pasta configurada no Apache Httpd. O **mod_rails** detecta que é uma aplicação Rails automaticamente e cuida do resto.

A documentação do Passenger é uma ótima referência:

<http://www.modrails.com/documentation/Users%20guide.html>

17.8 RUBY ENTERPRISE EDITION

Além do trabalho no **mod_rails**, os desenvolvedores da Phusion fizeram algumas modificações importantes no interpretador MRI. As mudanças podem trazer redução de até 30% no uso de memória em aplicações Rails.

O *patch* principalmente visa modificar um pouco o comportamento do Garbage Collector, fazendo com que ele não modifique o espaço de memória que guarda o código do Rails. Desta forma, os sistemas operacionais modernos conseguem usar o mesmo código Rails carregado na memória para todos os processos. Esta técnica é conhecida como Copy on Write; suportada pela maioria dos sistemas operacionais modernos.

Outra mudança importante promovida pelos desenvolvedores da Phusion foi o uso de uma nova biblioteca para alocação de memória, **tcmalloc**, no lugar da original do sistema operacional. Esta biblioteca é uma criação do Google.

17.9 EXERCÍCIOS: DEPLOY COM APACHE E PASSENGER

- 1) Abra o FileBrowser e copie o projeto **restaurantes** para o Desktop. o projeto está em /rr910/Aptana Studio Workspace/restaurantes

- 2) Abra o terminal e digite:

```
install-httpd
```

Feche e abra o terminal novamente Esse comando baixa httpd e compila na pasta /home/apache. No nosso caso=> /home/rr910/apache

- 3) Ainda no terminal entre no diretório do projeto e rode a migration para atualizar o banco em produção:

```
cd Desktop/restaurante
rake db:migrate:reset RAILS_ENV=production
```

- 4) Abra o arquivo de configuração do Apache:

```
gedit /home/rr910/apache/conf/httpd.conf
```

Altere a linha abaixo:

```
Listen 8080
```

Adicione a linha a baixo em qualquer lugar do arquivo:

```
Servername http://localhost:8080
```

- 5) Suba o Apache, no terminal rode:

```
apachectl start
```

Acesse <http://localhost:8080/> no browser e confira se o Apache subiu, deve aparecer a mensagem **It works!**.

- 6) vamos instalar o Passenger. no terminal rode:

```
gem install passenger
passenger-install-apache2-module
```

Quando o instalador surgir no terminal, pressione **enter**.

No fim, copie a instrução semelhante a essa:

```
LoadModule passenger_module
    /home/rr910/.gem/ruby/1.8/gems/passenger-2.2.5/ext/apache2/
    mod_passenger.so
PassengerRoot /home/rr910/.gem/ruby/1.8/gems/passenger-2.2.5

PassengerRuby /usr/bin/ruby1.8
```

- 7) Abra o arquivo de configuração do Apache:

```
gedit /home/rr910/apache/conf/httpd.conf
```

Adicione essas linhas ao final do arquivo:

```
<VirtualHost *:8080>
    DocumentRoot /home/rr910/Desktop/restaurante/public

</VirtualHost>

<Directory />
    Options FollowSymLinks
    AllowOverride None
    Order allow,deny
    Allow from all
</Directory>
```

8) No terminal, de o restart no apache:

```
apachectl restart
```

Acesse **<http://localhost:8080/restaurantes>**. Nossa aplicação está rodando no Apache com Passenger!

Índice Remissivo

ActiveRecord, 85
after_filter, 137
around_filter, 137

BDD, 191
before_filter, 137
Behavior Driven Development, 191
Boolean, 19
Builder, 136

Classes abertas, 27
comments, 11
Comparable, 45
Copy on Write, 208

Duck Typing, 57

Enumerable, 45
ERB, 122

for, 20

Gems, 5
Gemstone, 8
gets, 13

HAML, 136
hpricot, 184

I/O, 49
if / elsif / case / when, 19

Maglev, 8
MetaProgramming, 66
Migrations, 88
mongrel_cluster, 207
MSpec, 8

Observer, 62
open-uri, 184
Operadores Aritméticos e Números, 17
Operadores Booleanos, 19
OR, 21

ORM, 85

Paginação com kaminari, 180
Palavras Reservadas do Ruby, 12
puts, print e p, 11

rack, 139
Rake, 86
Ranges, 18
Regexp, 20
respond_to, 145
RSpec, 191
Ruby, 4
Ruby Enterprise Edition, 9

Search Engine Optimization, 142
Symbol, 18
Syntax Sugar, 30

tcmalloc, 208
Template Method, 59
Testes, 186